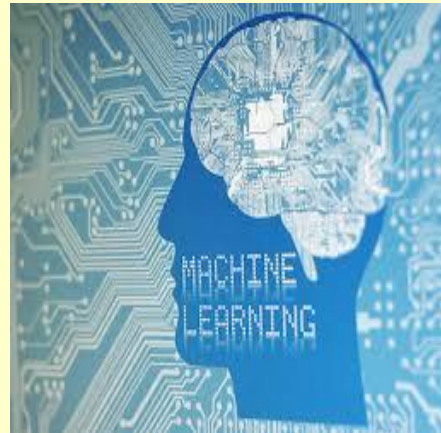


Introduction à Machine Learning sous python



Introduction à l'apprentissage automatique

Définition: Machine Learning (apprentissage automatique) concerne l'étude, la conception et le développement d'algorithmes qui permettent aux ordinateurs d'apprendre sans être explicitement programmés (Arthur Samuel).

Programmation classique



Machine Learning



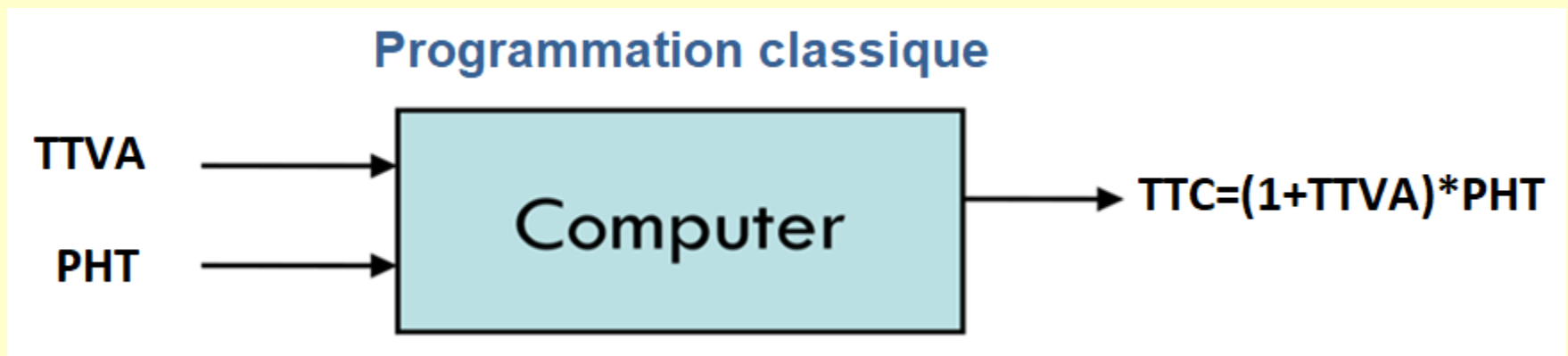
Introduction à l'apprentissage automatique

Programmation classique:

- Développement de programmes pour la gestion de stock
- Administration d'une entreprise,
- Gestion d'un cabinet de formations,

On peut trouver un algorithme qui permet de trouver les résultats à partir des données saisies par l'utilisateur.

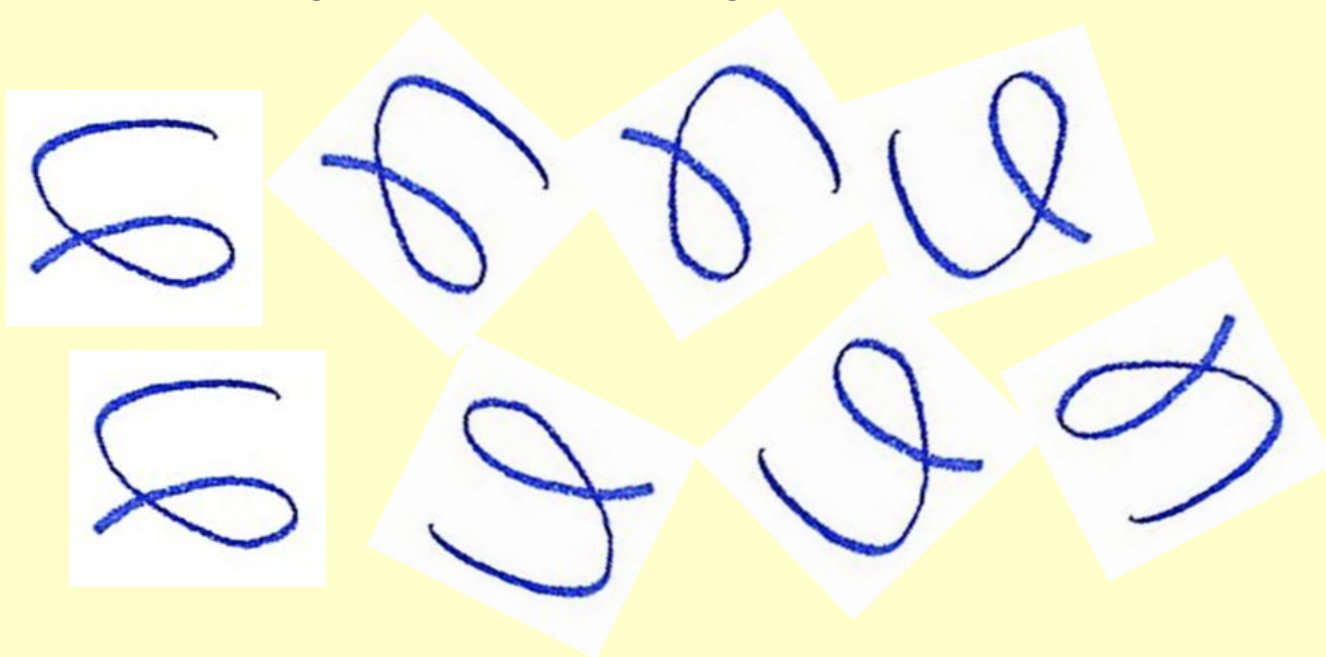
Exemple:



Introduction à l'apprentissage automatique

Machine Learning: On ne dispose pas d'une formule ou d'une méthode analytique qui permet de trouver les résultats à partir des données.

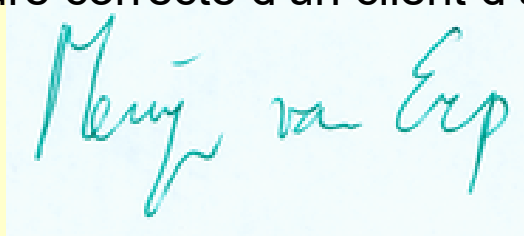
Connaissez vous une méthode analytique ou un algorithme précis qui permet de reconnaître l'une des images suivantes ? S'agit-il du chiffre 6 ou 9 ou la lettre v ?



Introduction à l'apprentissage automatique

Vérification de signature sur un chèque:

Connaissez vous une méthode analytique ou un algorithme précis qui permet de vérifier si l'image suivante (extraite de la partie à droite en bas d'un chèque) représente la signature correcte d'un client d'une banque ou non ?

A handwritten signature in blue ink on a light blue background. The signature is cursive and appears to read 'Mery va Exp'.

Machine learning pour la lutte contre covid-19:

ML pour prédire la propagation du virus, pour aider à diagnostiquer le virus, prédire la mortalité,...

Préparation d'un modèle de deep learning pour identifier le COVID-19 au scanner....

Quelques applications de Machine Learning

Exemples:

- **Prédiction des prix:** Estimer le prix d'une maison en fonction de sa superficie, sa localisation, possibilité de Parking ou non etc... Ces estimations sont faites en observant d'autres produits similaires pour en tirer des conclusions.
- **Diagnostic médical:** En se basant sur les données médicales d'un patient, l'algorithme peut diagnostiquer si le sujet est atteint d'une maladie donnée. Parfois, ces algorithmes peuvent alerter d'un incident grave de santé avant que cela n'arrive, notamment pour les crises cardiaques.

Quelques applications de Machine Learning

https://www.01net.com/actualites/ia-ces-chercheurs-ont-decouvert-un-puissant-antibiotique-grace-a-l-apprentissage-automatique-1862232.html?fbclid=IwAR0sdwCFrxJdMg2d75jAtGZ1EF_JH21ouRfr98v2hx2cEHpzYcH0R9QSTvM

Le 21 avril 2020, Des chercheurs du MIT (Massachusetts Institute of Technology) ont créé un algorithme capable de trouver automatiquement de nouveaux antibiotiques, ce qui devrait ravir l'industrie pharmaceutique en s'appuyant sur des techniques d'[apprentissage automatique](#).

Cette découverte est d'autant plus importante que les bactéries ont tendance à devenir de plus en plus résistantes. Cette nouvelle méthode fondée sur l'intelligence artificielle pourrait donc sauver les vies des millions de personnes.

Quelques applications de Machine Learning

- **Recommandation de produits:** Ce type de système se base sur les historiques d'achats, les recherches faites en ligne (Tracking Web) par un internaute pour lui recommander des produits qui pourront l'intéresser. Pour Amazon, cette fonctionnalité est critique car elle est au cœur de l'augmentation des volumes de vente et par conséquent des gains de la société.
- **Regroupement d'items:** Ce type de technique sert notamment pour l'application Iphoto d'Apple pour regrouper les images en fonction des gens qui s'y retrouvent. Généralement, les données n'ont pas d'étiquettes, et l'algorithme tentera de retrouver des items similaires et les regroupera dans un même groupe.

Quelques applications de Machine Learning

- **Conduite autonome:** En apprenant le comportement de conduite des humains, les algorithmes de Deep Learning avec l'apprentissage par renforcement (reinforcement learning) permettent d'apprendre des tâches complexes comme la conduite.
- **Banque et Assurance:**
 - Prédire qu'un client va quitter sa banque.
 - Définir ceux qui veulent changer leur assurance vie.
 - Améliorer la satisfaction client en proposant les offres les plus pertinentes possibles.
 - Prise de décision pour donner un crédit à un client ou non, prédire les risques,

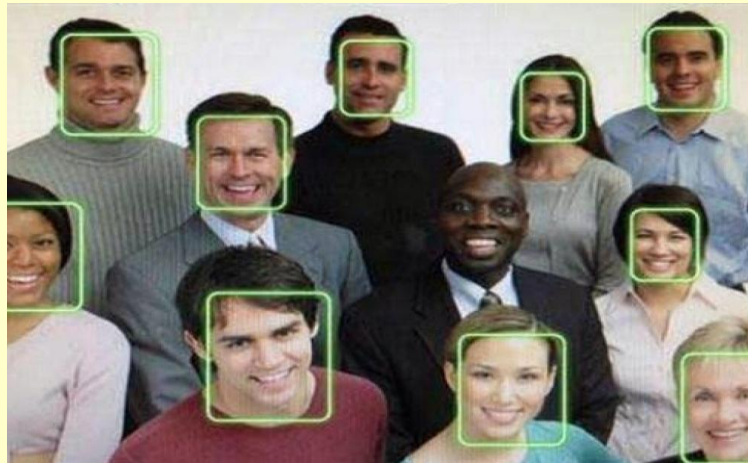
Quelques applications de Machine Learning

Une banque reçoit quotidiennement plusieurs demandes d'approbation de crédit. Elle veut automatiser leurs processus d'évaluation. La banque n'a pas de formule magique pour savoir quand il faut accorder un crédit, mais il a beaucoup de données.

L'existence de données nous ramène à l'approche d'apprentissage des données. Donc, la banque utilise l'historique des documents des anciens clients pour trouver la bonne formule d'approbation de crédit.

Quelques applications de Machine Learning

La reconnaissance faciale: une catégorie de logiciels biométriques. Le logiciel utilise des algorithmes de deep learning pour comparer une capture en direct ou une image numérique à l'empreinte stockée afin de vérifier l'identité d'un individu. Plusieurs techniques de reconnaissance faciale sont implémentées.



Peut être utilisée pour le contrôle d'absence des étudiants ou d'employés d'une entreprise...

Quelques applications de Machine Learning

La détection d'émotions dans le texte ou dans l'image est un usage classique du **machine learning** qu'on désigne parfois par le terme de « **sentiment analysis** ».



Peut être utilisé pour contrôler le comportement des employés en contact direct avec les clients...

Machine Learning pour quels problèmes?

1- Existence d'un modèle à apprendre: Il existe une corrélation entre les variables d'entrées et de sorties. On sait qu'un modèle existe même si on le connaît pas.

2-Modélisation mathématique est impossible: On ne peut pas résoudre le modèle mathématiquement (pas de solution analytique).

3- Existence des données: Il existe des données qui représentent le modèle.

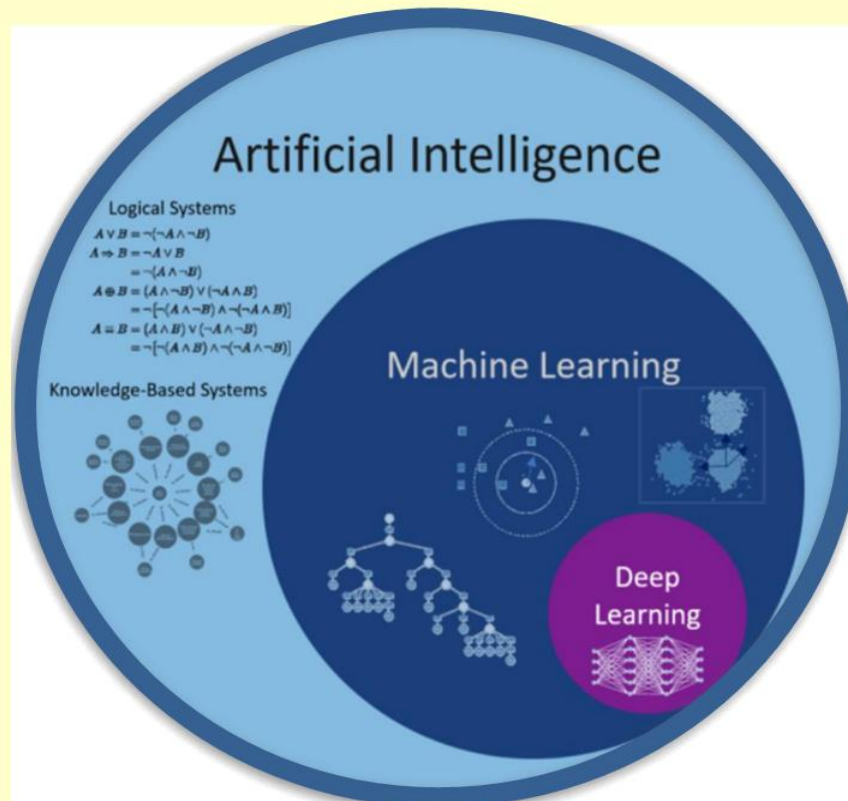
Datasets: exemples

<https://archive.ics.uci.edu/ml/index.php>

<https://www.kaggle.com/datasets>

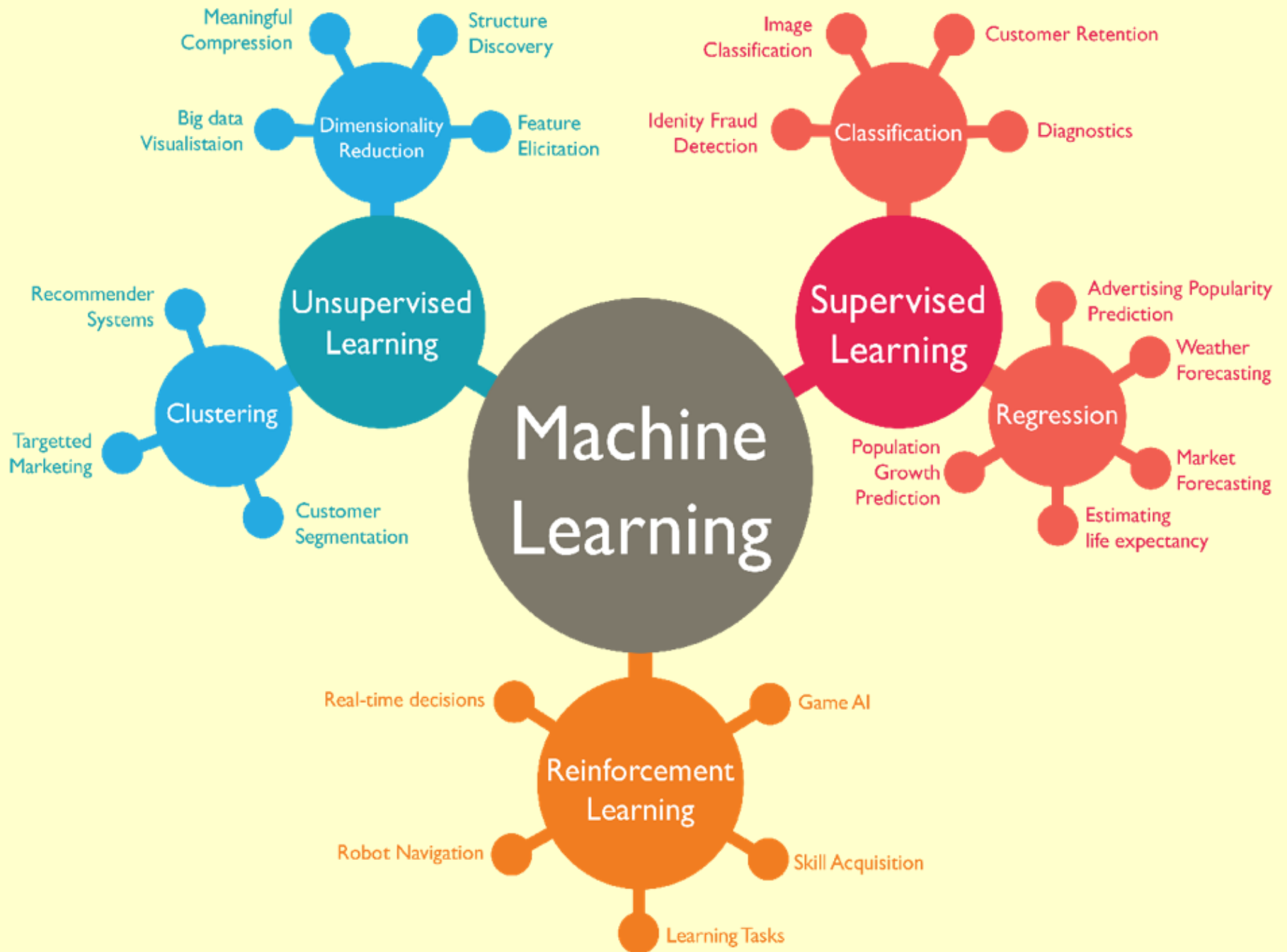
L'apprentissage automatique est une branche de l'intelligence artificielle

Le **Machine Learning** ou **Apprentissage automatique** est une branche de l'intelligence artificielle qui concerne la conception, l'analyse, le développement et l'implémentation de modèles, permettant à une machine d'apprendre à partir des données par un processus systématique afin de remplir une tâche.



Les types d'apprentissage automatique

- Apprentissage supervisé (supervised learning)
- Non supervisé (unsupervised learning)
- Semi supervisé (semi-supervised learning)
- Par renforcement (reinforcement learning)



Apprentissage supervisé (supervised learning)

Des données (entrées) annotées de leurs sorties pour entraîner le modèle, c'est-à-dire que à chaque un **entrée** **est** associée à une classe **cible (sortie)**, une fois entraîné, le modèle (l'algorithme de ML) devient capable de prédire (éventuellement avec un pourcentage d'erreur) la cible sur de **nouvelles données** non annotées.

Exemple:

Les données d'entrée représentent des images et la cible (ou *target* en anglais) représente la catégorie de photos.

Apprentissage supervisé (supervised learning)

Voiture



Oiseau



Chat



Chien



cheval



Apprentissage supervisé (supervised learning)



Apprentissage supervisé (supervised learning)

Techniques supervisées (prédictive)

Classification

**Régression
Estimation**

Arbre de décision

Réseaux de neurone

Naïve Bayes

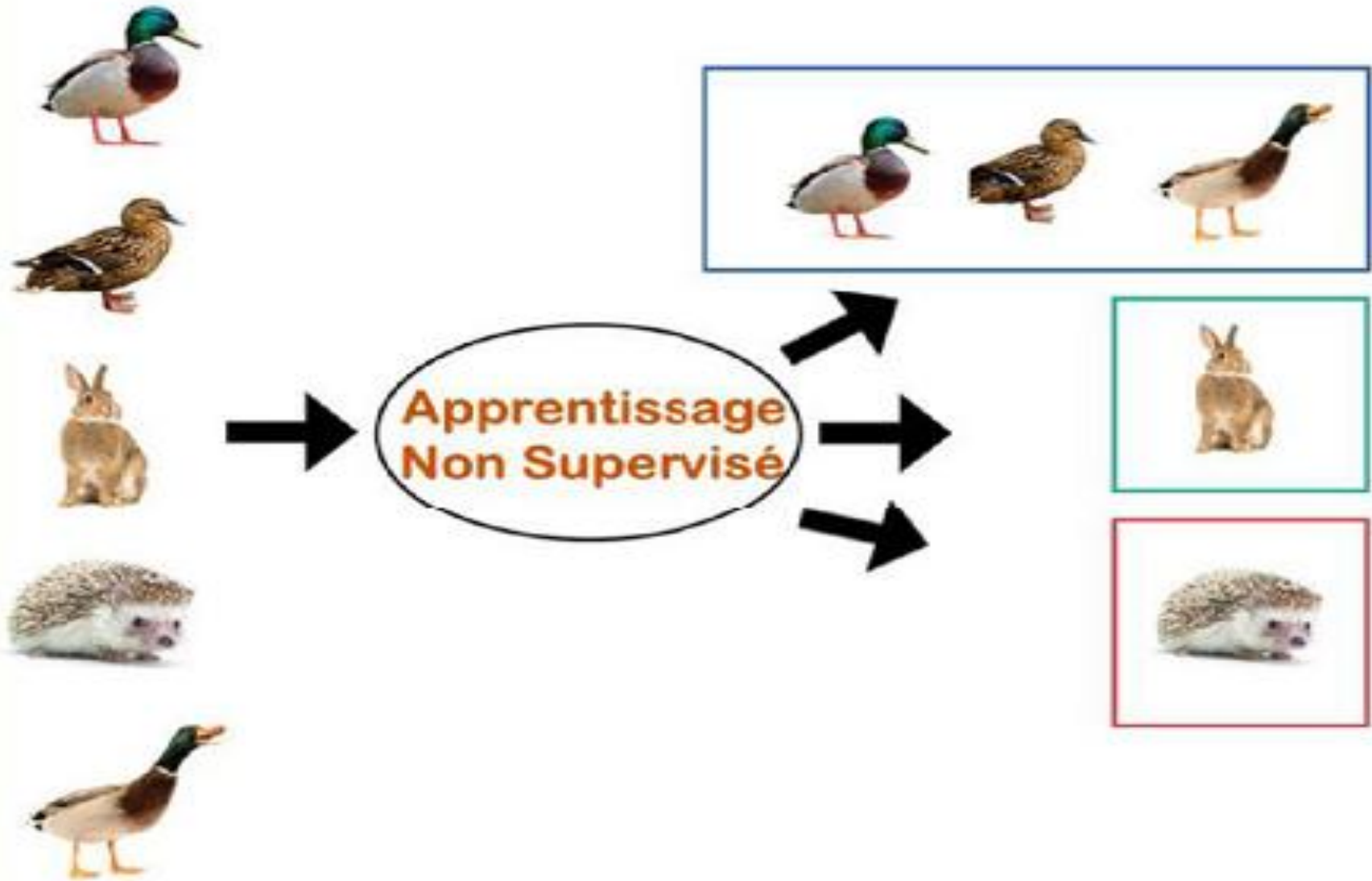
**Machines à
vecteurs supports
(SVM)**

K-Nearest Neighbour (K-NN)

Apprentissage non supervisé (unsupervised learning)

En apprentissage non supervisé, les données d'entrées ne sont pas annotées. L'algorithme d'entraînement s'applique dans ce cas à trouver les similarités et distinctions au sein de ces données, et à regrouper ensemble celles qui partagent des caractéristiques communes. Dans notre exemple, les photos similaires seraient ainsi regroupées automatiquement au sein d'une même catégorie.

Apprentissage non supervisé (unsupervised learning)



Apprentissage non supervisé (unsupervised learning)

Techniques non supervisées (descriptive)

Clustering

Associations

K-Means
K-moyennes

Apprentissage semi-supervisé (semi-supervised learning)

- L'**apprentissage semi-supervisé** est une classe de techniques d'apprentissage automatique qui utilise un ensemble de données étiquetées et non étiquetées. Il se situe ainsi entre l'apprentissage supervisé qui n'utilise que des données étiquetées et l'apprentissage non supervisé qui n'utilise que des données non étiquetées. Il a été démontré que l'utilisation de données non étiquetées, en combinaison avec des données étiquetées, permet d'améliorer significativement la qualité de l'apprentissage.
- Un autre intérêt provient du fait que l'étiquetage de données nécessite souvent l'intervention d'un utilisateur humain. Lorsque les jeux de données deviennent très grands, cette opération peut s'avérer fastidieuse. Dans ce cas, l'apprentissage semi-supervisé, qui ne nécessite que quelques étiquettes, revêt un intérêt pratique évident.

Apprentissage par renforcement(reinforcement learning)

L'apprentissage par renforcement consiste, pour un agent autonome (robot, etc.), à apprendre les actions à prendre, à partir d'expériences, de façon à optimiser une récompense quantitative au cours du temps. L'agent est plongé au sein d'un environnement, et prend ses décisions en fonction de son état courant. En retour, l'environnement procure à l'agent une récompense, qui peut être positive ou négative. L'agent cherche, au travers d'expériences itérées, un comportement décisionnel (appelé *stratégie* ou *politique*, et qui est une fonction associant à l'état courant l'action à exécuter) optimal, en ce sens qu'il maximise la somme des récompenses au cours du temps.

Les 7 étapes de l'apprentissage automatique

L'apprentissage automatique ne se résume pas à un ensemble d'algorithmes mais suit une succession d'étapes:

1) L'acquisition de données : l'algorithme se nourrissant des données en entrée, c'est une étape importante. Il en va de la réussite du projet, de récolter des données pertinentes et en quantité suffisante.

2) La préparation et le nettoyage de la donnée : les données recueillies doivent être retouchées avant utilisation. En effet, certains attributs sont inutiles, d'autre doivent être modifiés afin d'être compris par l'algorithme, et certains éléments sont inutilisables car leurs données sont incomplètes.

Les 7 étapes de l'apprentissage automatique

3) **Choix du modèle.**

4) **Formation et apprentissage du modèle**

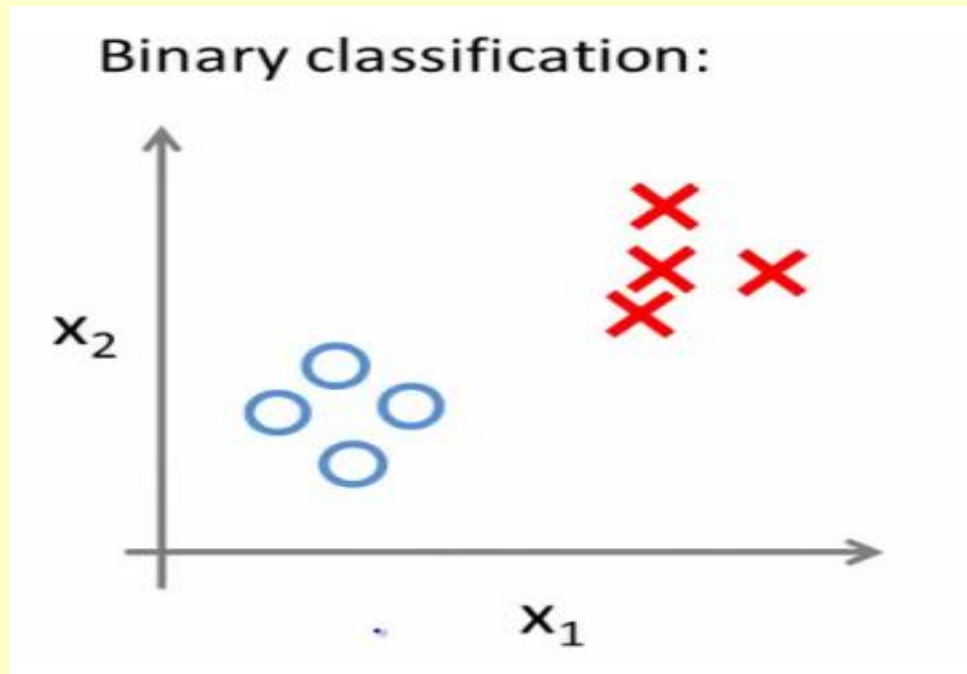
5) **L'évaluation** : une fois l'algorithme d'apprentissage automatique entraîné sur un premier jeu de donnée, on l'évalue sur un deuxième ensemble de données afin de vérifier que le modèle ne fasse pas de surapprentissage.

6) **Réglage des paramètres**: Ajuster les paramètres pour de meilleures performances

7) **Le déploiement** : le modèle est déployé en production pour faire des prédictions, et potentiellement utiliser les nouvelles données en entrée pour se ré-entraîner et être amélioré.

Classification binaire

- La classification est une tâche très répandue en Machine Learning. Dans ce genre de problématique, on cherche à mettre une étiquette (un label) sur une observation : une tumeur est-elle maligne ou non maligne, une transaction est-elle frauduleuse ou non frauduleuse ... ces deux cas sont des exemples de classification.
- Quand on a deux choix d'étiquettes possibles (tumeur maligne ou non), on parle de Binary Classification (classification binaire). Par ailleurs, l'étiquette Y aura deux valeurs possibles 0 ou 1. En d'autres termes $Y \in \{0, 1\}$

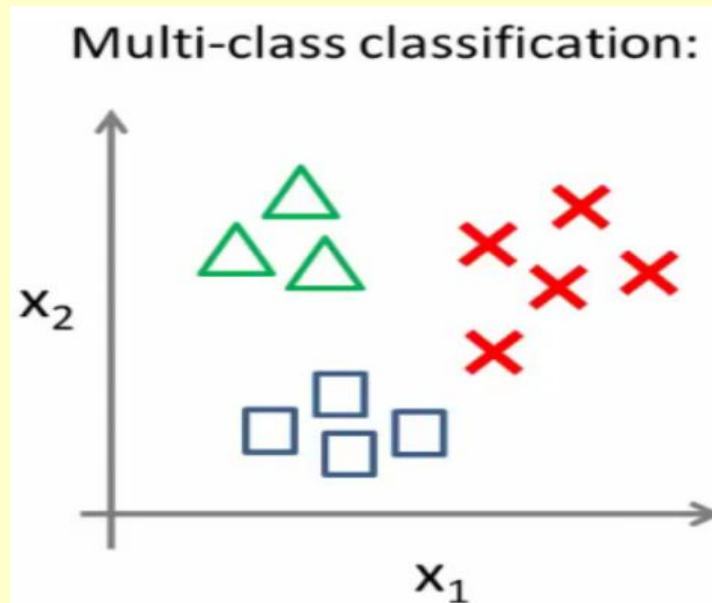


Classification binaire et multiclass

Le but du jeu c'est qu'on trouve une ligne (Boundary Decision) séparant les deux groupes (les cercles et les carrés).

Note : Pour l'exemple de la tumeur, on peut attribuer arbitrairement la classe (étiquette) 1 pour dire qu'il s'agit d'une tumeur maligne et la valeur 0 pour les tumeurs bénignes.

Quand notre problème a plusieurs étiquettes possibles (par exemple classifier un article dans une catégorie ("sport", "politique", "High-Tech")...), on parle de Multi-class classification. Dans ce cas on peut attribuer arbitrairement les numéros des classes aux observations du Training Set. Dans l'exemple vu en TP en haut pour la reconnaissance des chiffres : dix classes: 0,1,2,3,4,5,6,7,8,9

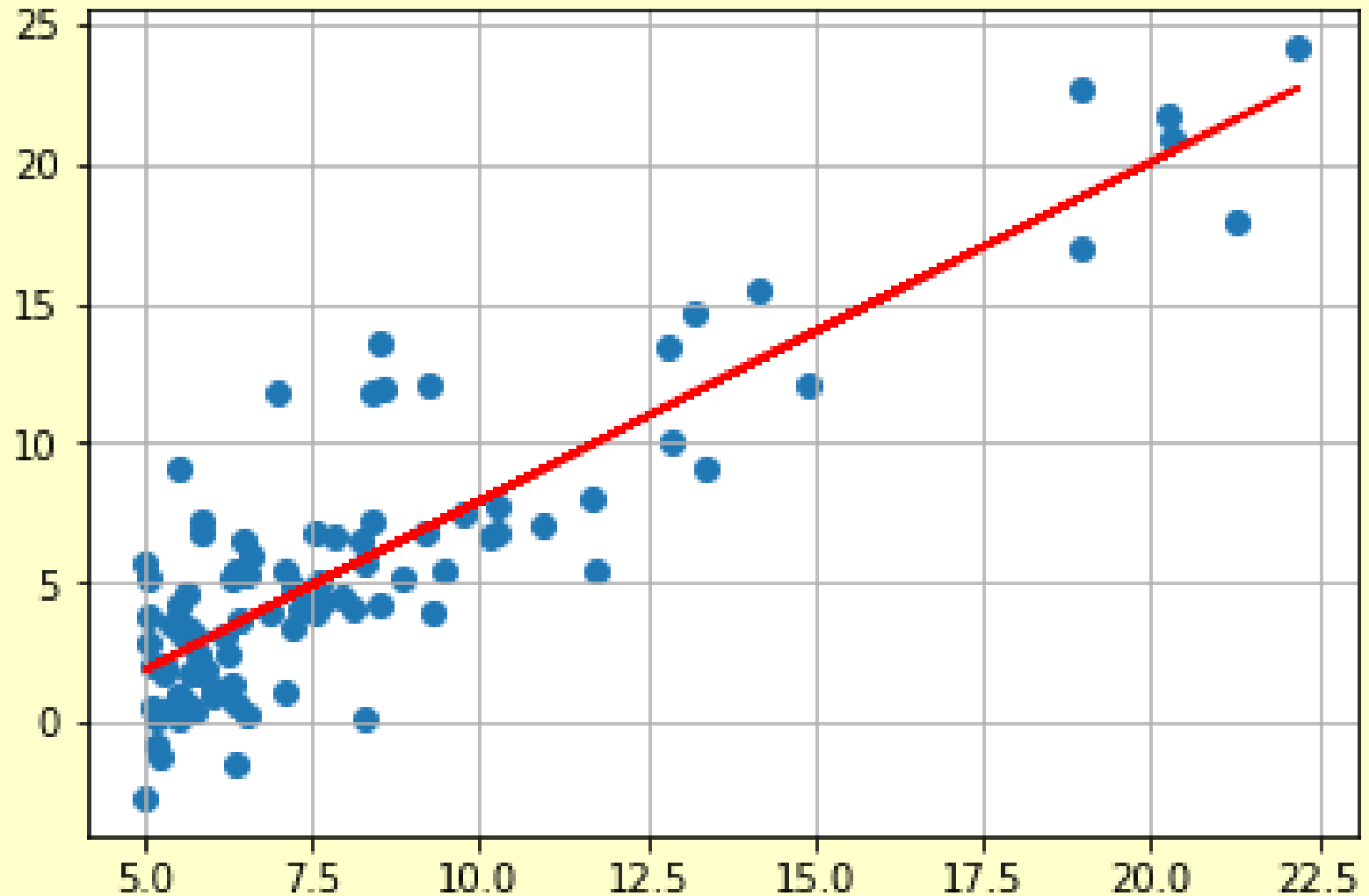


Régression linéaire

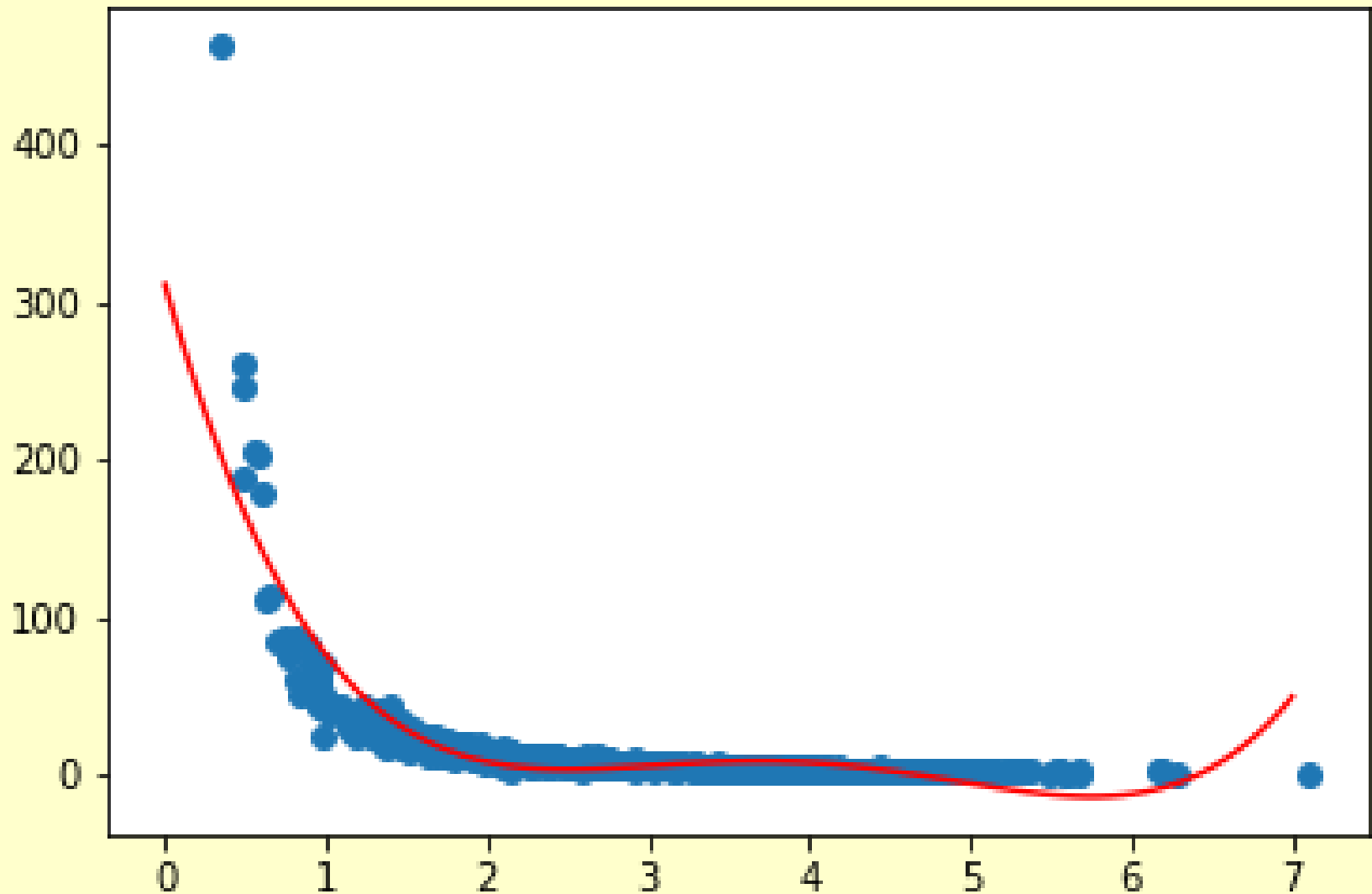
Les algorithmes de régression linéaire modélisent la relation entre des variables prédictives et une variable cible. La relation est modélisée par une fonction de prédiction. Le cas le plus simple est la régression linéaire univariée. Elle va trouver une fonction sous forme de droite pour estimer la relation.

La régression polynomiale permet de modéliser des relations complexes qui ne sont pas forcément linéaires.

Relation linéaire entre des variables prédictives et la variable cible



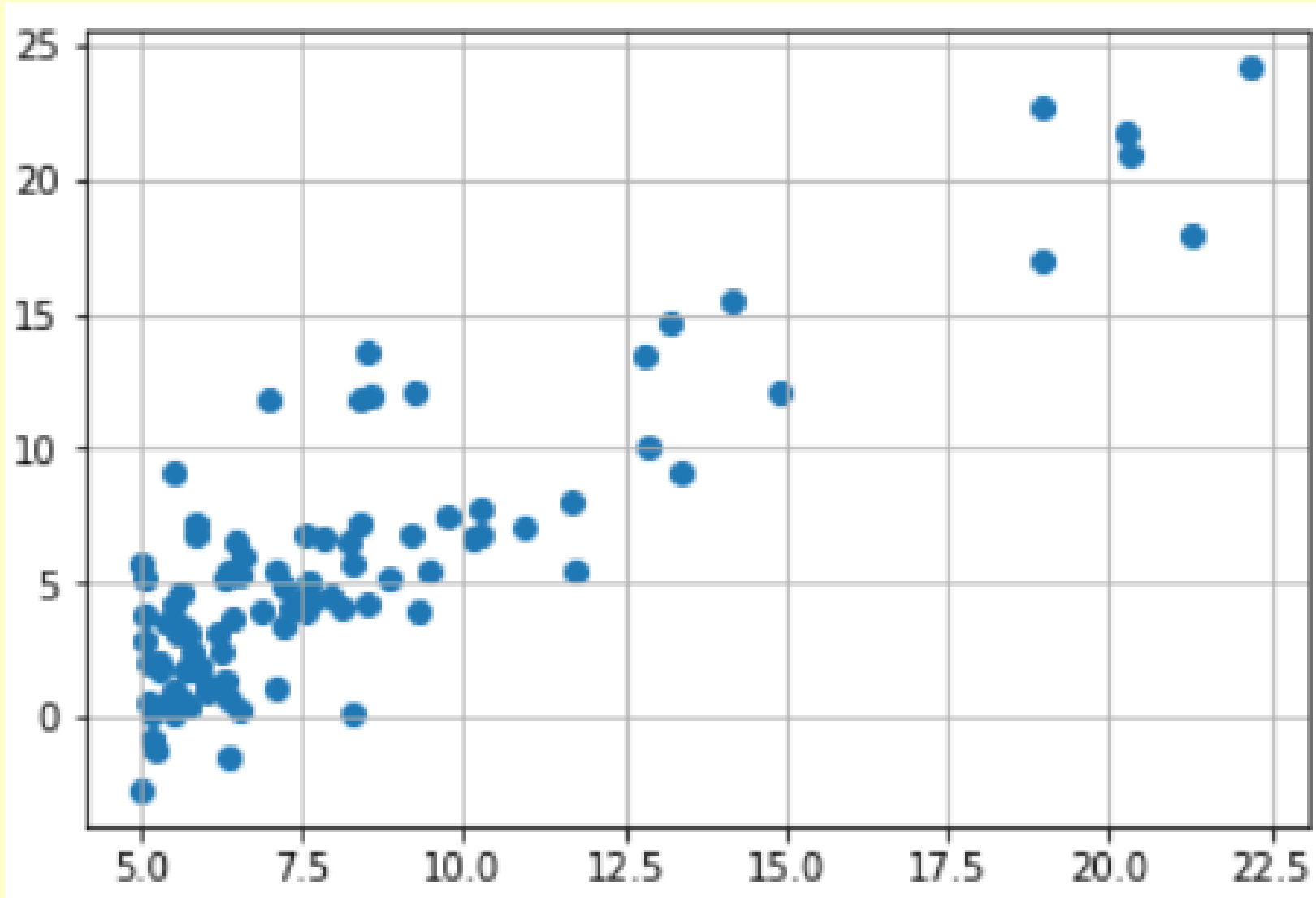
Relation non linéaire entre des variables prédictives et la variable cible



Régression linéaire par stats.linregress

```
1 import numpy as np; import pandas as pa
2 import matplotlib.pyplot as plt
3 %matplotlib inline
4 from scipy import stats; import os
5 chemin="E:\mes_donnees_2025_2026\ML"; os.chdir(chemin)
6 df = pa.read_csv("univariate_linear_regression_dataset.csv",header=0)
7 data=np.array(df); X =data[:,0] ; Y =data[:,1]
8 #X=np.arange(0,10,0.01); Y=np.random.randint(0,100,Len(X))
9 axes = plt.axes()
10 axes.grid()
11 plt.scatter(X,Y)
12 plt.show()
13 SL=stats.linregress(X, Y)
14 a=SL.slope; b=SL.intercept; coef_correlation=SL.rvalue
15 print(a,b) # slope c'est le a dans Y=aX+b et b=intercept
16 print("Equation:Y(X)="+str(a)+'*X'+str(b))
17
18 def predict(x):return a*x+b
19
20 axes = plt.axes(); axes.grid(); plt.scatter(X,Y)
21 LesYobtenuesParPredictionStatsLinregress = predict(X)
22 plt.plot(X,LesYobtenuesParPredictionStatsLinregress, c='r')
23 plt.show()
```

Régression linéaire par `stats.linregress`



Régression linéaire par `stats.linregress`

- Le fonction de prédiction pour une régression linéaire univariée est comme suit :

$$H(x)=\textit{intercept}+\textit{slope}*x$$

- avec : *slope*: représente la "pente" de la line de prédiction et *intercept* représente le point d'intersection avec l'axe des ordonnées

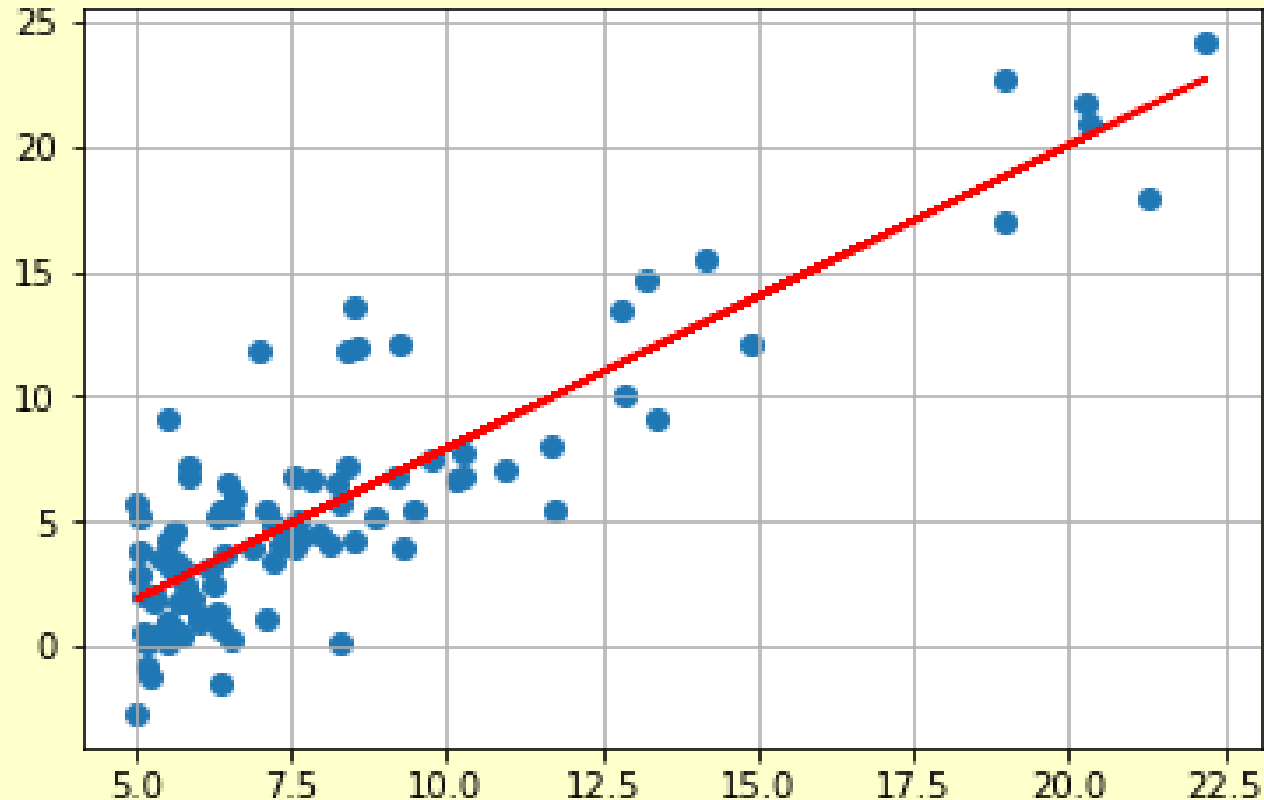
```
SL= stats.linregress(X, Y)
```

```
SL.slope est 1.2135472539083585
```

```
SL.intercept est -4.211504005424089
```

```
SL.rvalue:
```

```
0.8721572919685905
```



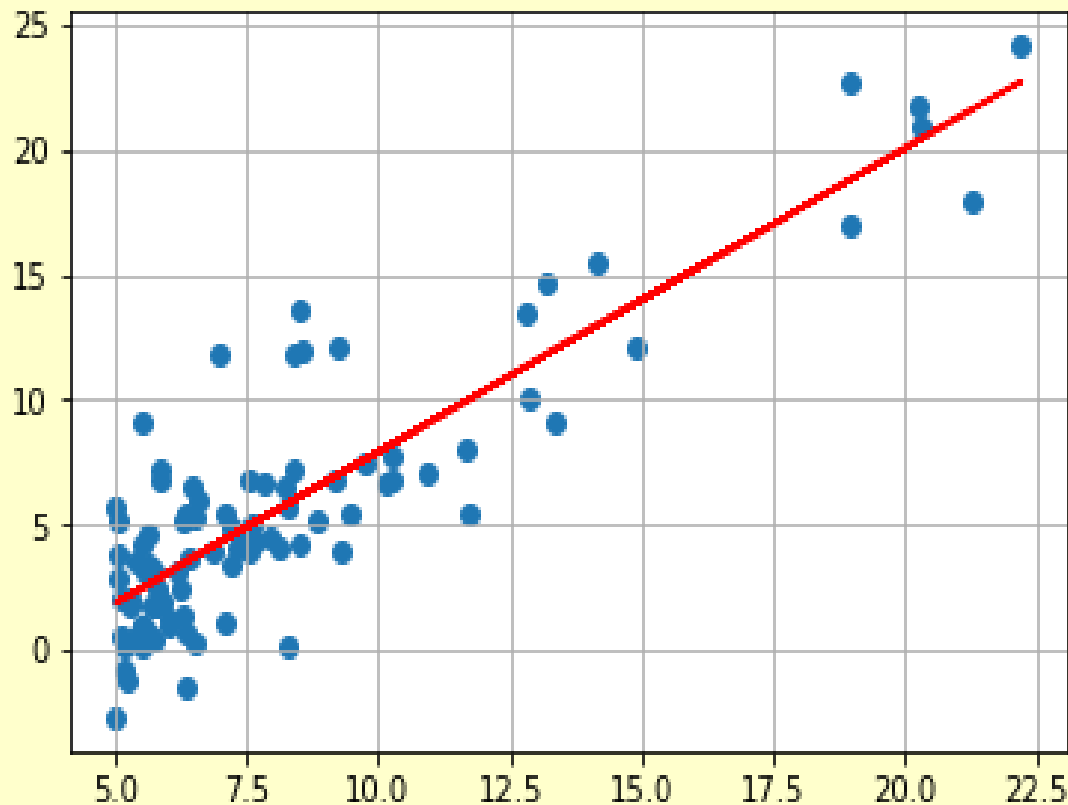
Régression linéaire par `stats.linregress`

- **Prédiction d'une nouvelle observation:**

On voit que pour la valeur $x = 22.5$, la valeur de y est environ 25.

Utilisons la fonction *predict* pour trouver une estimation de $H(x=22.5)$:

`predict(22.5)` → 23.093309207513975



Régression linéaire avec LinearRegressor

```
1 import numpy as np; import matplotlib.pyplot as plt
2 from sklearn.model_selection import train_test_split
3 from sklearn.linear_model import LinearRegression
4 from sklearn.metrics import mean_squared_error
5 import pandas as pa; import os
6 %matplotlib inline
7 chemin="E:\mes_donnees_2025_2026\ML"; os.chdir(chemin)
8 df = pa.read_csv("univariate_linear_regression_dataset.csv",header=0)
9 data=np.array(df); X = data[:,0]; Y = data[:,1]
10 XX=[[e]for e in X]; XX=np.array(XX)
11 YY=[[e]for e in Y]; YY=np.array(YY)
12 X_train, X_test, y_train, y_test = train_test_split(XX, YY, test_size=0.2, random_state=42)
13 model = LinearRegression()
14 model.fit(X_train, y_train)
15 y_pred = model.predict(X_test)
16 print("Coefficient (pente) :", model.coef_[0][0])
17 print("Intercept :", model.intercept_[0])
18 mse = mean_squared_error(y_test, y_pred)
19 print("Erreur quadratique moyenne (MSE) :", mse)
20 plt.scatter(X_test, y_test, color='black', label='Données réelles')
21 plt.plot(X_test, y_pred, color='blue', linewidth=3, label='Régression linéaire')
22 plt.xlabel('Variable indépendante'); plt.ylabel('Variable dépendante')
23 plt.legend(); plt.show()
```

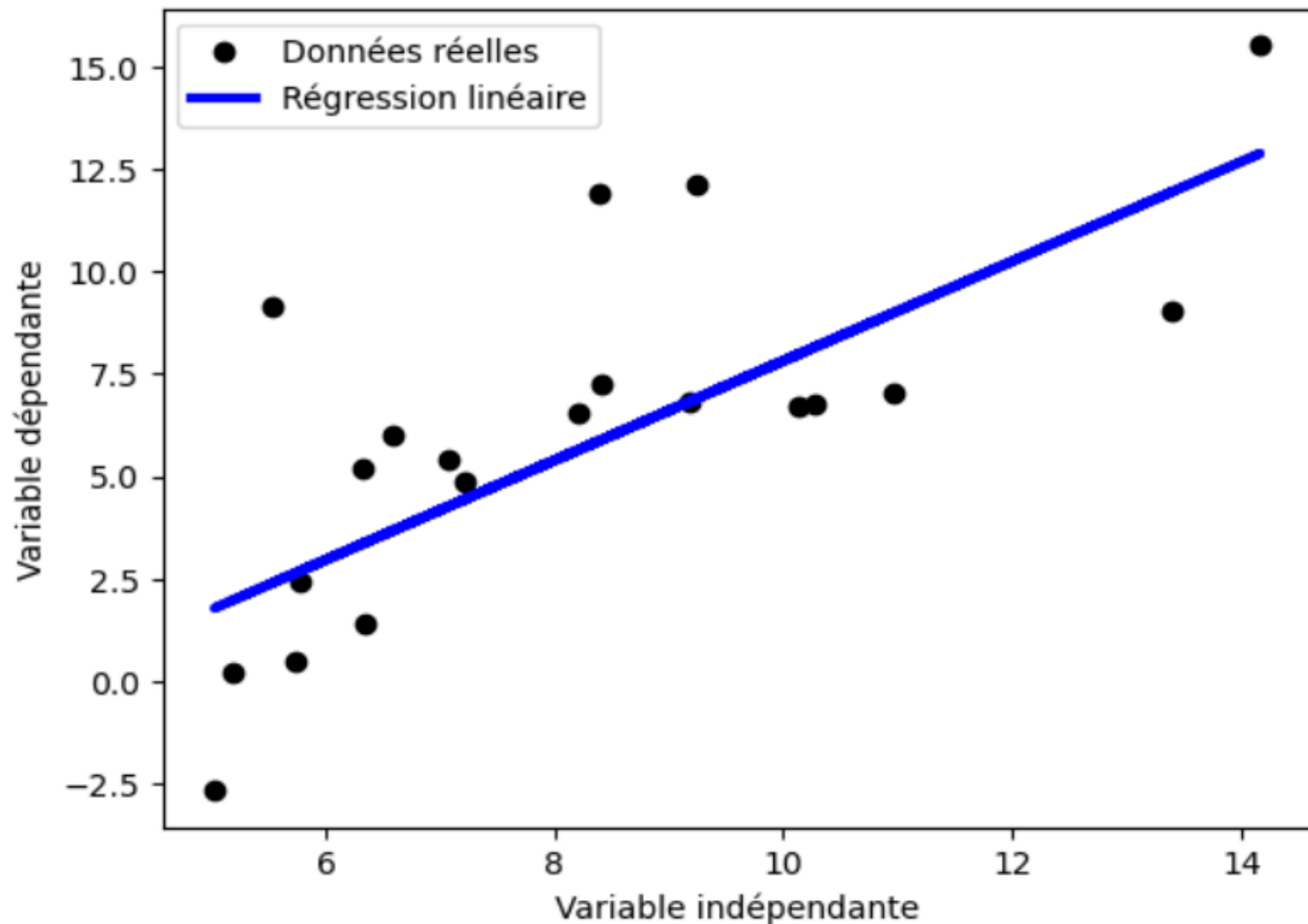
$$\underbrace{\text{MES}(\text{node})}_{\text{mean-squared-error}} = \frac{1}{N_{\text{node}}} \sum_{i \in \text{node}} (y^{(i)} - \hat{y}_{\text{node}})^2$$

Régression linéaire avec LinearRegressor

Coefficient (pente) : 1.2141408973281473

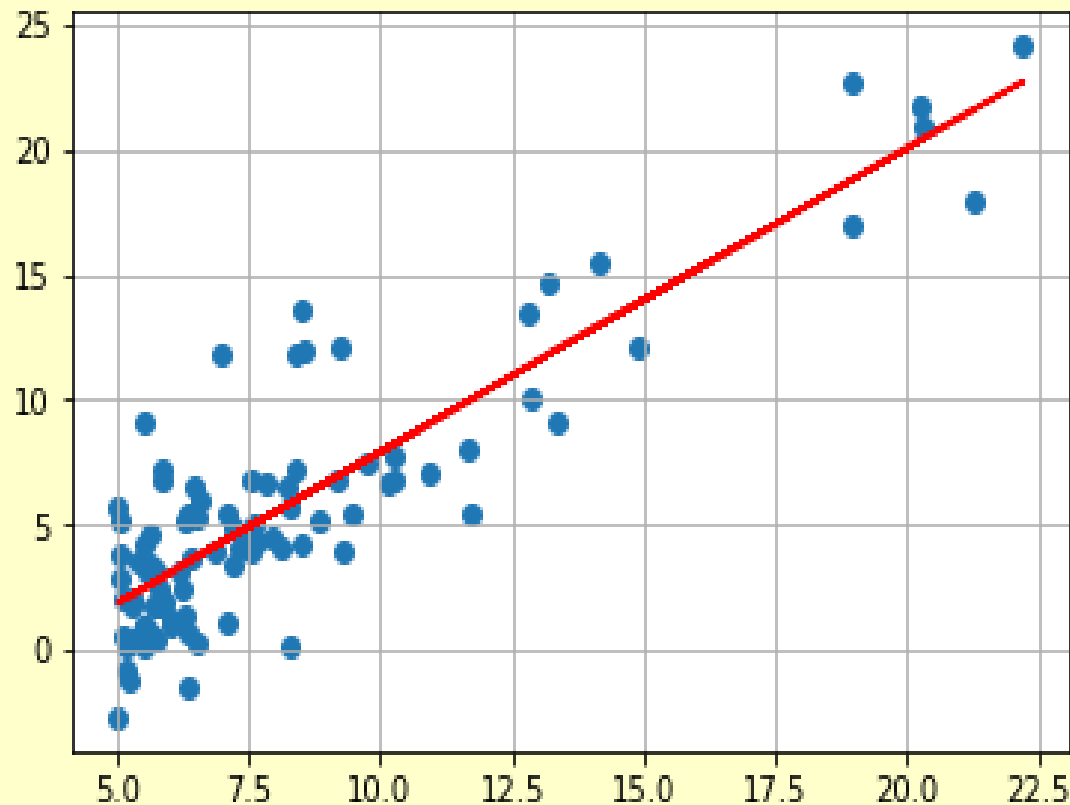
Intercept : -4.3260320338656015

Erreur quadratique moyenne (MSE) : 8.808279684441008



Régression linéaire avec l'algorithme de descente de gradient

Dans la pratique, les Data Scientists utilisent le package **sklearn**, qui permet d'écrire un tel code en 4 lignes, mais ici nous écrirons chaque fonction mathématique de façon **explicite**, ce qui est un **bon exercice** pour la compréhension du Machine Learning.



Régression linéaire avec l'algorithme de descente de gradient

$$X = (x \ 1) \ ; \ \theta = \begin{pmatrix} a \\ b \end{pmatrix} \text{ et } f(x) = \theta.X = (x \ 1) \begin{pmatrix} a \\ b \end{pmatrix} = a.x + b$$

Objectif: trouver θ qui donne une bonne prédiction d'une entrée x .

Pour développer un modèle linéaire avec la descente de gradient, il faut implémenter les 4 fonctions clefs suivantes :

- la fonction de notre modèle : $f(x) = X.\theta$
- la fonction Cout : $J(\theta) = \frac{1}{2m} \sum (X.\theta - Y)^2$
- le gradient : $\frac{\partial J(\theta)}{\partial \theta} = \frac{1}{m} X^T . (X.\theta - Y)$
- la descente de gradient : $\theta = \theta - \alpha \times \frac{\partial J(\theta)}{\partial \theta}$

α : est appelé taux d'apprentissage: learning_rate

m : est la taille de l'ensemble d'apprentissage

Objectif: trouver θ qui minimise $J(\theta)$

Régression linéaire avec l'algorithme de descente de gradient

- la fonction de notre modèle : $f(x) = X.\theta$
- la fonction Cout : $J(\theta) = \frac{1}{2m} \sum (X.\theta - Y)^2$
- le gradient : $\frac{\partial J(\theta)}{\partial \theta} = \frac{1}{m} X^T . (X.\theta - Y)$
- la descente de gradient : $\theta = \theta - \alpha \times \frac{\partial J(\theta)}{\partial \theta}$

```
def f(X, theta):  
    return X.dot(theta)  
def erreur_somme_des_distances(X, y, theta):  
    m = len(y)  
    return (1/(2*m))*np.sum((f(X, theta)-y)**2)  
def gradient(X, y, theta):  
    m = len(y)  
    return 1/m * X.T.dot(f(X, theta) - y)
```

Régression linéaire avec l'algorithme de descente de gradient

- la fonction de notre modèle : $f(x) = X.\theta$
- la fonction Cout : $J(\theta) = \frac{1}{2m} \sum (X.\theta - Y)^2$
- le gradient : $\frac{\partial J(\theta)}{\partial \theta} = \frac{1}{m} X^T . (X.\theta - Y)$
- la descente de gradient : $\theta = \theta - \alpha \times \frac{\partial J(\theta)}{\partial \theta}$

```
def gradient_descent(X, y, theta, learning_rate, n_iterations):  
    # création d'un tableau de stockage pour enregistrer l'évolution des er  
    historique_des_erreurs = np.zeros(n_iterations)  
    for i in range(0, n_iterations):  
        theta = theta - learning_rate * gradient(X, y, theta) # mise a jour  
        historique_des_erreurs[i] = erreur_somme_des_distances(X, y, theta)  
    return theta, historique_des_erreurs
```

Régression linéaire avec l'algorithme de descente de gradient

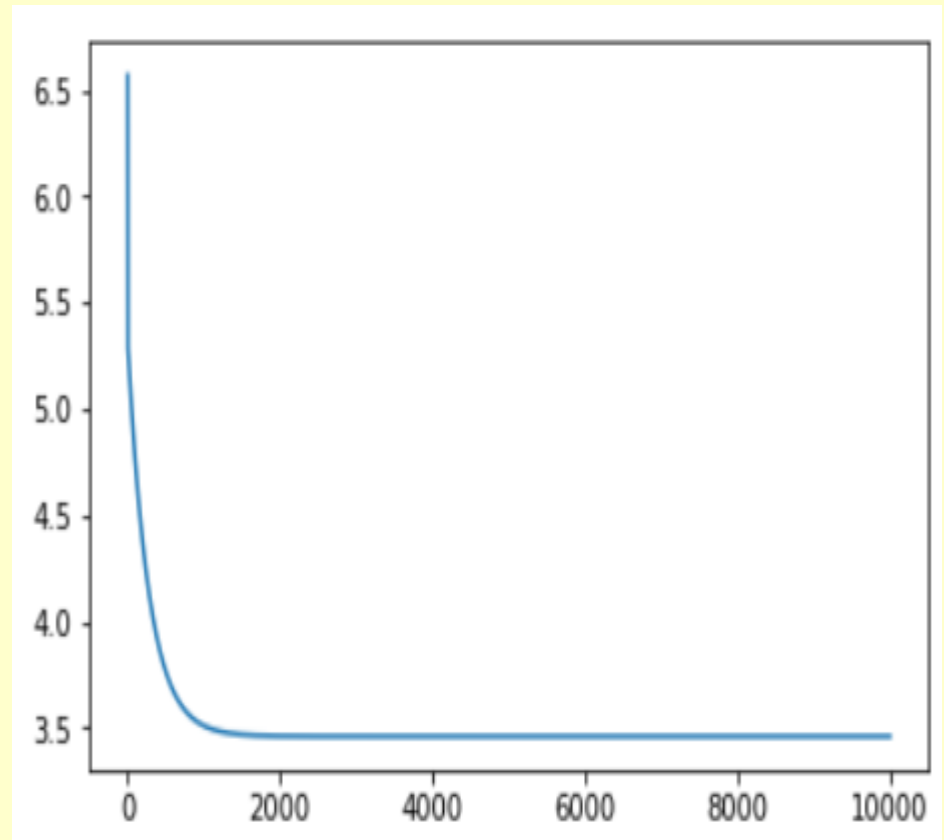
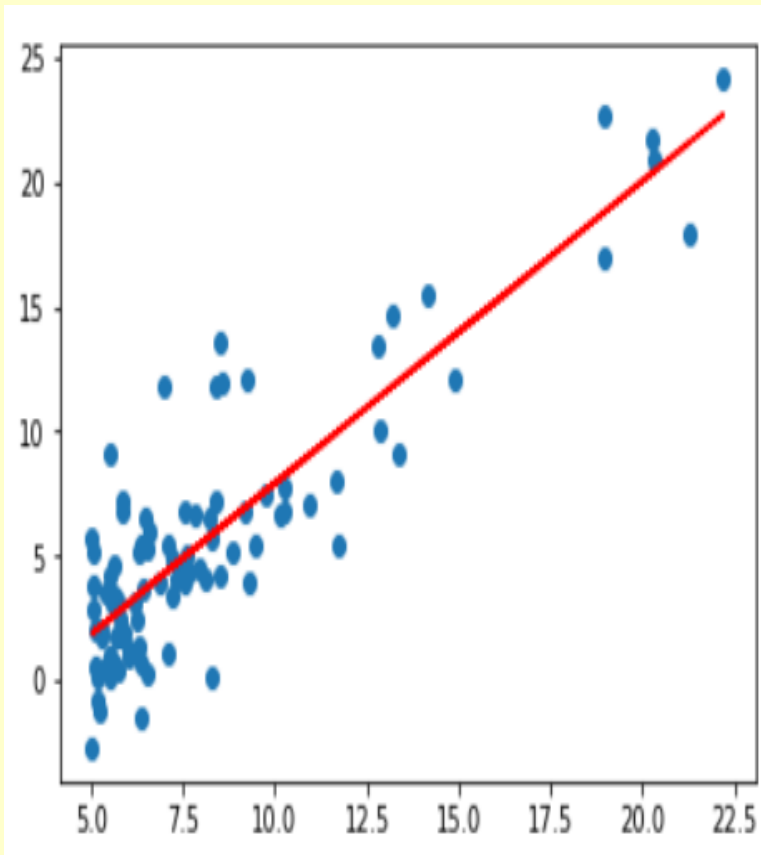
```
1 import numpy as np, os, pandas as pa
2 import matplotlib.pyplot as plt
3 %matplotlib inline
4 chemin="E:\mes_donnees_2025_2026\ML"; os.chdir(chemin)
5 df = pa.read_csv("univariate_linear_regression_dataset.csv",header=0)
6 x= df.iloc[:,0] ; n_samples = len(x);
7 x=np.array(x).reshape((n_samples, 1)) #96
8 y= df.iloc[:,1] ; y=np.array(y).reshape((n_samples, 1))
9 #x=np.array(x).reshape((n_samples, 1))
10 plt.scatter(x, y) # afficher les résultats. X en abscisse et y en ordonnée
11 plt.show()
12 # ajout de la colonne de biais a X
13 X = np.hstack((x, np.ones(x.shape)))
14
15 def f(X, theta):return X.dot(theta)
16 def erreur_somme_des_distances(X, y, theta):
17     m = len(y)
18     return (1/(2*m))*np.sum((f(X, theta)-y)**2)
19
20 def gradient(X, y, theta):
21     m = len(y)
22     return 1/m * X.T.dot(f(X, theta) - y)
```

Régression linéaire avec l'algorithme de descente de gradient

```
24 def gradient_descent(X, y, theta, learning_rate, n_iterations):
25     # création d'un tableau de stockage pour enregistrer l'évolution des erreurs
26     historique_des_erreurs = np.zeros(n_iterations)
27     for i in range(0, n_iterations):
28         theta = theta - learning_rate * gradient(X, y, theta) # mise a jour du theta
29         historique_des_erreurs[i] = erreur_somme_des_distances(X, y, theta)
30     return theta, historique_des_erreurs
31 # Exemple de test :
32 # création d'un vecteur parametre theta-
33 theta = np.random.randn(2, 1)
34 print(theta)
35 n_iterations = 10000
36 learning_rate = 0.012 # modifier pour voir l'effet
37 theta_final, historique_des_erreurs=gradient_descent(X, y, theta, learning_rate, n_iterations)
38 print(theta_final) # theta une fois que la machine a été entraînée
39 # création d'un vecteur prédictions qui contient les prédictions de notre modele final
40 predictions = f(X, theta_final)
41 # Affiche les résultats de prédictions (en rouge) par rapport a notre Dataset (en bleu)
42 plt.scatter(x, y); plt.plot(x, predictions, c='r')
43 plt.show(); plt.figure()
44 plt.plot(range(n_iterations), historique_des_erreurs)
45 plt.show(); print(f(np.array([22.5,1]), theta_final))
```

Régression linéaire avec l'algorithme de descente de gradient

`f(np.array([22.5,1]), theta_final)` → donne: `array([23.09330913])`



Régression linéaire avec l'algorithme de descente de gradient

$$f(x) = x^2 - x + 1 \Rightarrow \nabla f(x) = \frac{\partial f(x)}{\partial x} = f'(x) = 2x - 1$$

**Minimum au point x: $f'(x)=0$
 $X=0.5$**

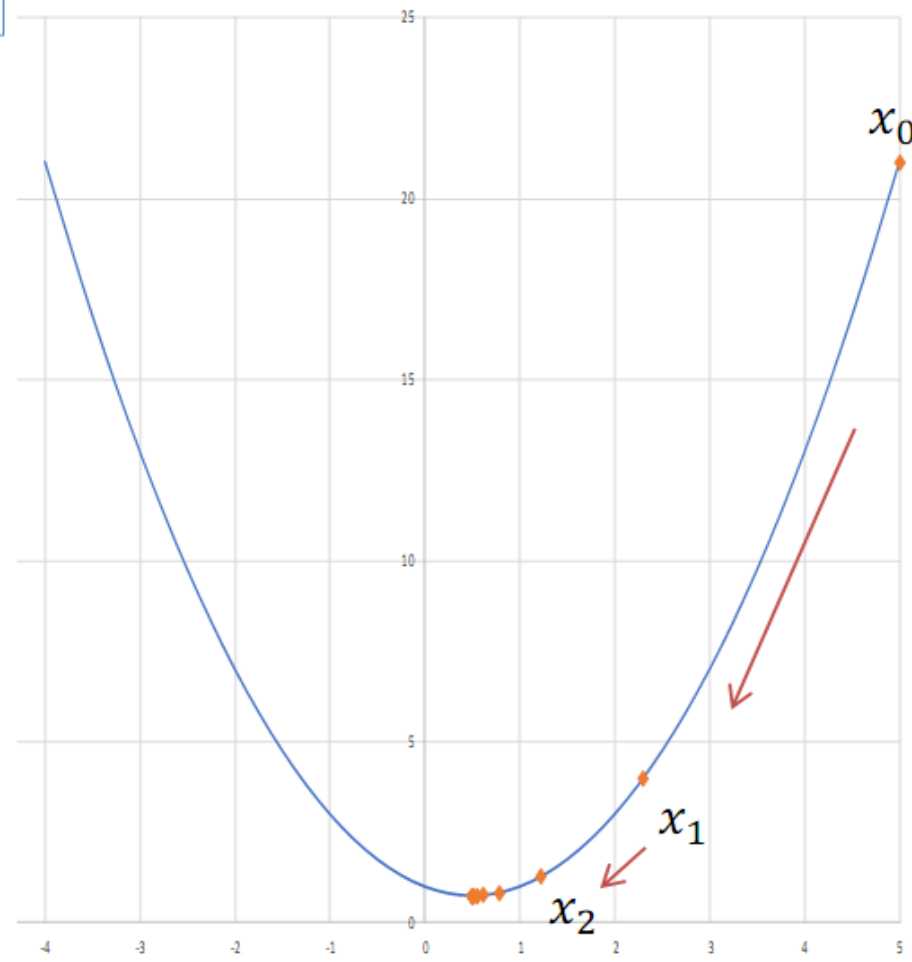
eta 0.3

$\eta = 0.3$

$x_0 = 5$

x	$f'(x_t)$	$f(x)$
5.0000		21.0000
2.3000	9.0000	3.9900
1.2200	3.6000	1.2684
0.7880	1.4400	0.8329
0.6152	0.5760	0.7633
0.5461	0.2304	0.7521
0.5184	0.0922	0.7503
0.5074	0.0369	0.7501
0.5029	0.0147	0.7500
0.5012	0.0059	0.7500
0.5005	0.0024	0.7500
0.5002	0.0009	0.7500
0.5001	0.0004	0.7500
0.5000	0.0002	0.7500

$$x_{t+1} = x_t - \eta \times \nabla f(x_t)$$



Régression linéaire avec l'algorithme de descente de gradient

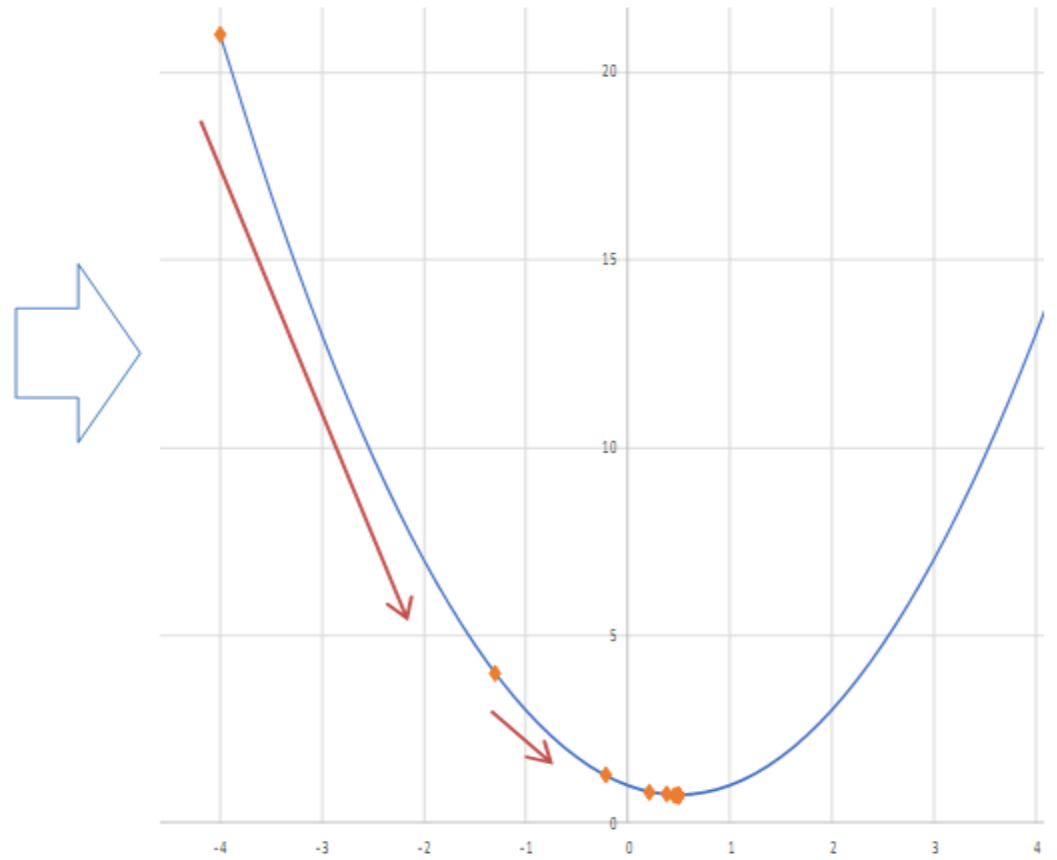
$$f(x) = x^2 - x + 1 \Rightarrow \nabla f(x) = \frac{\partial f(x)}{\partial x} = f'(x) = 2x - 1$$

**Minimum au point x: f(x)=0
X=0.5**

eta 0.3

x	f'(x_t)	f(x)
-4.0000		21.0000
-1.3000	-9.0000	3.9900
-0.2200	-3.6000	1.2684
0.2120	-1.4400	0.8329
0.3848	-0.5760	0.7633
0.4539	-0.2304	0.7521
0.4816	-0.0922	0.7503
0.4926	-0.0369	0.7501
0.4971	-0.0147	0.7500
0.4988	-0.0059	0.7500
0.4995	-0.0024	0.7500
0.4998	-0.0009	0.7500
0.4999	-0.0004	0.7500
0.5000	-0.0002	0.7500

$$x_{t+1} = x_t - \eta \times \nabla f(x_t)$$

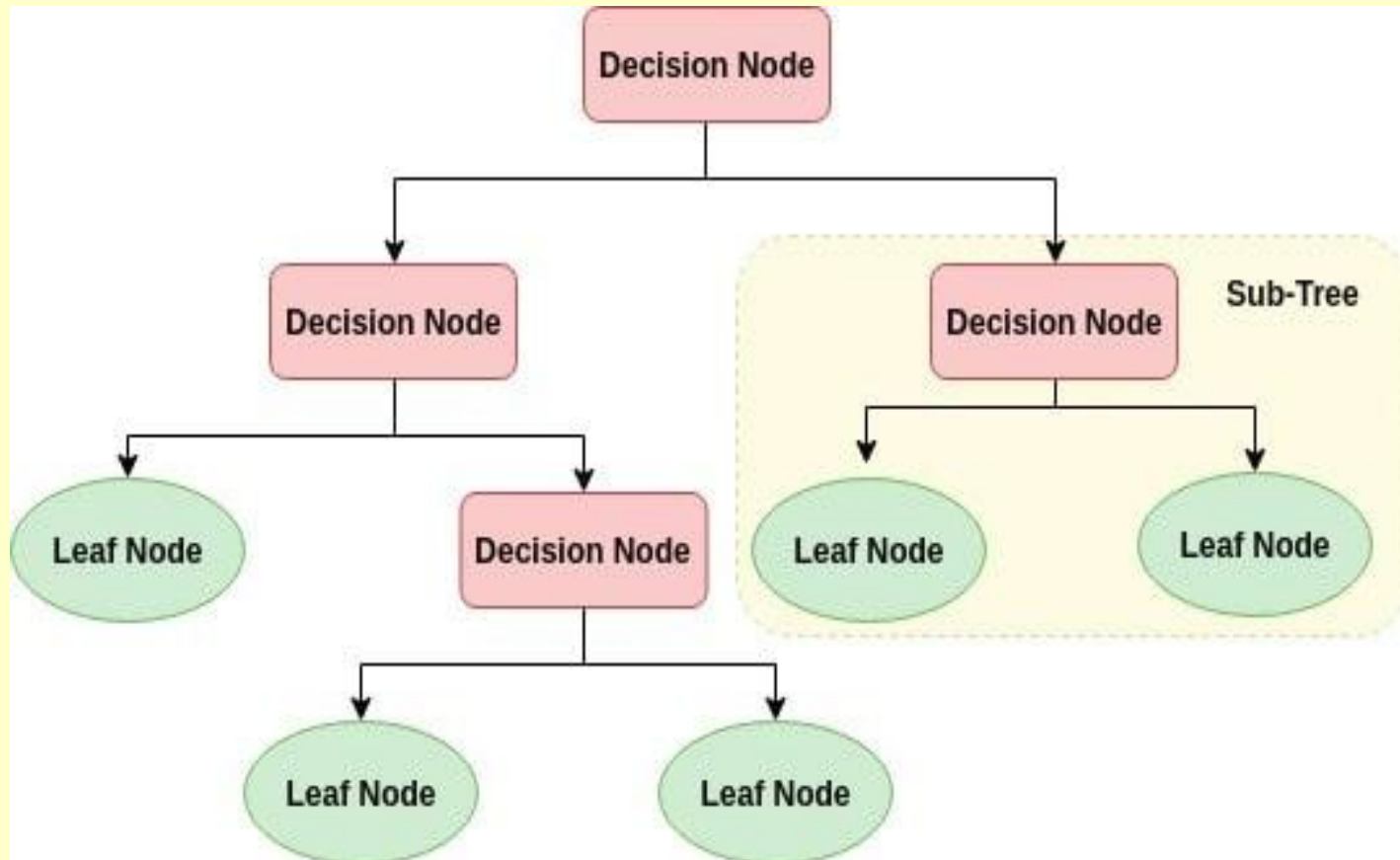


Application de Machine Learning sur la reconnaissance des nombres manuscrits



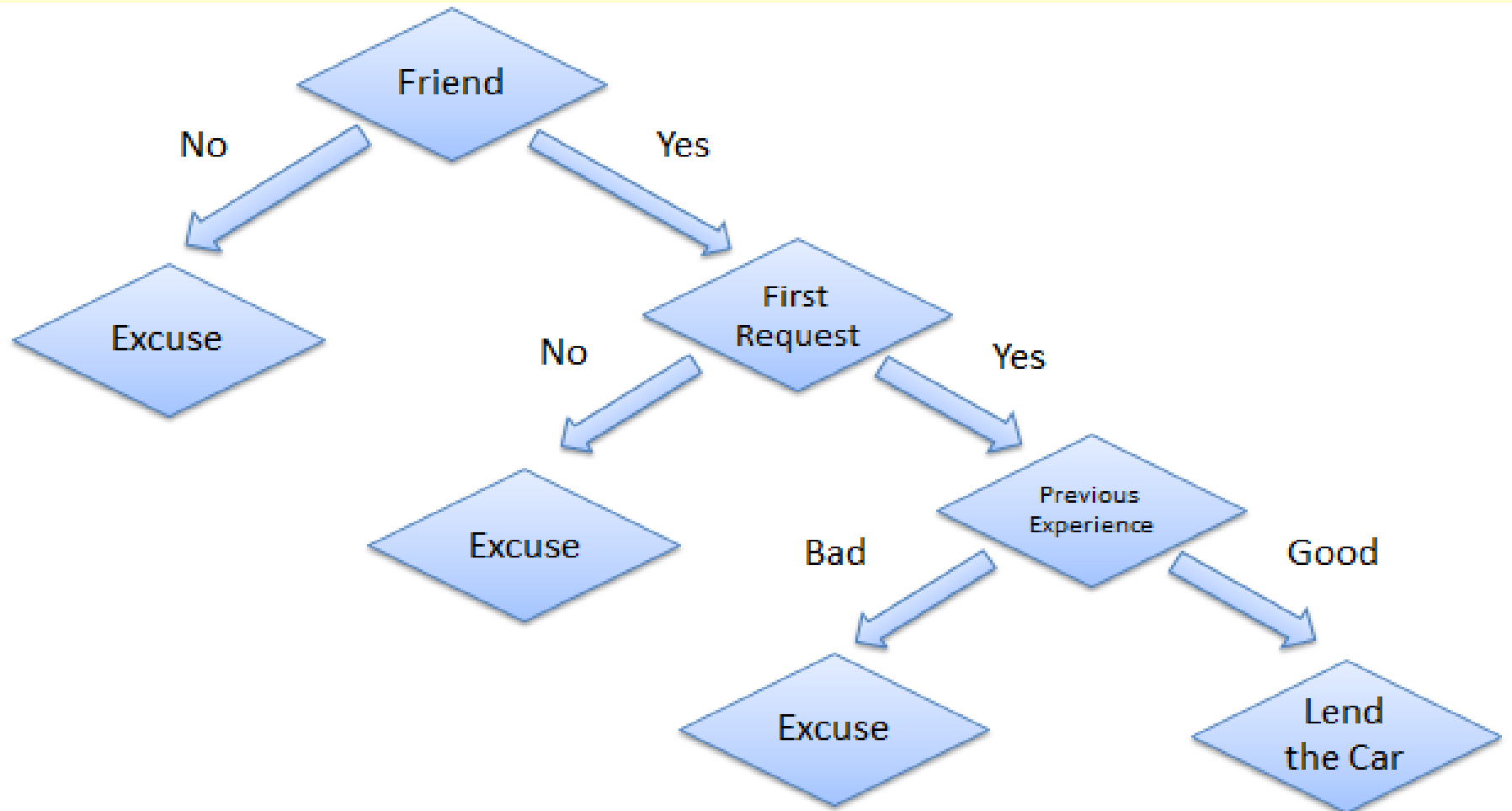
Les arbres de décision

L'arbre de décision est un algorithme qui se base sur **un modèle d'arbre** pour définir la décision finale. Chaque nœud comporte une condition (une question), et les branchements sont en fonction de cette condition. Plus on descend dans l'arbre, plus on cumule les conditions.



Les arbres de décision

Considérez un scénario dans lequel une personne vous demande de lui prêter votre voiture pour une journée, et vous devez décider de lui prêter ou non ta voiture.



Application de Machine Learning sur la reconnaissance des nombres manuscrits

```
1 import numpy as np, os
2 %matplotlib inline
3 import matplotlib.pyplot as plt
4 import pandas
5 from sklearn.tree import DecisionTreeClassifier
6 os.chdir("E:\mes_donnees_2025_2026\ML")
7 data=np.array(pandas.read_csv("images_chiffres_codees_niveau_de_gris.csv"))
8 clf=DecisionTreeClassifier()
9 from sklearn.model_selection import train_test_split
10 x=data[:,1:]
11 label=data[:,0]
12 x_train, x_test, y_train, y_test =train_test_split(x,label, test_size=0.2)
13 clf.fit(x_train,y_train)
14 p=clf.predict(x_test)
15 s=clf.score(x_test, y_test)
16 print("Le pourcentage de réussite="+str(s*100)+'%')
```

Le pourcentage de réussite=67.7734375%

Application de Machine Learning sur la reconnaissance des nombres manuscrits

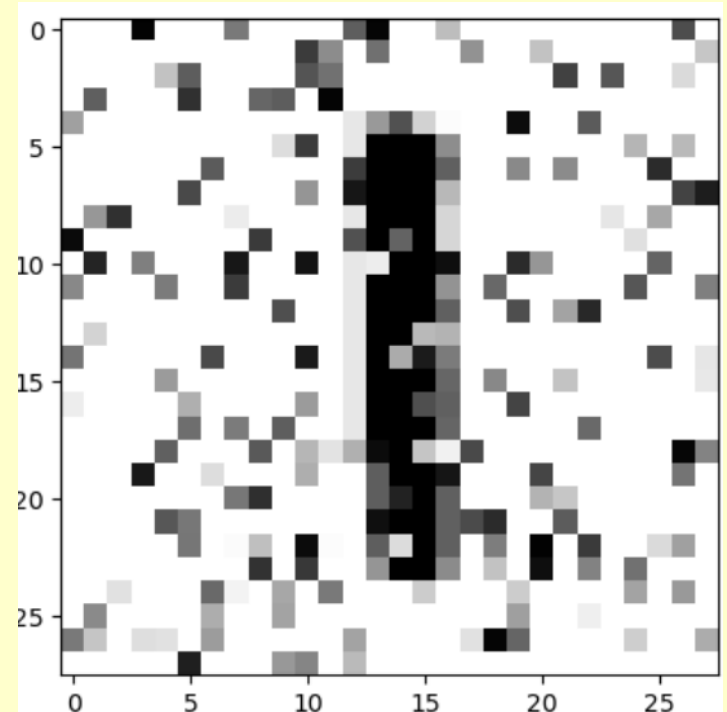
```
1 import numpy as np, os
2 %matplotlib inline
3 import matplotlib.pyplot as plt
4 import pandas
5 from sklearn.tree import DecisionTreeClassifier
6 os.chdir("E:\mes_donnees_2025_2026\ML")
7 data=np.array(pandas.read_csv("images_chiffres_codees_niveau_de_gris_complet.csv"))
8 clf=DecisionTreeClassifier()
9 from sklearn.model_selection import train_test_split
10 x=data[:,1:]
11 label=data[:,0]
12 x_train, x_test, y_train, y_test =train_test_split(x,label, test_size=0.2)
13 clf.fit(x_train,y_train)
14 p=clf.predict(x_test)
15 s=clf.score(x_test, y_test)
16 print("Le pourcentage de réussite="+str(s*100)+'%')
```

Le pourcentage de réussite=86.10714285714286%

Application de Machine Learning sur la reconnaissance des nombres manuscrits

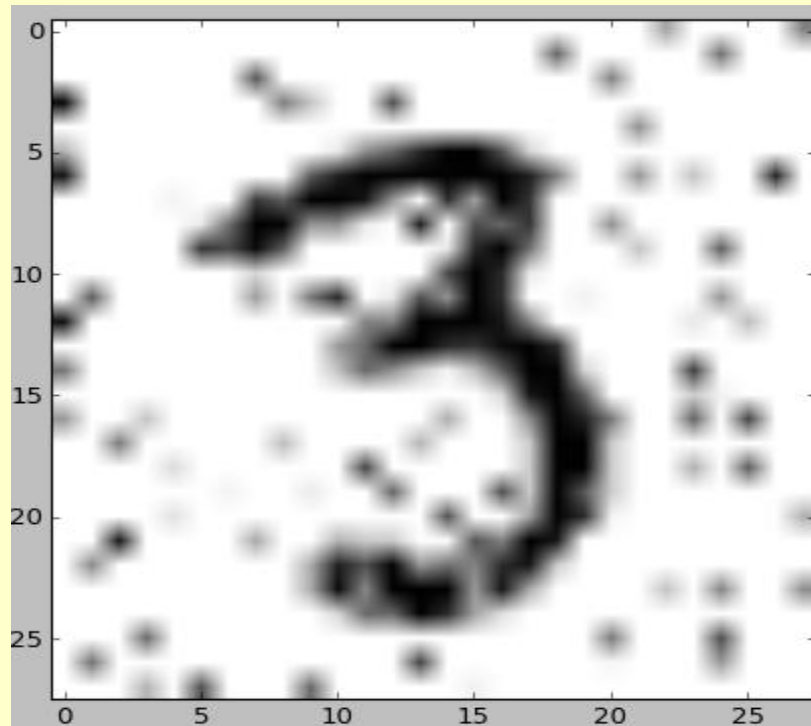
```
21 d=x_test[18]
22 Nombre_de_pixels_errones=200
23 for i in range(Nombre_de_pixels_errones):
24     position=np.random.randint(0,784,1)[0]
25     bruit=np.random.randint(-200,200,1)[0]
26     d[position]+=bruit
27     d[position]=d[position]%255
28 print(clf.predict([d]))
29 d.shape=(28,28)
30 plt.imshow(255-d,cmap='gray')
31 plt.show()
```

Le pourcentage de réussite=71.6796875%
[9]



Application de Machine Learning sur la reconnaissance des nombres manuscrits

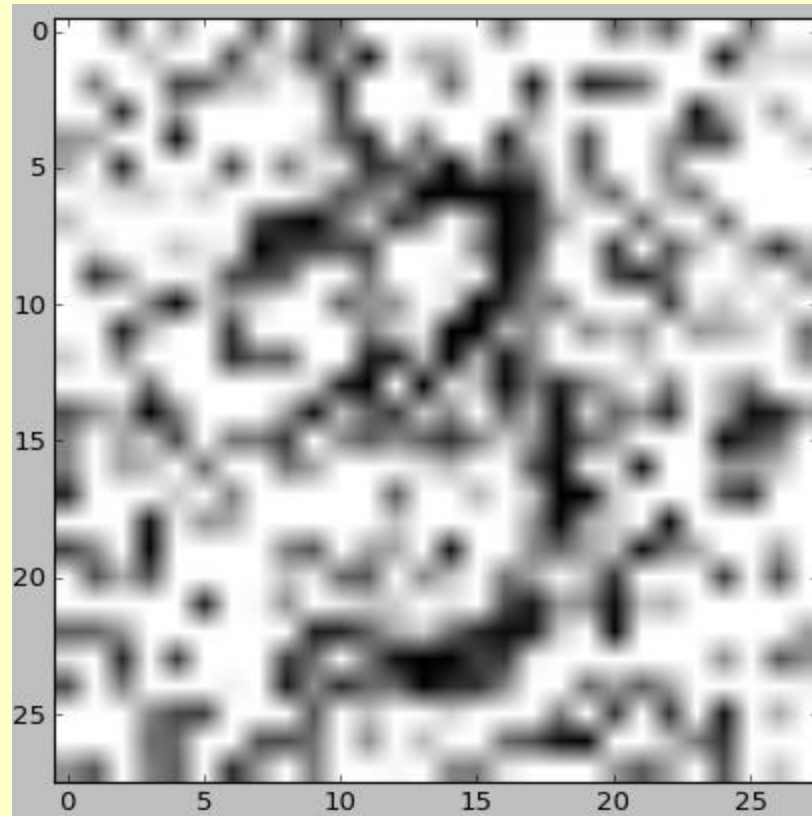
Avec un Nombre_de_pixels_errones=100 on obtient une image comme la Suivante (exemple pour le chiffre 3):



L'instruction `print(clf.predict([d]))` a donné la valeur 3 donc une reconnaissance réussie

Application de Machine Learning sur la reconnaissance des nombres manuscrits

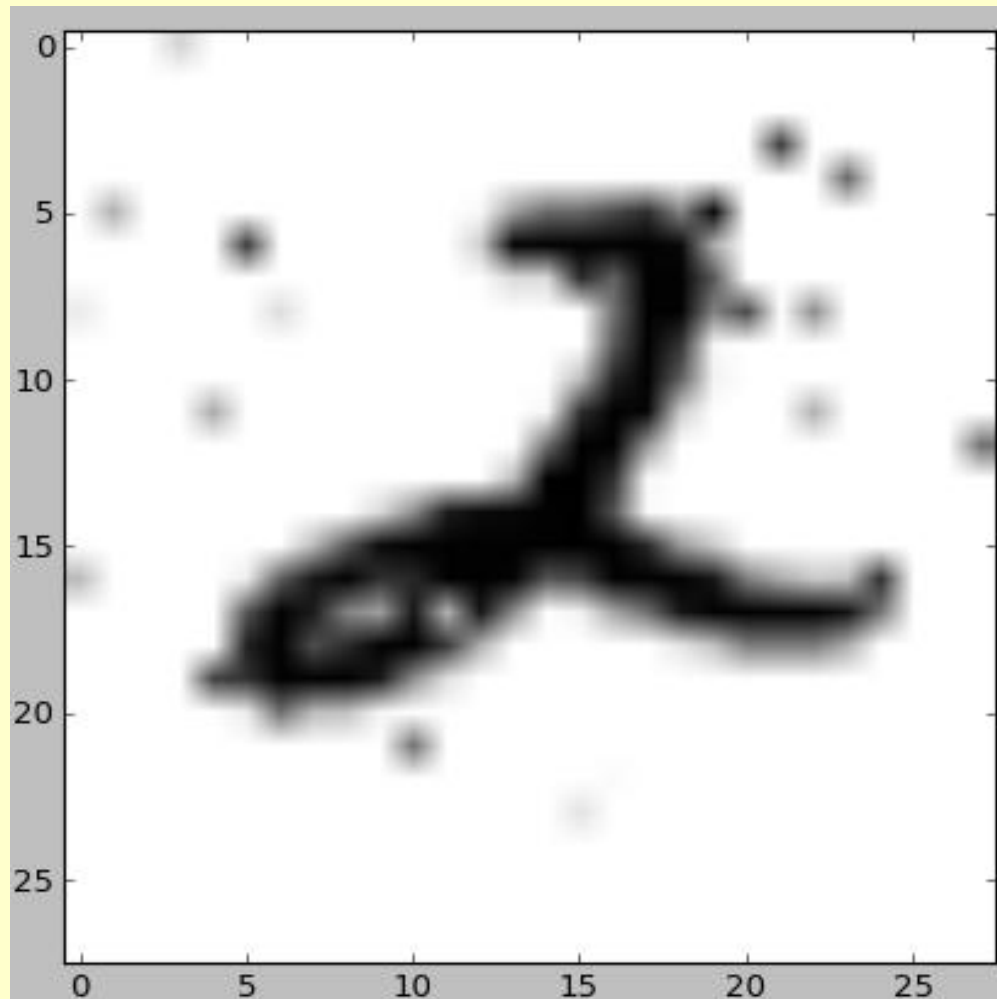
Avec un Nombre_de_pixels_errones=500 on obtient l'image suivante:



L'instruction `print(clf.predict([d]))` a donné la valeur 5 donc une reconnaissance non réussie à cause du bruit très fort.

Application de Machine Learning sur la reconnaissance des nombres manuscrits

Le programme donné en haut a pu reconnaître l'image suivante: chiffre 2



KNN: k Nearest Neighbors

C'est un algorithme qui peut servir autant pour la classification que la régression. Il est surnommé « nearest neighbors » (K plus proches voisins en français) car le principe de ce modèle consiste en effet à choisir les **k** données les plus proches du point étudié afin d'en prédire sa valeur (sa classe).

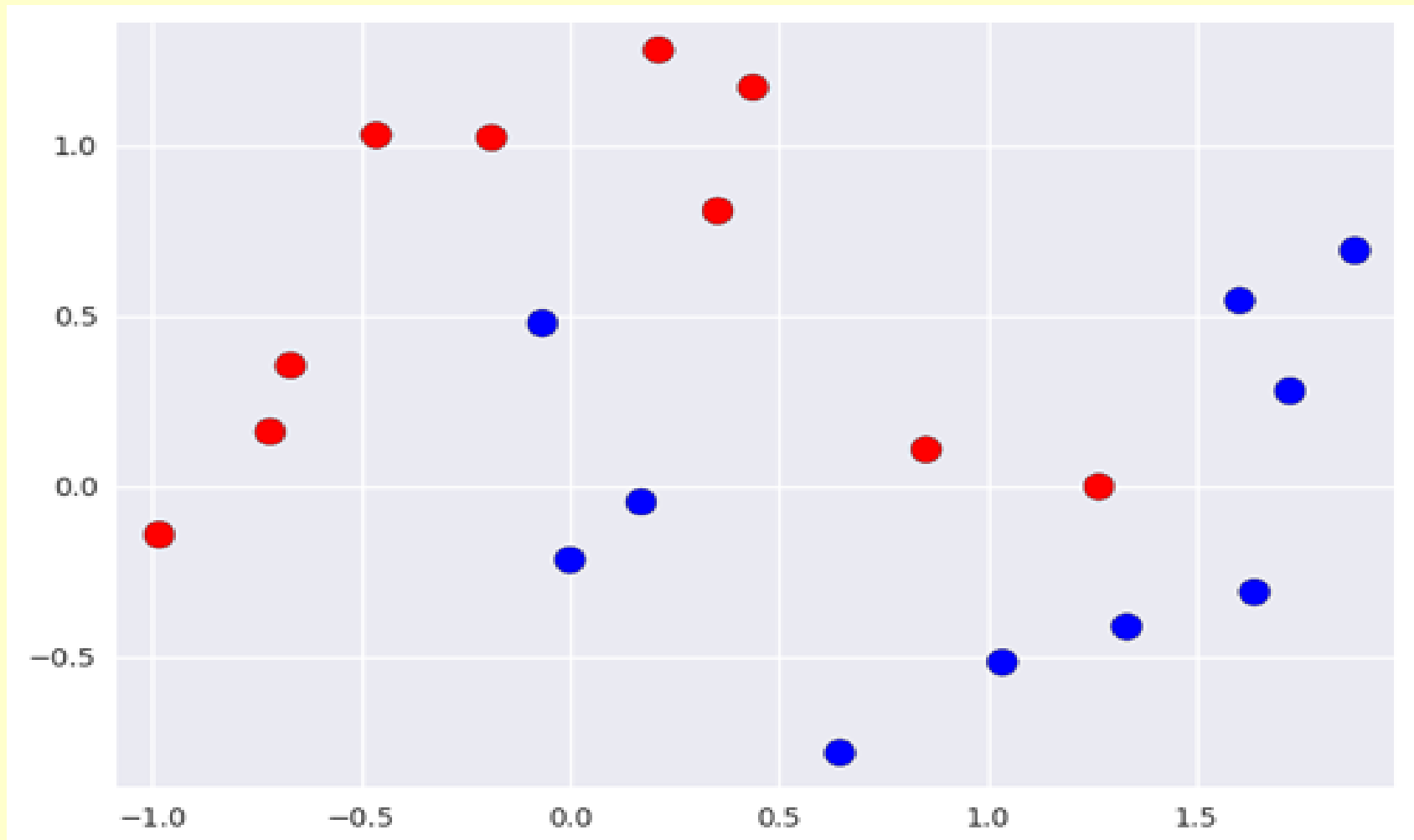
- K-NN est une méthode d'apprentissage **supervisé**, qui fonctionne sous le principe de : “Dis moi qui sont tes voisins, je te dirais qui tu es...”.
- Avant d'utiliser un vrai jeu de données, on va prendre un petit exemple afin de comprendre le fonctionnement de l'algorithme KNN: un jeu de données d'entraînement, avec deux classes, rouge et bleu. L'*input* est donc *bidimensionnel* ici, et la *target* est la couleur à classer.

KNN: k Nearest Neighbors

C'est un algorithme qui peut servir autant pour la classification que la régression. Il est surnommé « nearest neighbors » (K plus proches voisins en français) car le principe de ce modèle consiste en effet à choisir les **k** données les plus proches du point étudié afin d'en prédire sa valeur (sa classe).

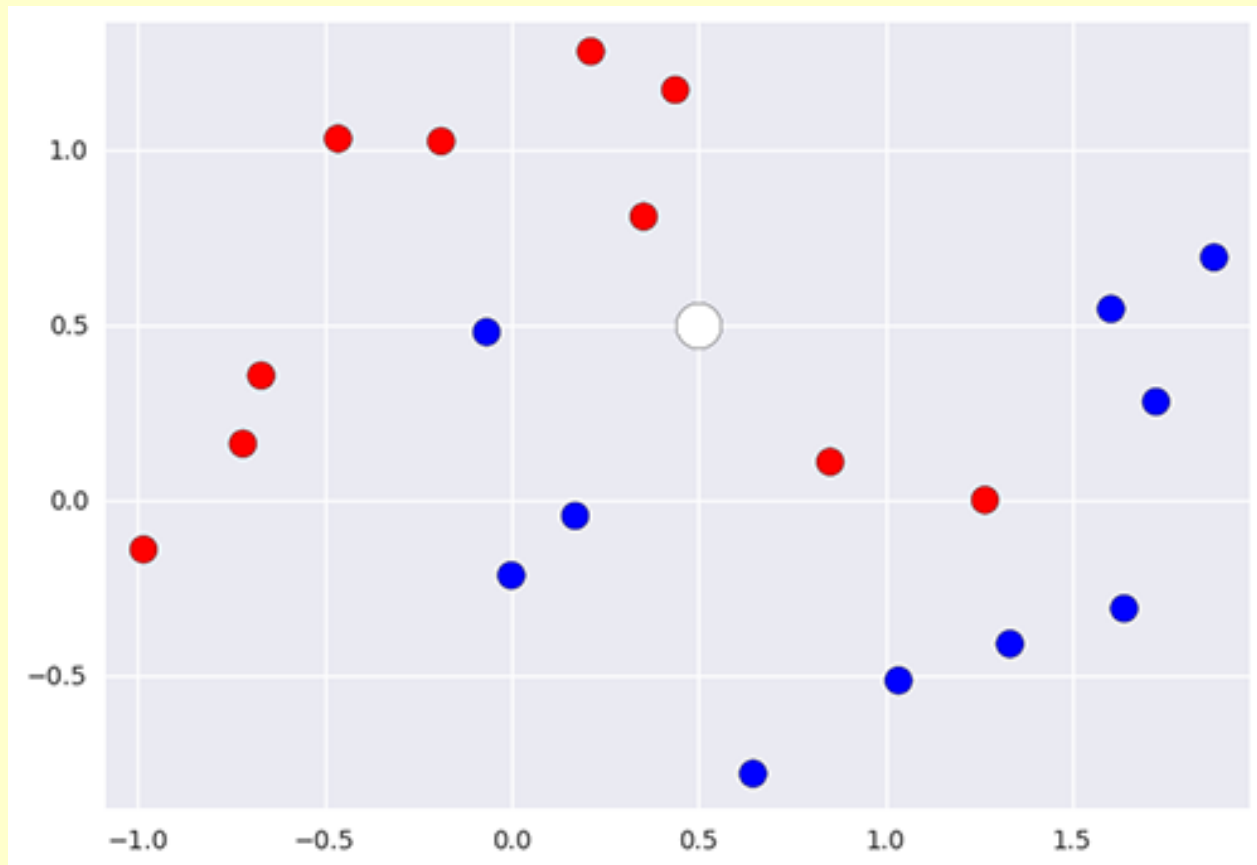
- K-NN est une méthode d'apprentissage **supervisé**, qui fonctionne sous le principe de : **“Dis moi qui sont tes voisins, je te dirais qui tu es...”**.
- Avant d'utiliser un vrai jeu de données, on va prendre un petit exemple afin de comprendre le fonctionnement de l'algorithme KNN: un jeu de données d'entraînement, avec deux classes, rouge et bleu. L'*input* est donc *bidimensionnel* ici, et la *target* est la couleur à classer.

KNN: k Nearest Neighbors



KNN: k Nearest Neighbors

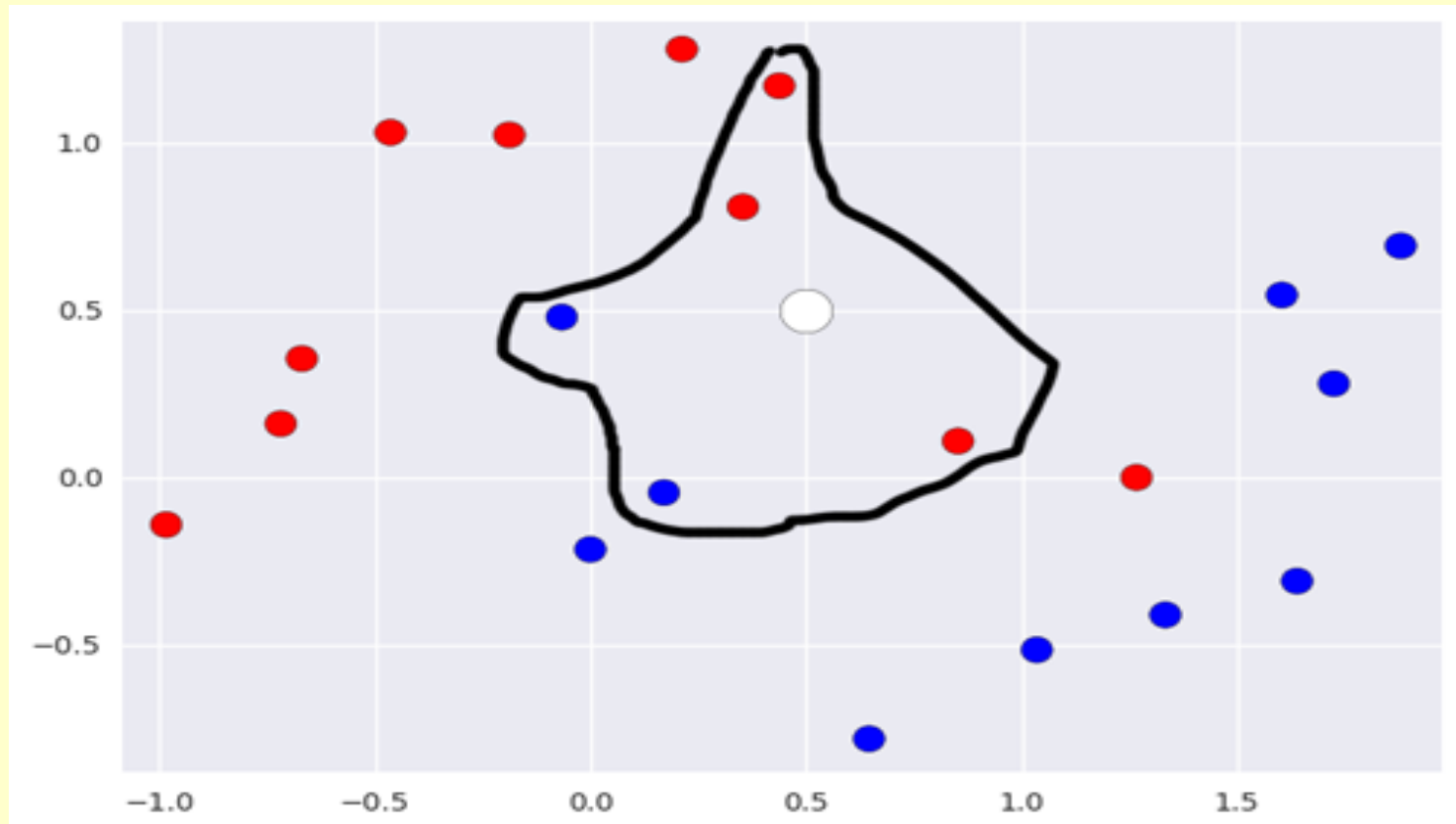
Si on a une nouvelle entrée dont on veut prédire la classe, comment pourrait-on faire ?



KNN: k Nearest Neighbors

on va simplement regarder les **k** voisins les plus proches de ce point et regarder quelle classe constitue la majorité de ces points afin d'en en déduire la classe du nouveau point. Par exemple ici, si on utilise le 5-NN, on peut prédire que la nouvelle donnée appartient à la classe **rouge** puisqu'elle a 3 rouges et 2 bleus dans son entourage.

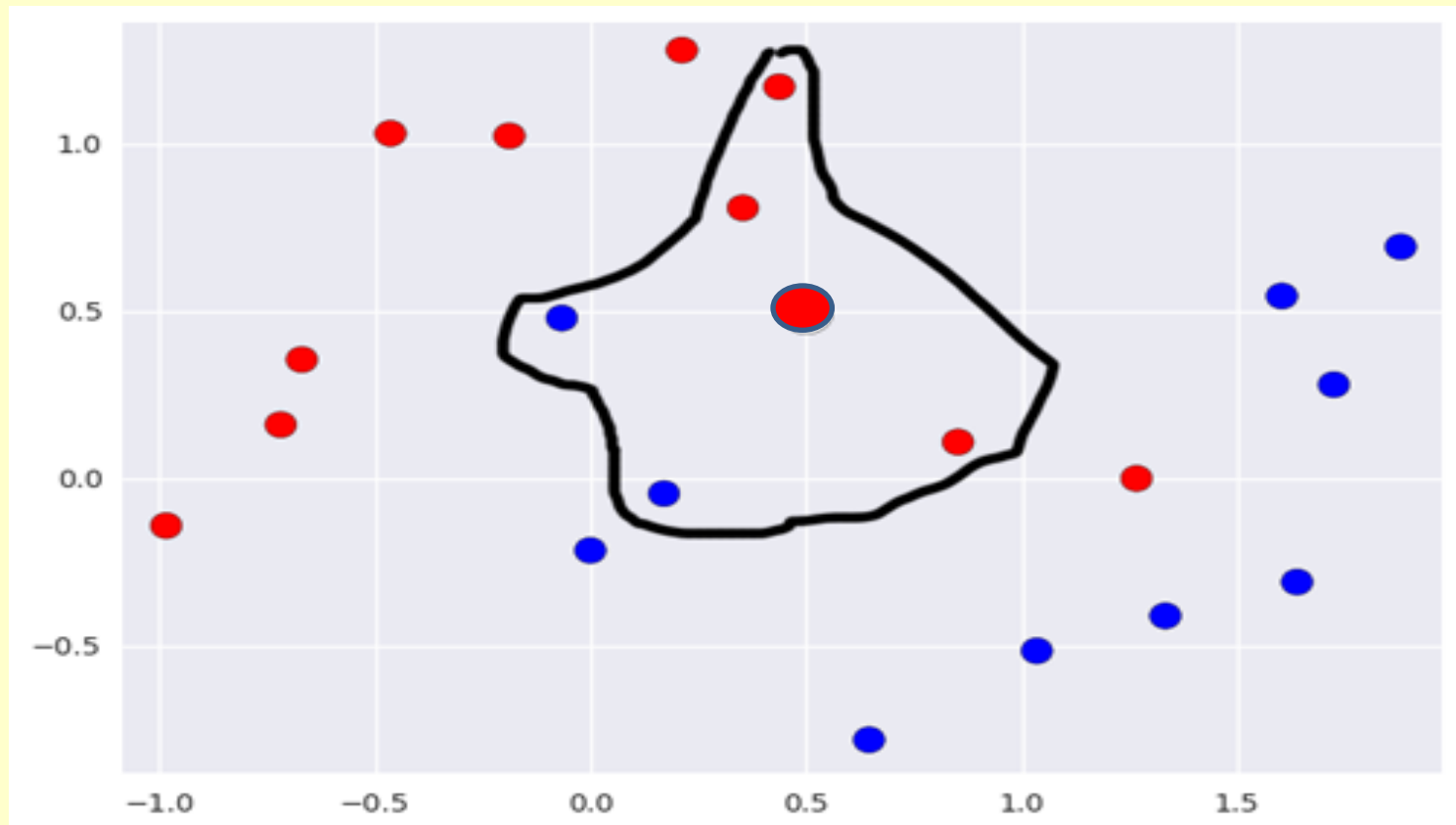
Vote Majoritaire



KNN: k Nearest Neighbors

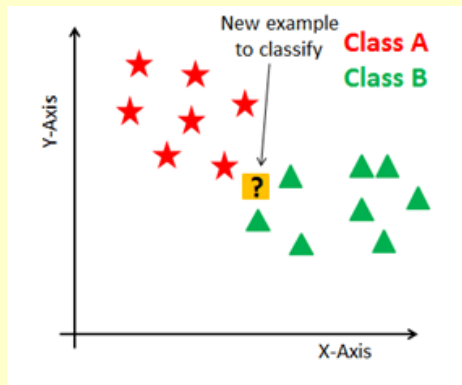
on va simplement regarder les **k** voisins les plus proches de ce point et regarder quelle classe constitue la majorité de ces points afin d'en en déduire la classe du nouveau point. Par exemple ici, si on utilise le 5-NN, on peut prédire que la nouvelle donnée appartient à la classe **rouge** puisqu'elle a 3 rouges et 2 bleus dans son entourage.

Vote Majoritaire

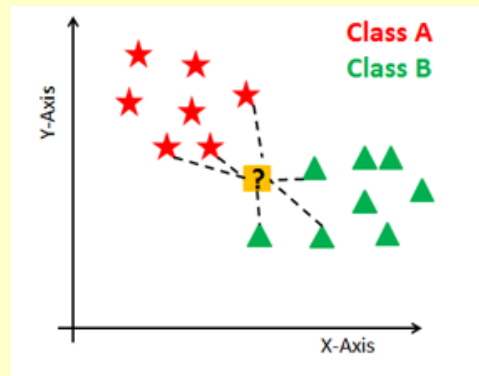


KNN: k Nearest Neighbors

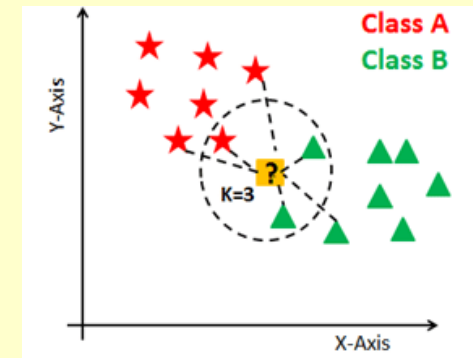
1. Initialiser la nouvelle donnée



2. Calculer la Distance



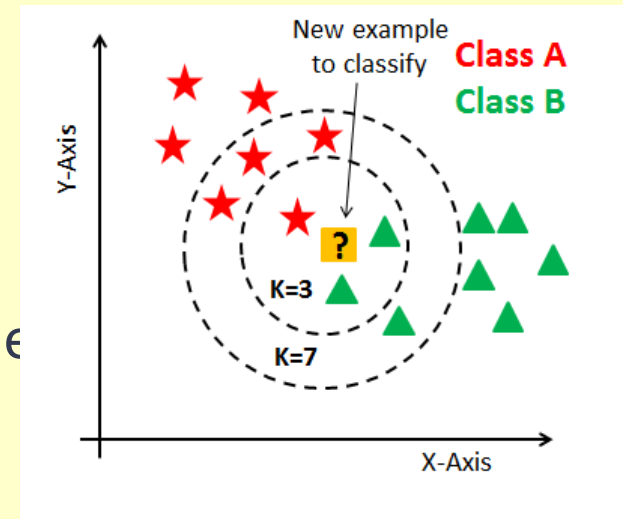
3. Définir les k plus proches voisins & Voter pour les étiquettes



- **Distance Euclidienne** : $d(x, y) = \sqrt{\sum_{i=1}^n (x_i - y_i)^2}$
- **Distance de Manhattan** : $d(x, y) = \sum_{i=1}^n |x_i - y_i|$
- **Distance de Minkowski** : $d(x, y) = \sqrt[q]{\sum_{i=1}^n |x_i - y_i|^q}$

KNN: k Nearest Neighbors

- K est **un hyperparamètre** ou une variable de contrôle pour le modèle de prédiction.
- Chaque jeu de données a ses propres exigences.
- K doit être un nombre **impair** pour simplifier le vote majoritaire.
- Générer le modèle sur différentes valeurs de k et vérifier leurs performances.



KNN: k Nearest Neighbors

Les étapes de KNN:

- 1) Choisir la distance, la mesure de similarité entre les échantillons(instances)
- 2) Choisir (déterminer) la valeur de K.
- 3) Calculer la distance entre la nouvelle entrée et toutes les données de la base d'apprentissage.
- 4) Déterminer les classes des K plus proches voisins
- 5) Donner la prédiction pour la nouvelle entrée.

Avantages	Inconvénients
• Apprentissage rapide • Ne nécessite pas une généralisation de modèle • Peut être utile en cas de données non linéaires. • Méthode facile à comprendre .	• Prédiction Lente et coûteuse en temps et en mémoire, car il faut stocker tous les exemples et il faut les revoir de nouveau pour chaque prédiction. • Ne convient pas pour les grandes dimensions.

KNN: L'algorithme

1. Charger les données
2. Initialiser k au nombre de plus proches voisins choisi
3. Pour chaque exemple dans les données:
 - 3.1 Calculer la distance entre notre requête et l'observation itérative actuelle de la boucle depuis les données.
 - 3.2 Ajouter la distance et l'indice de l'observation concernée à une collection ordonnée de données
 - 3.4. Trier cette collection ordonnée contenant distances et indices de la plus petite distance à la plus grande (dans l'ordre croissant).
5. Sélectionner les k premières entrées de la collection de données triées (équivalent aux k plus proches voisins)
6. Obtenir les étiquettes des k entrées sélectionnées
7. Si **régression**, retourner la moyenne des k étiquettes
8. Si **classification**, retourner le mode (valeur la plus fréquente/commune) des k étiquettes

KNN: k Nearest Neighbors

```
1 X = [[0], [1], [2], [3]]
2 y1 = [0, 0, 1, 1]
3 from sklearn.neighbors import KNeighborsClassifier
4 #valerur de k pour knn et choix de la distance
5 neigh = KNeighborsClassifier(n_neighbors=3,metric="euclidean")
6 neigh.fit(X, y1)
7 print('Knn predefinie-classification:',neigh.predict([[2.1]]))
8 from sklearn.neighbors import KNeighborsRegressor
9 knn_regression =KNeighborsRegressor(3)
10 y2 = [15.3, 8.6, 6.5, 4.29]
11 knn_regression.fit(X,y2)
12 print('Knn predefinie-regression:',knn_regression.predict([[2.1]]))
```

Knn predefinie-classification: [1]

Knn predefinie-regression: [6.46333333]

```

1 import numpy as np,os
2 %matplotlib inline
3 import matplotlib.pyplot as plt,pandas
4 from sklearn.neighbors import KNeighborsClassifier
5 neigh = KNeighborsClassifier(n_neighbors=5)#valerur de k pour knn
6 # il faut travailler avec le data set complet, ici juste une partie
7 os.chdir("E:\mes_donnees_2025_2026\ML")
8 data=np.array(pandas.read_csv("images_chiffres_codees_niveau_de_gris.csv"))
9 x=data[0:1000,1:]
10 label=data[0:1000,0]
11 neigh.fit(x,label)
12 xtest=data[1000:2000,1:]
13 test_label=data[1000:2000,0]
14 p=neigh.predict(xtest)
15 count=0
16 for i in range(0,1000):
17     count+=1 if p[i]==test_label[i] else 0
18 print("Accuracy=",str((count/1000)*100)+'%')

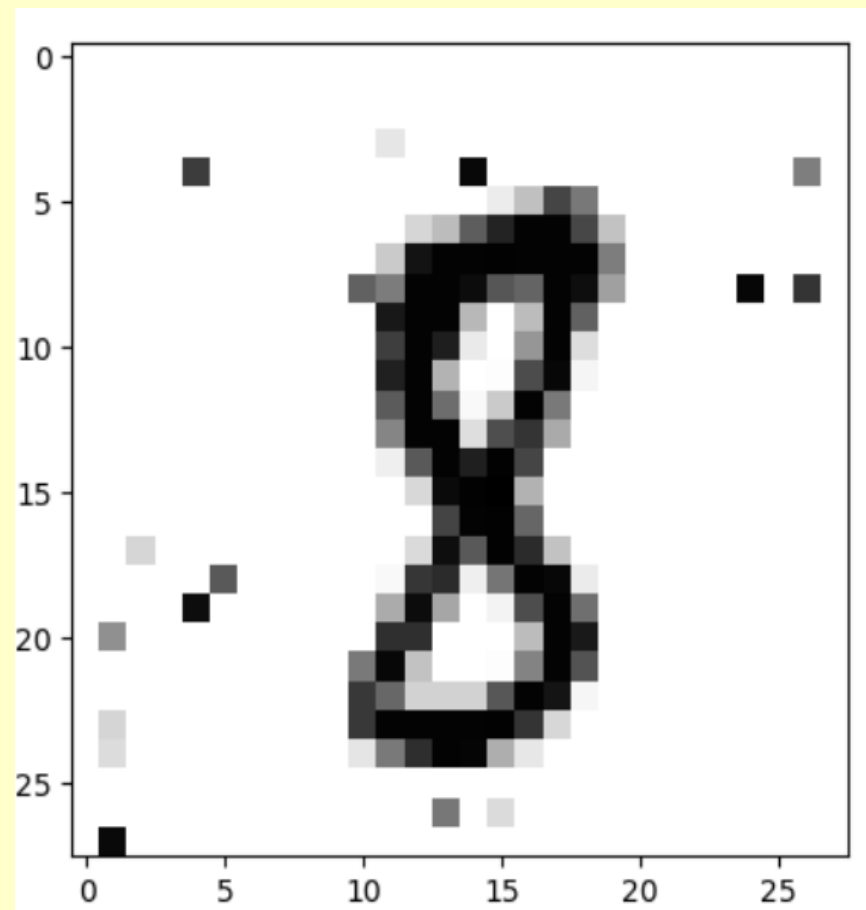
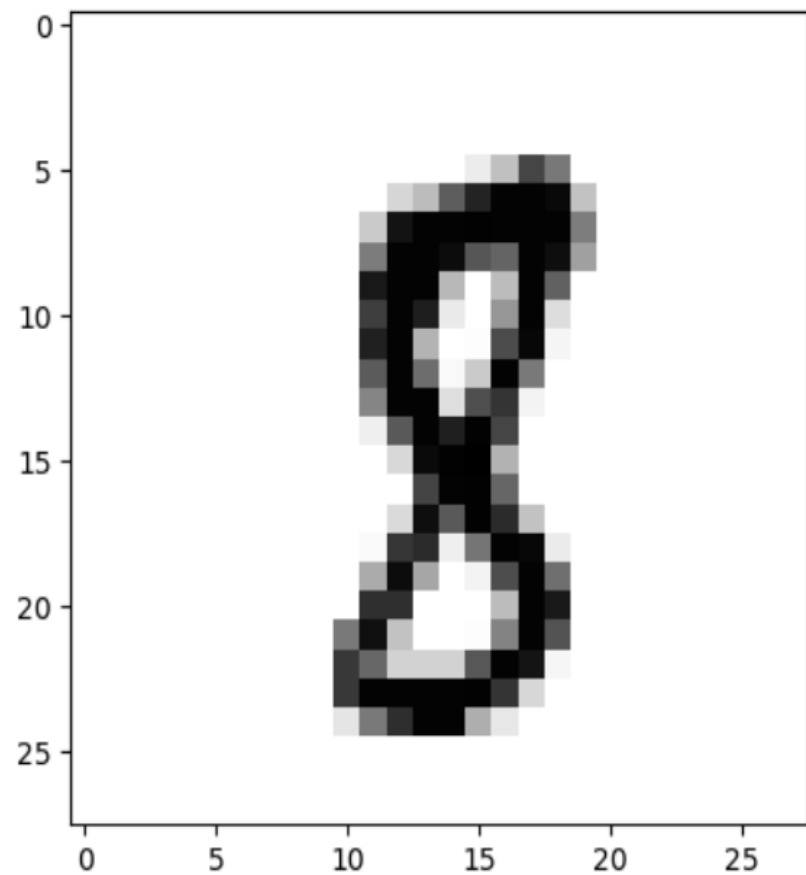
```

```
20 d=xtest[120]; chiffre_vrai=test_label[120]
21 print(chiffre_vrai)
22 copie_d=d.copy()
23 Nombre_de_pixels_errones=20
24 for i in range(Nombre_de_pixels_errones):
25     position=np.random.randint(0,784,1)[0]
26     bruit=np.random.randint(-200,200,1)[0]
27     d[position]+=bruit
28     d[position]=d[position]%255
29 print(neigh.predict([d]))
30 copie_d.shape=(28,28); copie_d=255-copie_d
31 plt.imshow(copie_d,cmap='gray')
32 plt.show()
33 d.shape=(28,28); d=255-d
34 plt.imshow(d,cmap='gray')
35 plt.show()
```

Accuracy= 86.8%

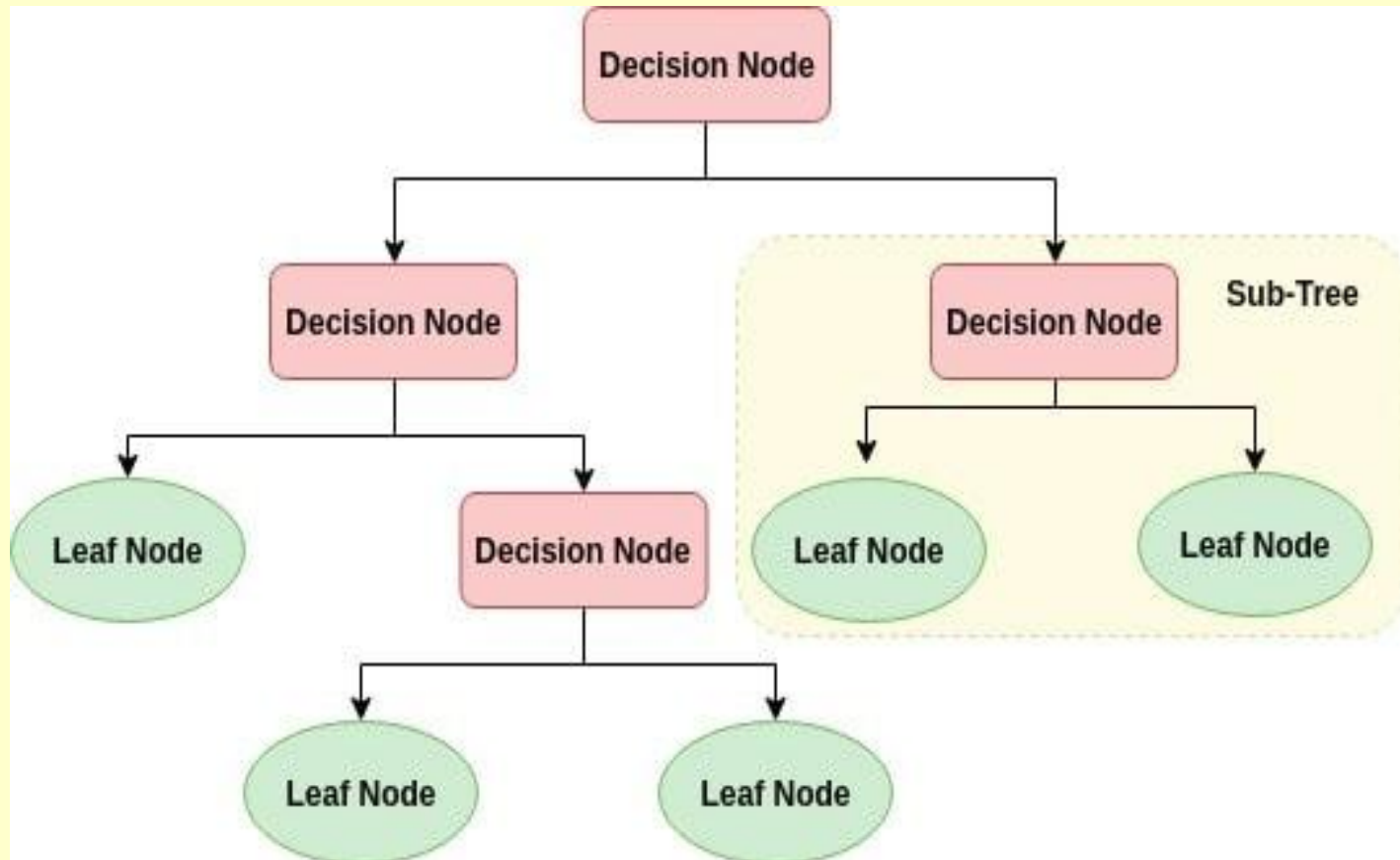
8

[8]



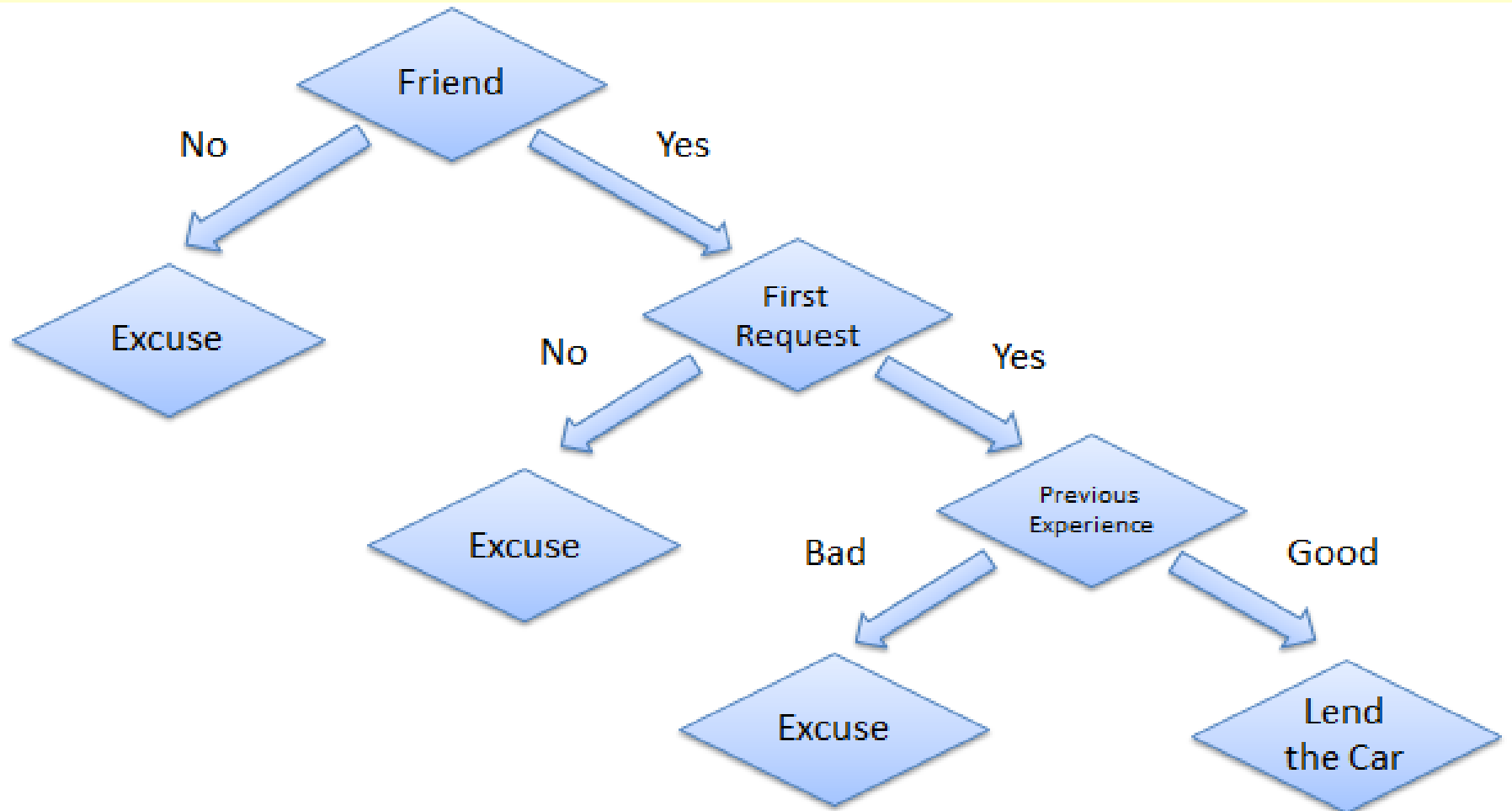
Les arbres de décision

L'arbre de décision est un algorithme qui se base sur **un modèle d'arbre** pour définir la décision finale. Chaque nœud comporte une condition (une question), et les branchements sont en fonction de cette condition. Plus on descend dans l'arbre, plus on cumule les conditions.



Les arbres de décision

Considérez un scénario dans lequel une personne vous demande de lui prêter votre voiture pour une journée, et vous devez décider de lui prêter ou non ta voiture.



Les arbres de décision

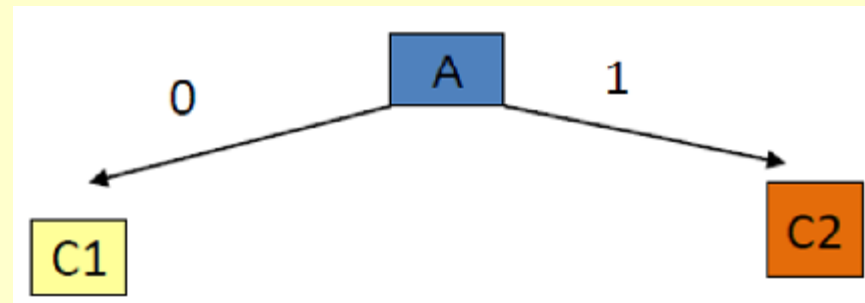
L'utilisation des arbres de décision pour l'analyse prédictive présente plusieurs avantages:

- Les arbres de décision peuvent être utilisés pour prédire des valeurs continues et discrètes, c'est-à-dire qu'ils fonctionnent bien pour les tâches de régression et de classification.
- Ils nécessitent relativement moins d'efforts pour former l'algorithme.
- Ils peuvent être utilisés pour classer des données séparables de manière non linéaire.
- Ils sont rapides et efficaces par rapport à KNN et à d'autres algorithmes de classification (revoir TP1-TP2).

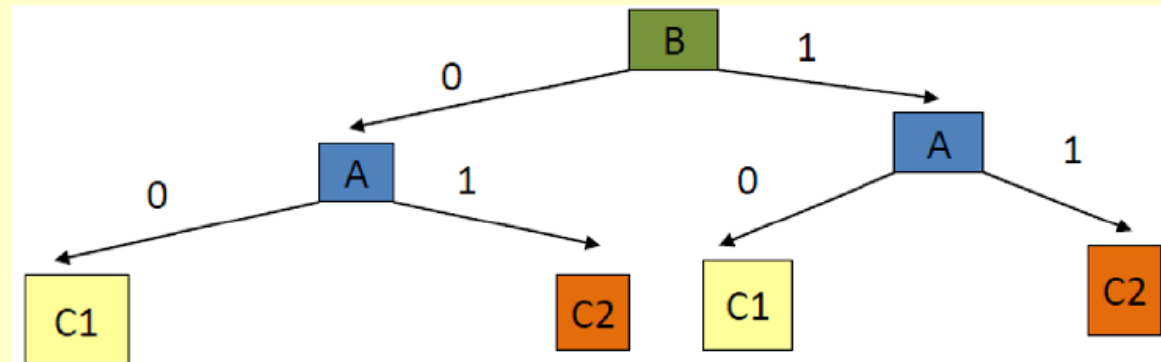
Choix de l'attribut de partitionnement

Partitionnement selon l'attribut A

A	B	Classe
0	1	C1
0	0	C1
1	1	C2
1	0	C2



Partitionnement selon l'attribut B



Choix de l'attribut de partitionnement

- Un arbre de décision représente **la suite** ($\neq$ de ensemble) de questions à poser pour classer un nouveau exemple.
- Dans l'exemple précédent c'est l'attribut A qui apporte plus d'information à la classification, il est mieux que l'attribut B.
- **L'objectif est de minimiser le nombre de questions à poser.**
- **Question:** comment quantifier l'information apportée par chaque attribut ?

Les algorithmes de construction d'un arbre de décision:

L'entropie

- Une formule pour calculer l'homogénéité d'un échantillon.
- Un échantillon complètement homogène a une **entropie de 0**.
- La formule pour l'entropie est:

$$Entropie(S) = -1 * \sum_{c \in Classes} portion(c) * \log_2(portion(c))$$

Avec S est l'ensemble des instances (les lignes du data_train...)

Choix de l'attribut de partitionnement

Numéro de ligne	A	B	Classe
L1	0	1	C1
L2	0	0	C1
L3	1	1	C2
L4	1	0	C2

S: l'ensemble des instances contient les lignes L1,L2,L3 et L4.

Classes={C1,C2}

$$Entropie(S) = -1 * \sum_{c \in Classes(S)} portion(c) * \log_2(portion(c))$$

$$Entropie(S) = -1 * (0.5 * \log_2(0.5) + 0.5 * \log_2(0.5))$$

```
from math import *
```

```
>>> -1*(0.5*log2(0.5)+0.5*log2(0.5))
```

```
>>> 1.0
```

Les algorithmes de construction d'un arbre de décision: calcul du Gain d'information

- Il faut sélectionner l'attribut **avec le plus grand gain d'information**.
- Si on fait un partitionnement de l'ensemble d'instances **S** selon l'attribut **Attribut** nous obtenons des sous ensembles d'instances **S_i** avec i compris entre 1 et le nombre k de valeurs distincts de l'attribut **Attribut**:

$$S = \bigcup_{i=1}^k S_i$$

- Il permet d'avoir l'attribut qui crée les branches les plus homogènes.
- Le gain d'information est basé sur la diminution de l'entropie après la division d'un ensemble de données par rapport à un attribut.
- **Le Gain d'information est donné par la formule:**

$$\text{Gain}(S, \text{Attribut}) = \text{Entropie}(S) - \sum_{i=1}^k p_i * \text{Entropie}(S_i)$$

Avec p_i est la portion du sous ensemble d'instances **S_i** par rapport à l'ensemble d'instances **S**:

$$p_i = \frac{\text{Nombre de lignes de } S_i}{\text{Nombre de lignes de } S}$$

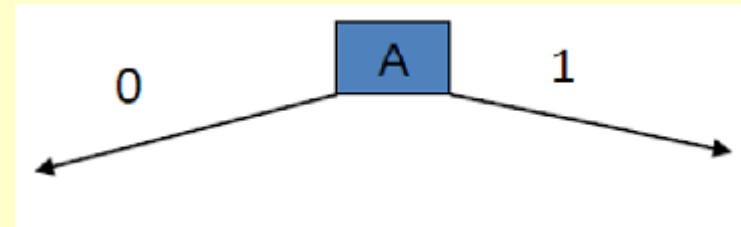
Choix de l'attribut de partitionnement

S: l'ensemble des instances
contient les lignes L1,L2,L3 et L4.

Classes={C1,C2}

Numéro de ligne	A	B	Classe
L1	0	1	C1
L2	0	0	C1
L3	1	1	C2
L4	1	0	C2

Partitionnement selon l'attribut A



S1 contient
{L1===C1
L2===C1}

S2 contient
{L3===C2
L4===C2}

$$E(S1) = -1 * (\text{portion}(C1) * \log_2(\text{portion}(C1))) \\ = -1 * (1 * \log_2(1)) = 0$$

$$E(S2) = - (\text{portion}(C2) * \log_2(\text{portion}(C2))) = 0$$

$$\text{Gain}(S,A) = 1 - (\text{portion}(S1) * E(S1) + \text{portion}(S2) * E(S2)) \\ = 1 - ((2/4) * E(S1) + (2/4) * E(S2)) \\ = 1 - ((2/4) * (0) + (2/4) * (0))$$

$$\text{Gain}(S,A) = 1$$

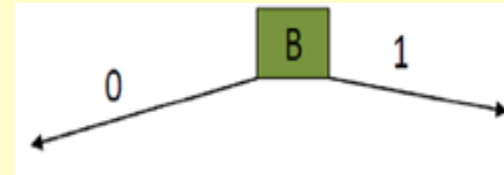
Choix de l'attribut de partitionnement

S: l'ensemble des instances
contient les lignes L1,L2,L3 et L4.

Classes={C1,C2}

Numéro de ligne	A	B	Classe
L1	0	1	C1
L2	0	0	C1
L3	1	1	C2
L4	1	0	C2

Partitionnement selon l'attribut B



S1

{L2==C1
L4==C2}

S2

{L1==C1
L3==C2}

$$E(S1) = -1 * (\text{portion}(C1) * \log_2(\text{portion}(C1)) + \text{portion}(C2) * \log_2(\text{portion}(C2)))$$

$$= -1 * (0.5 * \log_2(0.5) + 0.5 * \log_2(0.5)) = 1.0$$

$$E(S2) = -1 * (\text{portion}(C1) * \log_2(\text{portion}(C1)) + \text{portion}(C2) * \log_2(\text{portion}(C2)))$$

$$= -1 * (0.5 * \log_2(0.5) + 0.5 * \log_2(0.5)) = 1.0$$

$$\text{Gain}(S,B) = 1 - (\text{portion}(S1) * E(S1) + \text{portion}(S2) * E(S2))$$

$$= 1 - ((2/4) * E(S1) + (2/4) * E(S2)) = 1 - ((2/4) * (1.0) + (2/4) * (1.0)) = 0$$

Gain(S,B)=0 < Gain(S,A)=1 donc on doit faire le partitionnement selon A qui donne le plus grand gain.

Choix de l'attribut de partitionnement: **Indice de GINI**

- Si on fait un partitionnement de l'ensemble d'instances **S** selon l'attribut **Attribut** nous obtenons des sous ensembles d'instances **S_i** avec i compris entre 1 et le nombre k de valeurs distincts de l'attribut **Attribut**:

$$S = \bigcup_{i=1}^k S_i$$

- **L'indice de GINI** qui mesure le désordre est donné par la formule:

$$GINI(S) = 1 - \sum_{c \in \text{Classes}} (\text{portion}(c))^2$$

- **On prend l'attribut qui donne la plus petite mesure de désordre:**

$$\text{Mesure_de_Désordre}(S, \text{Attribut}) = \sum_{i=1}^k p_i * GINI(S_i)$$

Avec p_i est la portion du sous ensemble d'instances **S_i** par rapport à l'ensemble d'instances **S**:

$$p_i = \frac{\text{Nombre de lignes de } S_i}{\text{Nombre de lignes de } S}$$

Choix de l'attribut de partitionnement: **Indice de GINI**

S: l'ensemble des instances
contient les lignes L1,L2,L3 et L4.
Classes={C1,C2}

Numéro de ligne	A	B	Classe
L1	0	1	C1
L2	0	0	C1
L3	1	1	C2
L4	1	0	C2

$$GINI(S) = 1 - \sum_{i=1}^k (p_i)^2$$

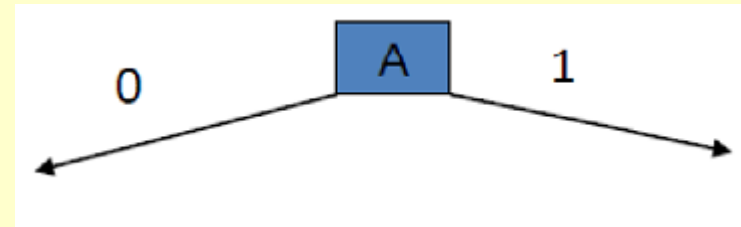
$$\begin{aligned} GINI(S) &= 1 - ((\text{portion}(C1))^2 + (\text{portion}(C2))^2) \\ &= 1 - ((0.5)^2 + (0.5)^2) \\ &= 0.5 \end{aligned}$$

Choix de l'attribut de partitionnement: Indice de GINI

S: l'ensemble des instances
contient les lignes L1,L2,L3 et L4.
Classes={C1,C2}

Numéro de ligne	A	B	Classe
L1	0	1	C1
L2	0	0	C1
L3	1	1	C2
L4	1	0	C2

Partitionnement selon l'attribut A



S1 contient
{L1==C1
L2==C1}

S2 contient
{L3==C2
L4==C2}

$$\text{GINI}(S1) = 1 - ((\text{portion}(C1) * (\text{portion}(C1))) = 0$$

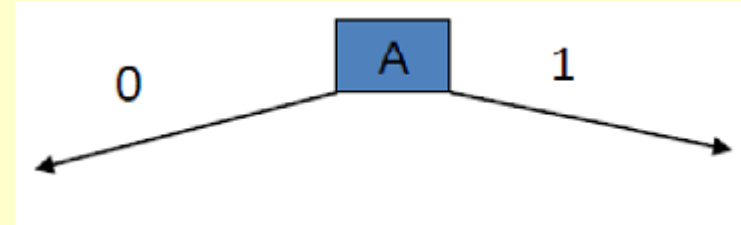
$$\text{GINI}(S2) = 1 - ((\text{portion}(C2) * (\text{portion}(C2))) = 0$$

$$\text{GINI}(S) = 1 - \sum_{i=1}^k (p_i)^2$$

Choix de l'attribut de partitionnement: **Indice de GINI**

Numéro de ligne	A	B	Classe
L1	0	1	C1
L2	0	0	C1
L3	1	1	C2
L4	1	0	C2

Partitionnement selon l'attribut A



S1 contient
 {L1===C1
 L2===C1}

S2 contient
 {L3===C2
 L4===C2}

$$GINI(S1) = 1 - ((\text{portion}(C1) * (\text{portion}(C1))) = 0$$

$$GINI(S2) = 1 - ((\text{portion}(C2) * (\text{portion}(C2))) = 0$$

$$\text{Mesure_de_Désordre}(S, \text{Attribut}) = \sum_{i=1}^k p_i * GINI(S_i)$$

$$\text{Mesure_de_desordre}(S, A) = 0.5 * 0 + 0.5 * 0 = 0$$

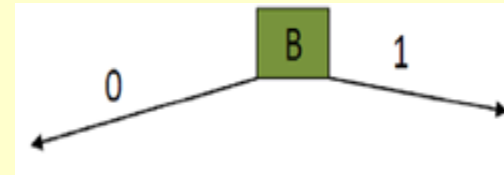
Choix de l'attribut de partitionnement: Indice de GINI

S: l'ensemble des instances
contient les lignes L1,L2,L3 et L4.

Classes={C1,C2}

Numéro de ligne	A	B	Classe
L1	0	1	C1
L2	0	0	C1
L3	1	1	C2
L4	1	0	C2

Partitionnement selon l'attribut B



S1

{L2==C1
L4==C2}

S2

{L1==C1
L3==C2}

$$GINI(S) = 1 - \sum_{i=1}^k (p_i)^2$$

$$\begin{aligned} GINI(S1) &= 1 - ((\text{portion}(C1) * (\text{portion}(C1) + (\text{portion}(C2) * (\text{portion}(C2))) = 1 - (0.5^2 + 0.5^2) = 0.5 \\ GINI(S2) &= 1 - ((\text{portion}(C1) * (\text{portion}(C1) + (\text{portion}(C2) * (\text{portion}(C2))) = 1 - (0.5^2 + 0.5^2) = 0.5 \end{aligned}$$

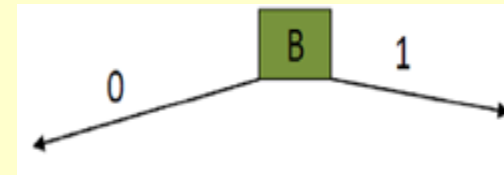
Choix de l'attribut de partitionnement: Indice de GINI

S: l'ensemble des instances
contient les lignes L1,L2,L3 et L4.

Classes={C1,C2}

Numéro de ligne	A	B	Classe
L1	0	1	C1
L2	0	0	C1
L3	1	1	C2
L4	1	0	C2

Partitionnement selon l'attribut B



S1

{L2==C1
L4==C2}

S2

{L1==C1
L3==C2}

$$\begin{aligned} \text{GINI}(S1) &= 1 - ((\text{portion}(C1) * (\text{portion}(C1) + (\text{portion}(C2) * (\text{portion}(C2))) = 1 - (0.5^2 + 0.5^2) = 0.5 \\ \text{GINI}(S2) &= 1 - ((\text{portion}(C1) * (\text{portion}(C1) + (\text{portion}(C2) * (\text{portion}(C2))) = 1 - (0.5^2 + 0.5^2) = 0.5 \end{aligned}$$

$$\text{Mesure_de_Désordre}(S, \text{Attribut}) = \sum_{i=1}^k p_i * \text{GINI}(S_i)$$

$$\text{Mesure_de_desordre}(S, B) = 0.5 * 0.5 + 0.5 * 0.5 = 0.5$$

Mesure_de_desordre(S,B) > Mesure_de_desordre(S,A) donc on doit faire le partitionnement selon A qui donne la plus petite mesure de désordre.

Choix de l'attribut de partitionnement: **Indice de GINI**

- Dans les exemples précédents nous avons considéré des **attributs catégoriels**: Douleur(poitrine, ailleurs), inanime(oui, non)
- **Attributs continus** (age, poids, taux de réussite,)
- Ici nous considérons que les attributs A,B et C sont continus

A, B, C, Y	
1, 1, 2, oui	L1
1, 4, 6, oui	L2
1, 1, 3, oui	L3
1, 5, 7, non	L4
2, 2, 3, non	L5
2, 6, 5, non	L6
2, 8, 9, non	L7
2, 3, 3, non	L8

Si on prend un attribut Note_du_Bac (avec deux chiffres après la virgule), si on a un data set qui contient cet attribut. Combien de sous ensemble obtenez vous si vous faites un regroupement selon cet attribut ?

Exemple: Note_du_Bac Note_du_Bac=[8,9.2,9.6,11.5,13,14,15,17,16.6,18,19]
Gain d'information si on utilise la formule précédente ? Est-il maximal ?

Matrice de confusion

Définition: Matrice de confusion

Soit un Data_test qui contient m échantillons (m exemples) dont les classes sont "Positif " ou " Négatif ". Avec un modèle de ML on classe ces m échantillons. Ces échantillons seront réparties en 4 groupes:

- Le premier groupe contient les V_p échantillons +(positifs) qui sont classés +
- Le deuxième contient les F_p échantillons + qui sont classés -
- Le troisième groupe contient les V_n échantillons - qui sont classés -
- Le quatrième groupe contient les F_n échantillons - qui sont classés +

Matrice de confusion :

	Classé +	Classé -
Exemple +	V_p	F_n
Exemple -	F_p	V_n

$$\text{Taux d'erreur} = 1 - \text{taux de réussite(accuracy)} = \frac{F_p + F_n}{F_p + F_n + V_p + V_n}$$

Matrice de confusion

CIFAR-10 Confusion Matrix

True Class	airplane	923	4	21	8	4	1	5	5	23	6	92.3%	7.7%
	automobile	5	972	2					1	5	15	97.2%	2.8%
	bird	26	2	892	30	13	8	17	5	4	3	89.2%	10.8%
	cat	12	4	32	826	24	48	30	12	5	7	82.6%	17.4%
	deer	5	1	28	24	898	13	14	14	2	1	89.8%	10.2%
	dog	7	2	28	111	18	801	13	17		3	80.1%	19.9%
	frog	5		16	27	3	4	943	1	1		94.3%	5.7%
	horse	9	1	14	13	22	17	3	915	2	4	91.5%	8.5%
	ship	37	10	4	4		1	2	1	931	10	93.1%	6.9%
	truck	20	39	3	3			2	1	9	923	92.3%	7.7%
		88.0%	93.9%	85.8%	79.0%	91.4%	89.7%	91.6%	94.1%	94.8%	95.0%		
		12.0%	6.1%	14.2%	21.0%	8.6%	10.3%	8.4%	5.9%	5.2%	5.0%		
		airplane	automobile	bird	cat	deer	dog	frog	horse	ship	truck		
Predicted Class													

Les arbres de décision

```
1 import numpy as np
2 import pandas as pd
3 from sklearn.metrics import confusion_matrix
4 from sklearn.tree import DecisionTreeClassifier
5 from sklearn.metrics import *
```

```
1 data = pd.read_csv('D:\\balance-scale1.data').as_matrix()
2 # Printing the dataset shape
3 print ("Dataset Length: ", len(data))
4 print ("Dataset Shape: ", data.shape)
5 print(data)
```

```
[[1 1 2 'oui']
 [1 4 6 'oui']
 [1 1 3 'oui']
 [1 5 7 'non']
 [2 2 3 'non']
 [2 6 5 'non']
 [2 8 9 'non']
 [2 3 3 'non']
 [2 6 9 'non']
 [2 8 9 'non']
 [1 8 7 'oui']
 [1 9 6 'oui']
 [1 0 4 'oui']]
```

Les arbres de décision

```
1 X = data[:,0:3]; Y = data[:,3] ; X_test=data[8:,:0:3]; y_test=data[8:,:3]
2 clf_gini = DecisionTreeClassifier(criterion = "gini")
3 clf_gini.fit(X, Y)
4 clf_entropy = DecisionTreeClassifier(criterion = "entropy")
5 clf_entropy.fit(X, Y)
6 print("Results Using Gini Index:")
7 y_pred_gini = clf_gini.predict(X_test)
8 print("Confusion Matrix: ",confusion_matrix(y_test, y_pred_gini))
9 print(X_test,y_test,y_pred_gini)
10 print ("Accuracy : ",accuracy_score(y_test,y_pred_gini)*100)
11 print("Results Using Entropy:")
12 y_pred_entropy = clf_entropy.predict(X_test)
13 print("Confusion Matrix: ",confusion_matrix(y_test, y_pred_entropy))
14 print ("Accuracy : ",accuracy_score(y_test,y_pred_entropy)*100)
```

Les arbres de décision

Results Using Gini Index:

Confusion Matrix:

	prédite non	prédite oui
Les non	2	0
Les oui	1	2

Classes correctes données par l'expert ['non' 'non' 'oui' 'oui' 'oui']

Classes prédites par AD-Gini ['non' 'non' 'non' 'oui' 'oui']

Accuracy : 80.0

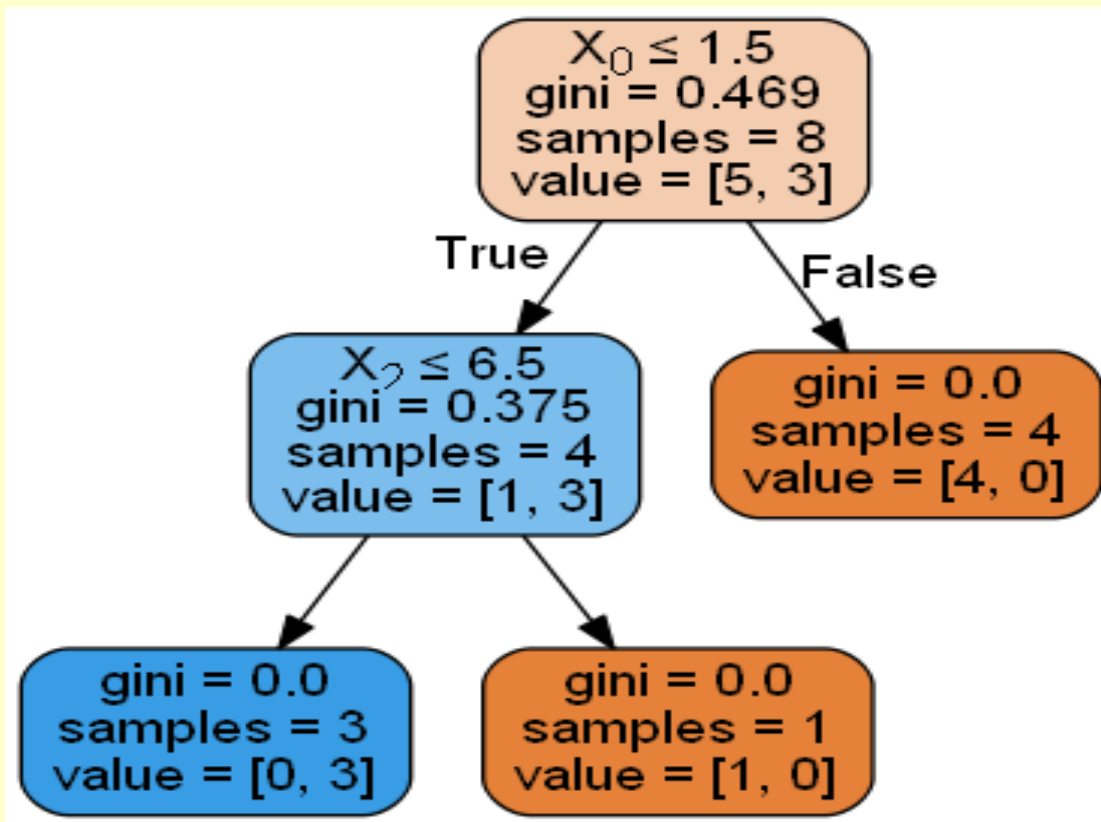
Results Using Entropy:

Confusion Matrix: $\begin{bmatrix} 2 & 0 \\ 2 & 1 \end{bmatrix}$

Accuracy : 60.0

Le nombre d'exemples qui sont bien classés est égal à la trace de la matrice de confusion: $\text{trace}(\text{MC}) = \text{somme de la diagonale (MC)}$

Les arbres de décision

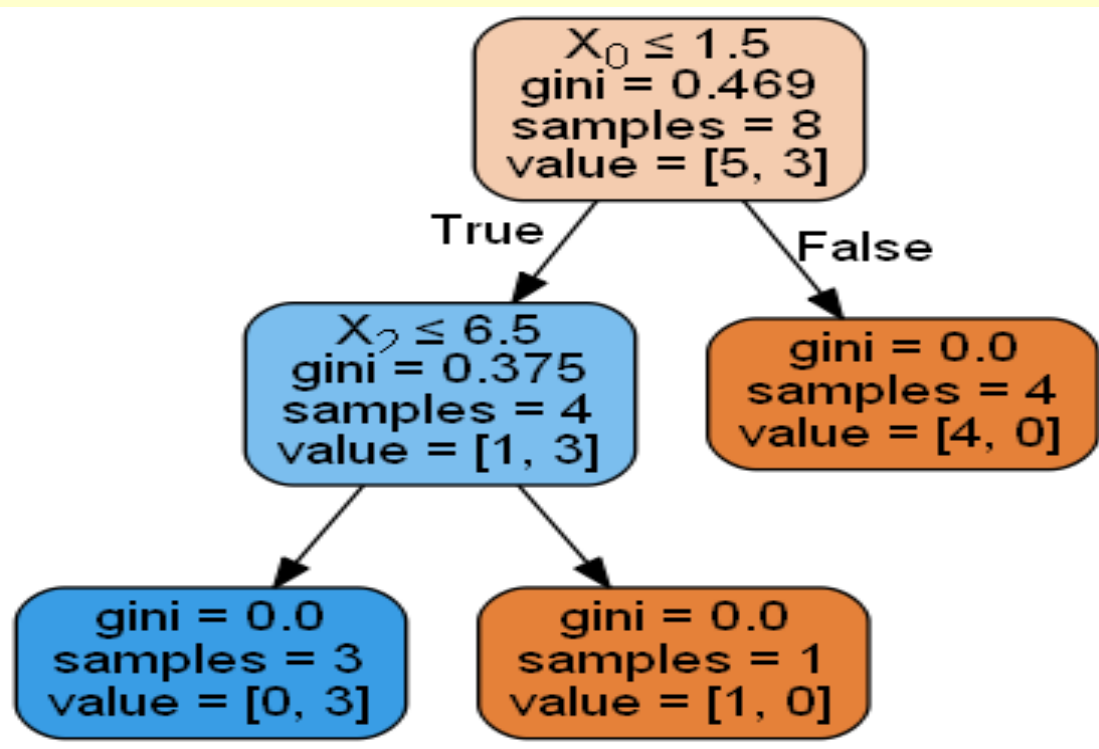


A,B,C,Y	
1,1,2,oui	L1
1,4,6,oui	L2
1,1,3,oui	L3
1,5,7,non	L4
2,2,3,non	L5
2,6,5,non	L6
2,8,9,non	L7
2,3,3,non	L8

Le data train $S=\{L1,L2,L3,L4,L5,L6,L7,L8\}$ contient 8 échantillons (samples=8)

5 échantillons ont la classe "non" et 3 échantillons ont la classe "oui" (value=[5,3])

Les arbres de décision



A,B,C,Y				
1	1	2	oui	L1
1	4	6	oui	L2
1	1	3	oui	L3
1	5	7	non	L4
2	2	3	non	L5
2	6	5	non	L6
2	8	9	non	L7
2	3	3	non	L8

S1 (rows L1-L4) and **S2** (rows L5-L8) are indicated by red labels.

La condition $X_0 \leq 1.5$ (ici X_0 =colonne A) permet de diviser l'ensemble $S=\{L1,L2,L3,L4,L5,L6,L7,L8\}$ en deux ensembles de cardinal 4:

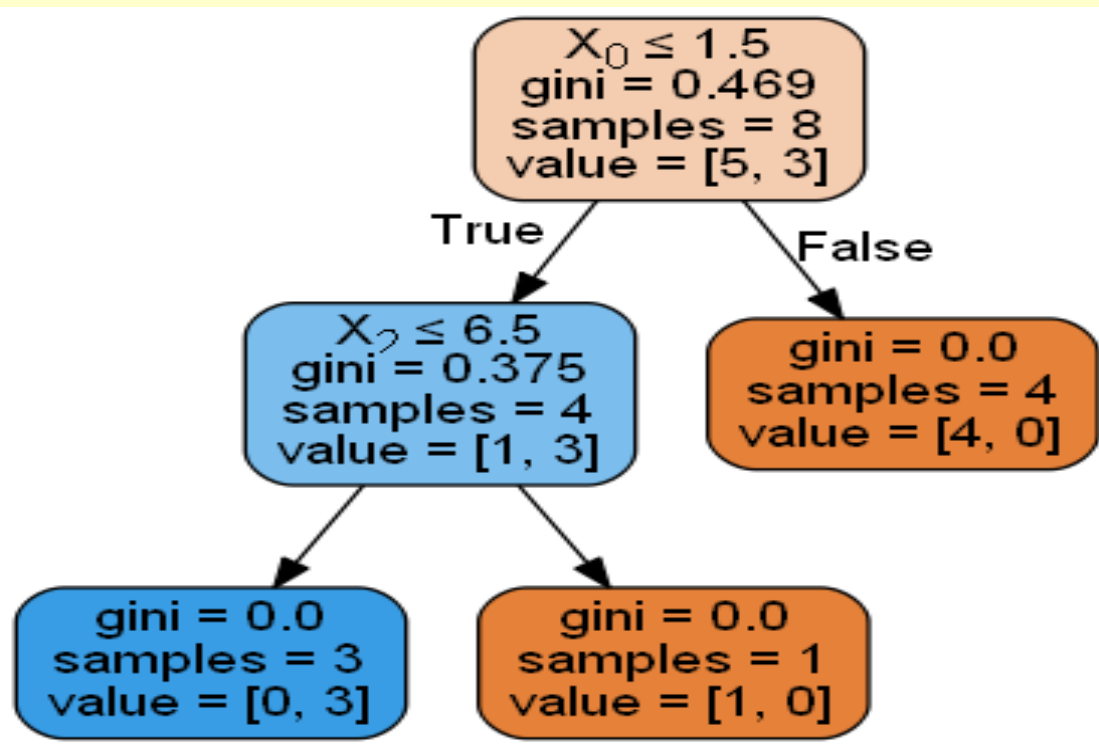
$S1=\{L1,L2,L3,L4\}$ et $S2=\{L5,L6,L7,L8\}$

Les échantillons de S1 sont ceux qui vérifient la condition:

$X_0 \leq 1.5$ (le mot True), dans S1: 3 échantillons ont la classe "oui" et

1 échantillon a la classe "non", donc l'ensemble S1 n'est pas homogène !

Les arbres de décision



A,B,C,Y				
1	1	2	oui	L1
1	4	6	oui	L2
1	1	3	oui	L3
1	5	7	non	L4

S1

La condition $X_2 \leq 6.5$ (ici X_2 =colonne C) permet de diviser l'ensemble $S1=\{L1,L2,L3,L4\}$ en deux ensembles :

$S3=\{L1,L2,L3\}$ et $S4=\{L4\}$

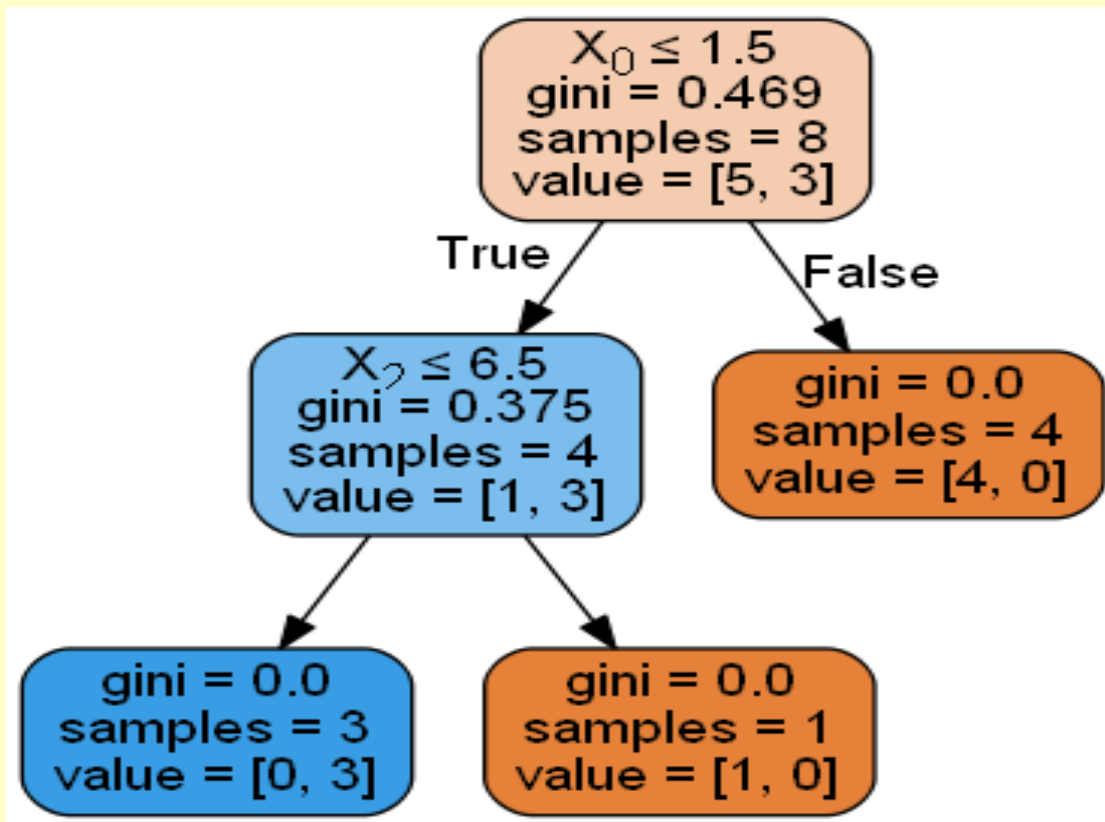
Les échantillons de $S3$ sont ceux qui vérifient la condition: $X_2 \leq 6.5$

dans $S3$, tous les échantillons ont la classe "oui" et $S3$ est homogène

Les échantillons de $S4$ sont ceux qui ne vérifient pas la condition: $X_2 \leq 6.5$

dans $S4$, tous les échantillons ont la classe "non" et $S4$ est homogène

Les arbres de décision



A,B,C,Y				
1	1	2	oui	L1
1	4	6	oui	L2
1	1	3	oui	L3
1	5	7	non	L4
2	2	3	non	L5
2	6	5	non	L6
2	8	9	non	L7
2	3	3	non	L8

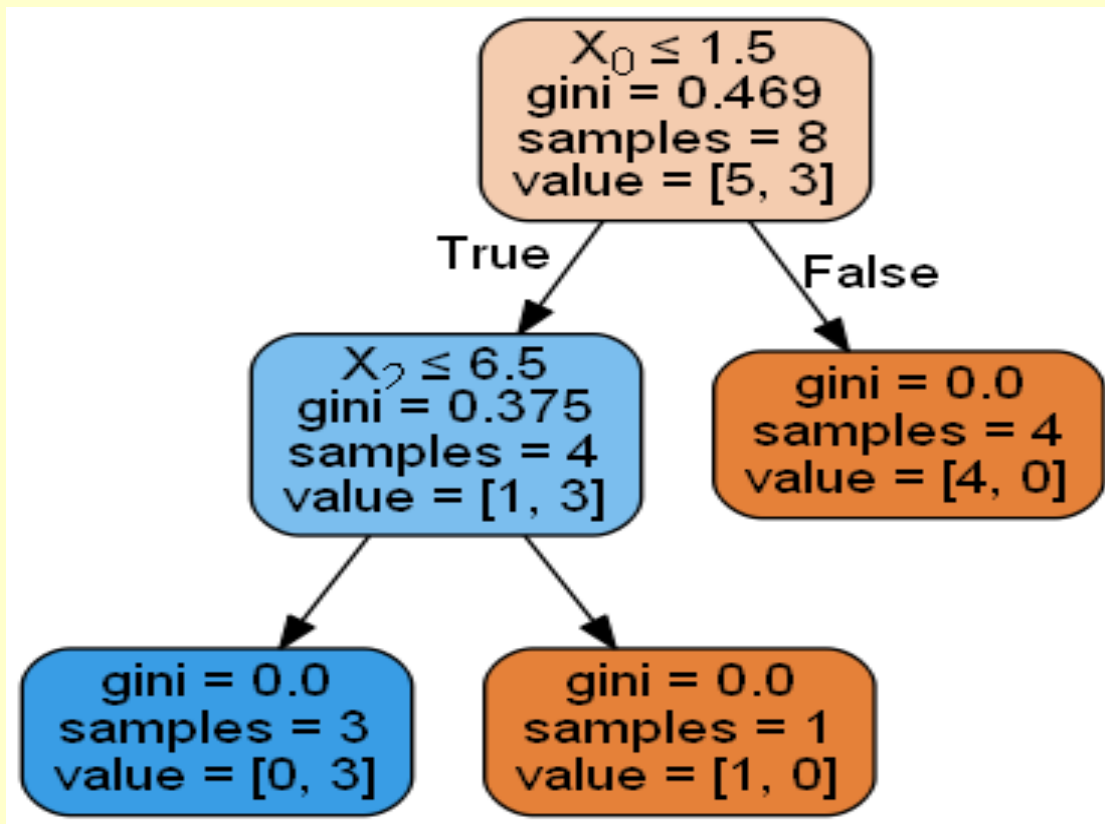
S1 points to rows L1, L2, L3, L4. **S2** points to rows L5, L6, L7, L8.

La condition $X_0 \leq 1.5$ (ici X_0 =colonne A) permet de diviser l'ensemble $S=\{L1,L2,L3,L4,L5,L6,L7,L8\}$ en deux ensembles de cardinal 4:

$S1=\{L1,L2,L3,L4\}$ et $S2=\{L5,L6,L7,L8\}$

Les échantillons de S2 sont ceux qui ne vérifient pas la condition: $X_0 \leq 1.5$ (le mot False) et ils sont tous dans la classe "non"

Les arbres de décision

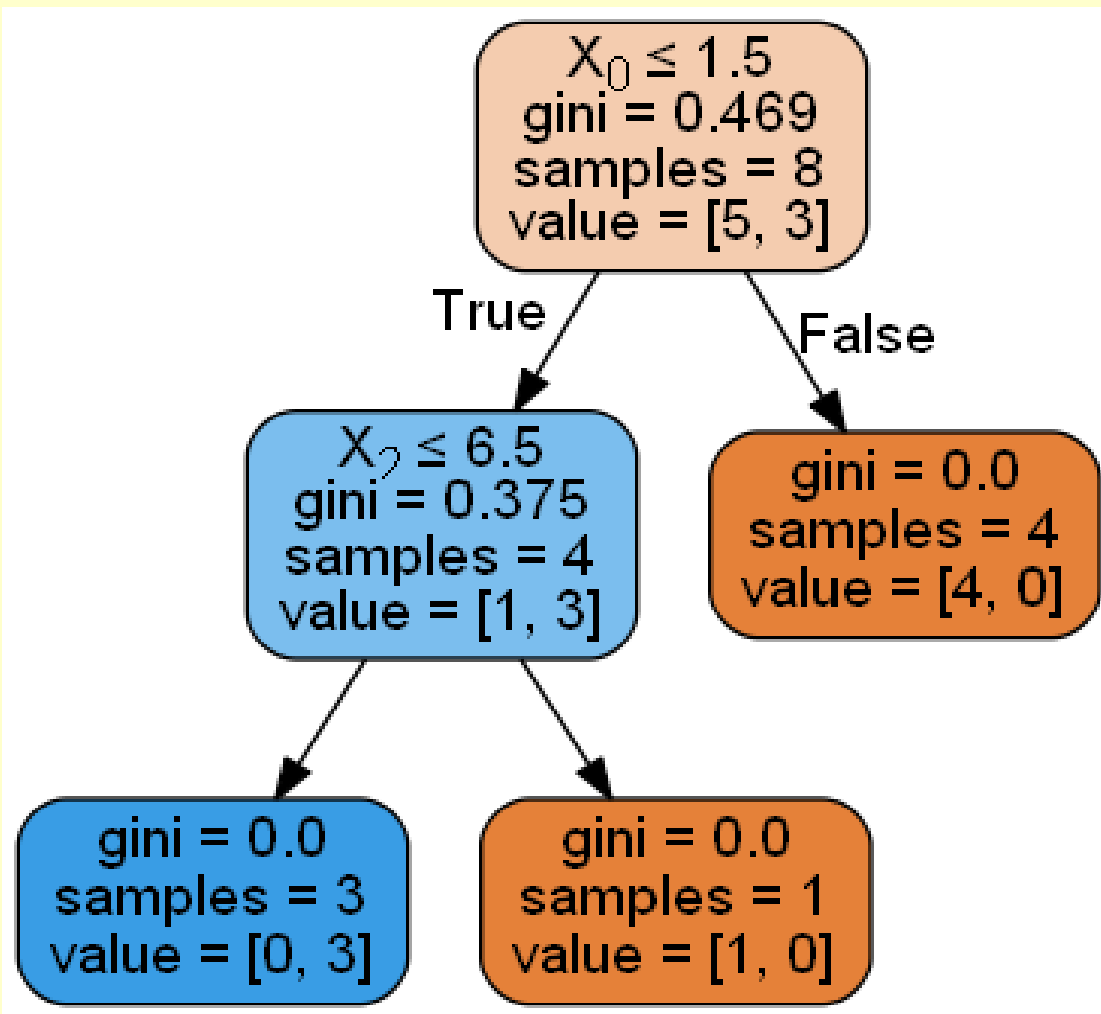


A,B,C,Y				
1	1	2	oui	L1
1	4	6	oui	L2
1	1	3	oui	L3
1	5	7	non	L4
2	2	3	non	L5
2	6	5	non	L6
2	8	9	non	L7
2	3	3	non	L8

S1 (rows L1-L4) and **S2** (rows L5-L8) are indicated by red text to the left of the table.

Les échantillons de S2 sont ceux qui ne vérifient pas la condition: $X_0 \leq 1.5$ (le mot False) et ils sont tous dans la classe "non" donc S2 est homogène,

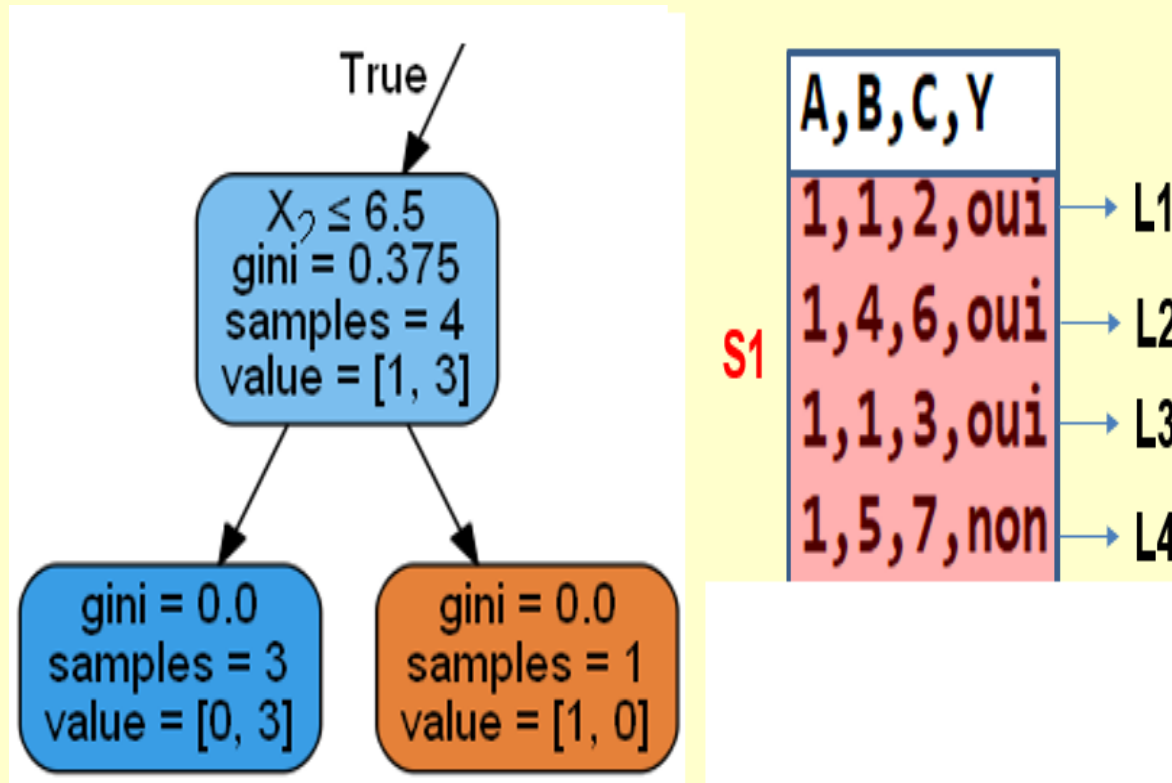
Les arbres de décision



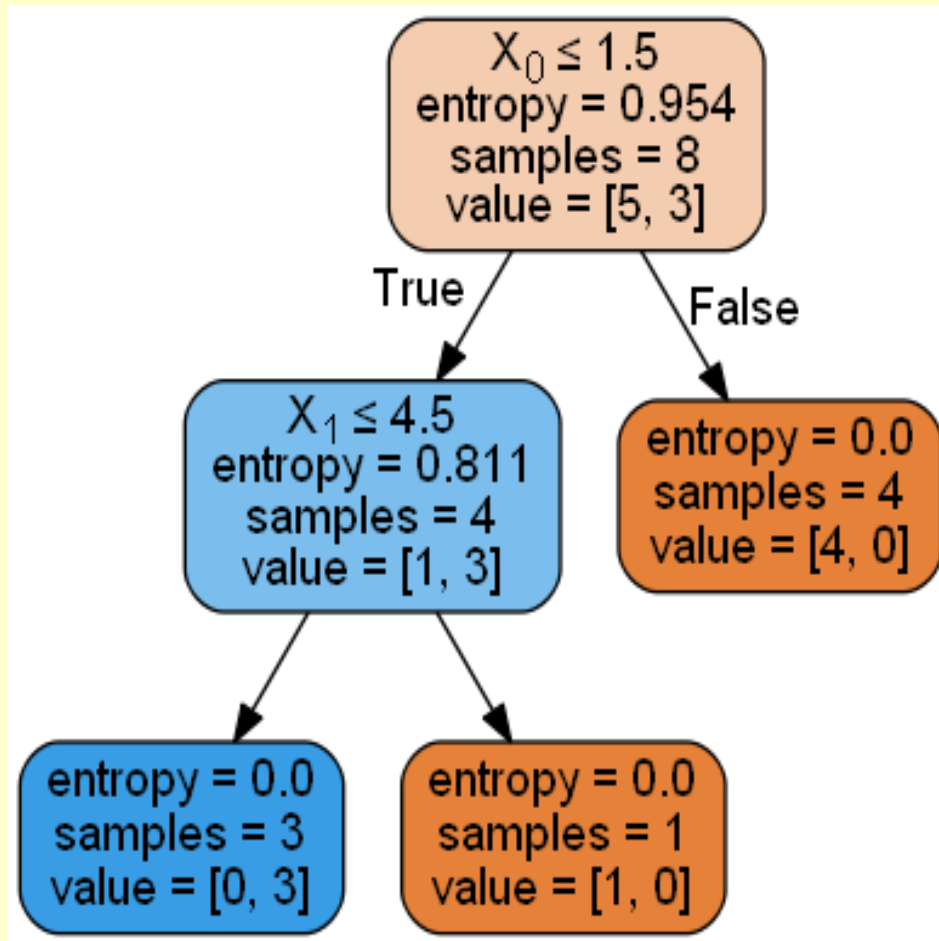
A, B, C, Y				
1	1	2	oui	L1
1	4	6	oui	L2
1	1	3	oui	L3
1	5	7	non	L4
2	2	3	non	L5
2	6	5	non	L6
2	8	9	non	L7
2	3	3	non	L8

S1 (rows L1-L4) and **S2** (rows L5-L8) are indicated by red text.

Les arbres de décision



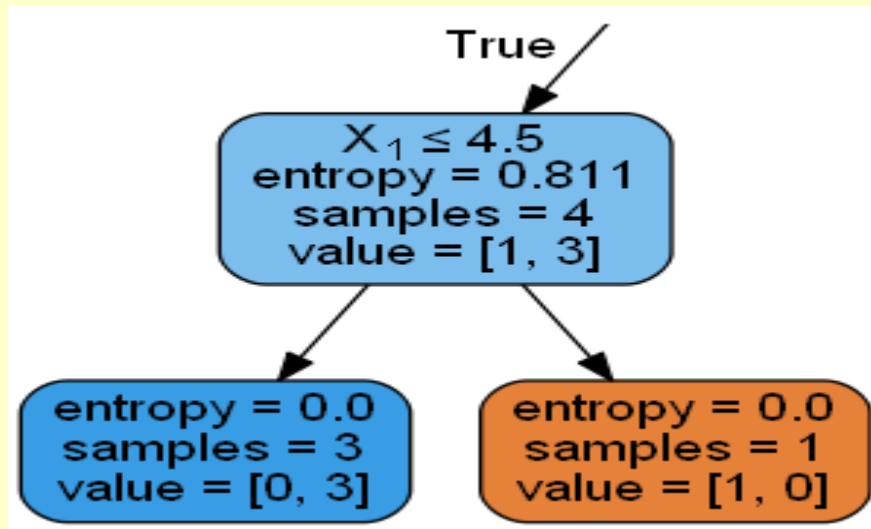
Les arbres de décision



A, B, C, Y	
1, 1, 2, oui	L1
1, 4, 6, oui	L2
1, 1, 3, oui	L3
1, 5, 7, non	L4
2, 2, 3, non	L5
2, 6, 5, non	L6
2, 8, 9, non	L7
2, 3, 3, non	L8

Tp 22

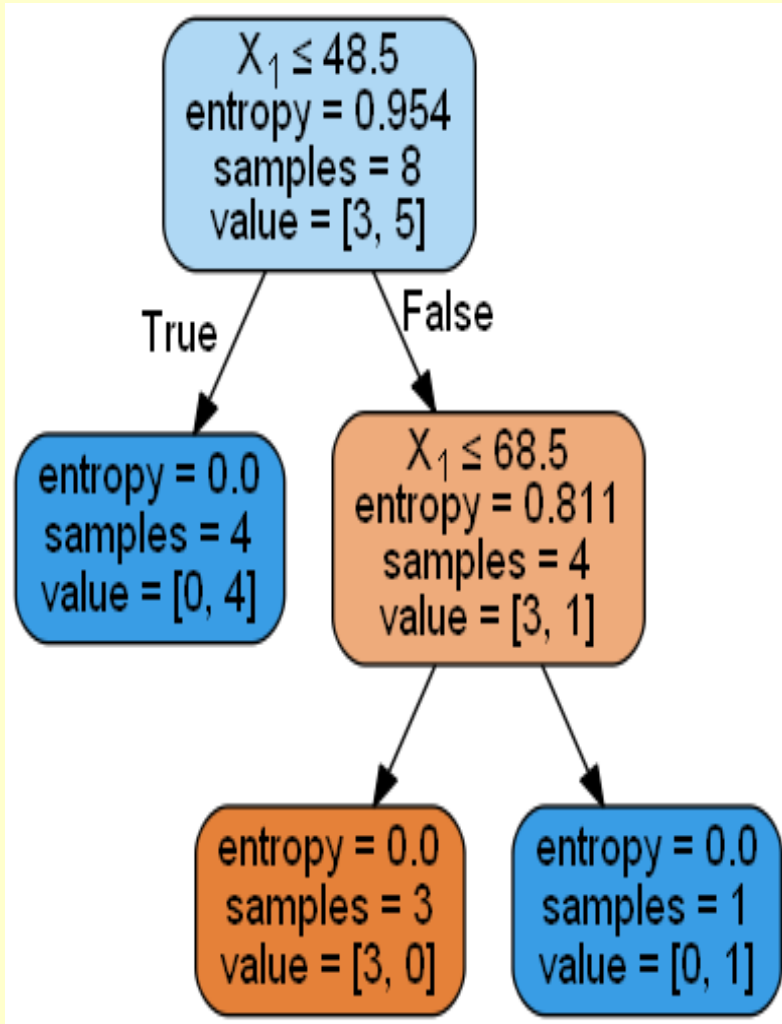
Les arbres de décision



S1

A, B, C, Y	
1, 1, 2, oui	→ L1
1, 4, 6, oui	→ L2
1, 1, 3, oui	→ L3
1, 5, 7, non	→ L4

Les arbres de décision



['poitrine' 45 'oui' 'oui']	——	[1 45 1 'oui']
['ailleurs' 25 'oui' 'oui']	——	[2 25 1 'oui']
['poitrine' 35 'non' 'oui']	——	[1 35 0 'oui']
['Tete' 70 'oui' 'oui']	——	[0 70 1 'oui']
['ailleurs' 34 'non' 'oui']	——	[2 34 0 'oui']
['poitrine' 60 'non' 'non']	——	[1 60 0 'non']
['ailleurs' 67 'non' 'non']	——	[2 67 0 'non']
['Tete' 52 'oui' 'non']	——	[0 52 1 'non']

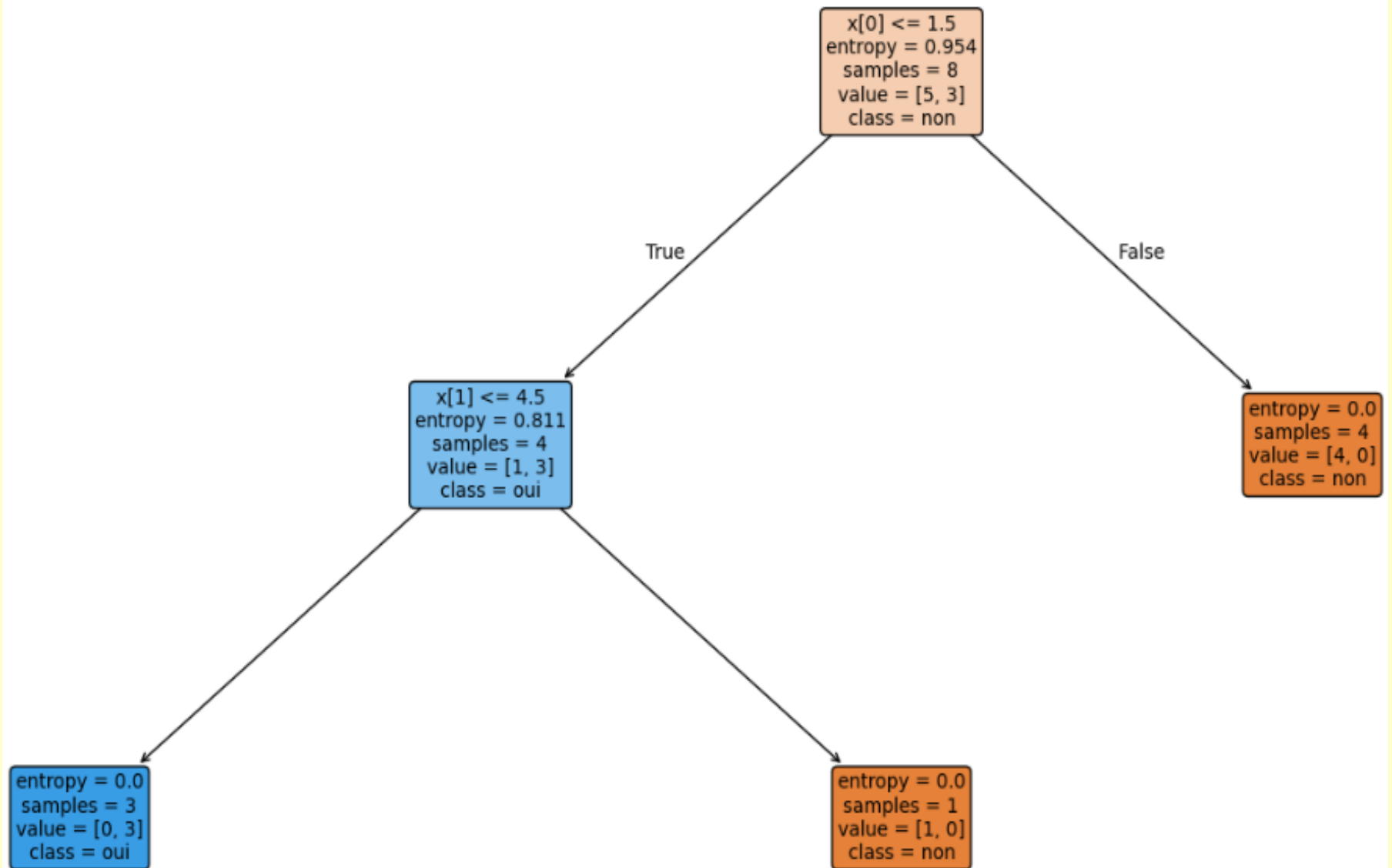
```
import matplotlib.pyplot as plt
from sklearn.tree import plot_tree

plt.figure(figsize=(20, 10))

# If you have feature names and class names
feature_names = X.columns.tolist() if hasattr(X, 'columns') else None
class_names = [str(c) for c in clf_entropy.classes_] if hasattr(clf_entropy, 'classes_') else None

plot_tree(clf_entropy,
          filled=True,
          rounded=True,
          feature_names=feature_names,
          class_names=class_names,
          fontsize=10)

plt.savefig('entropy_tree.png', dpi=600, bbox_inches='tight', facecolor='white')
plt.show()
```



Les arbres de décision: Credit Scoring

Le processus d'acceptation d'un crédit à la consommation doit être très rapide en particulier lorsque les clients ont besoin d'un crédit en magasin au moment de leur achat (exemple : paiement en trois fois sans frais, etc.).

Depuis très longtemps, les banques et les organismes de crédit ont développé des méthodes automatiques et instantanées de notation de la solvabilité des clients et de leur capacité à rembourser le crédit. Ces outils sont très utiles pour une acceptation rapide d'octroi de crédit, mais également pour déterminer le montant de l'enveloppe d'autorisation de crédit ou pour effectuer le suivi du risque des clients et des portefeuilles de prêts.

Le score de crédit est une fonction qui attribue une valeur de qualité de crédit à un client ou un prêt en fonction de variables explicatives telles que le ratio d'endettement de l'emprunteur, son comportement de compte ou tout autre grandeur qui est corrélée au défaut de l'emprunteur.

Les arbres de décision

```
df = pd.read_csv("D:\\CreditScoring.csv")
print(df.isnull().sum())
#remplir les cases vides (NaN) par la moyenne
df.fillna(value=df.mean(),inplace=True)
print("+++++++ 1 ++++++")
print(df.isnull().sum())
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score
# removing the features BAD, JOB, REASON
x_basic = df.drop(columns=["BAD", "JOB", "REASON"]); y = df["BAD"]
print("+++++++ 2 ++++++")
print(x_basic.isnull().sum())
x_basic_tr, x_basic_te, y_tr, y_te = train_test_split(\
    x_basic, y, test_size = .33, random_state=1)
from sklearn.tree import DecisionTreeClassifier
model = DecisionTreeClassifier()
model.fit(x_basic_tr, y_tr)
y_pre = model.predict(x_basic_te)
a = accuracy_score(y_te, y_pre)
print("accuracy score : ", a)
```

Les arbres de décision

		++++ 1 +++++	+++++ 2 ++++++
BAD	0	BAD	0
LOAN	0	LOAN	0
MORTDUE	518	MORTDUE	0
VALUE	112	VALUE	0
REASON	252	REASON	252
JOB	279	JOB	279
YOJ	515	YOJ	0
DEROG	708	DEROG	0
DELINQ	580	DELINQ	0
CLAGE	308	CLAGE	0
NINQ	510	NINQ	0
CLNO	222	CLNO	0
DEBTINC	1267	DEBTINC	0
dtype: int64			dtype: int64
			accuracy score : 0.8647686832740213

Les arbres de décision

```
1 X_test=np.array(x_basic_te); y_test=np.array(y_te)
2 print(X_test[127])
3 print(model.predict([X_test[127]]),end=' '); print(y_test[127])
```

```
[1.57000000e+04  8.77540000e+04  1.03045000e+05  2.10000000e+01
 0.00000000e+00  0.00000000e+00  1.47450177e+02  4.00000000e+00
 2.30000000e+01  2.91817426e+01]
[0]  0
```

```
1 print(X_test[0])
2 print(model.predict([X_test[0]]),end=' '); print(y_test[0])
```

```
[2.16000000e+04  1.54991000e+05  1.01776049e+05  8.92226814e+00
 2.00000000e+00  4.00000000e+00  1.65600830e+02  0.00000000e+00
 4.30000000e+01  3.80149173e+01]
[1]  1
```

Les arbres de décision pour la regression

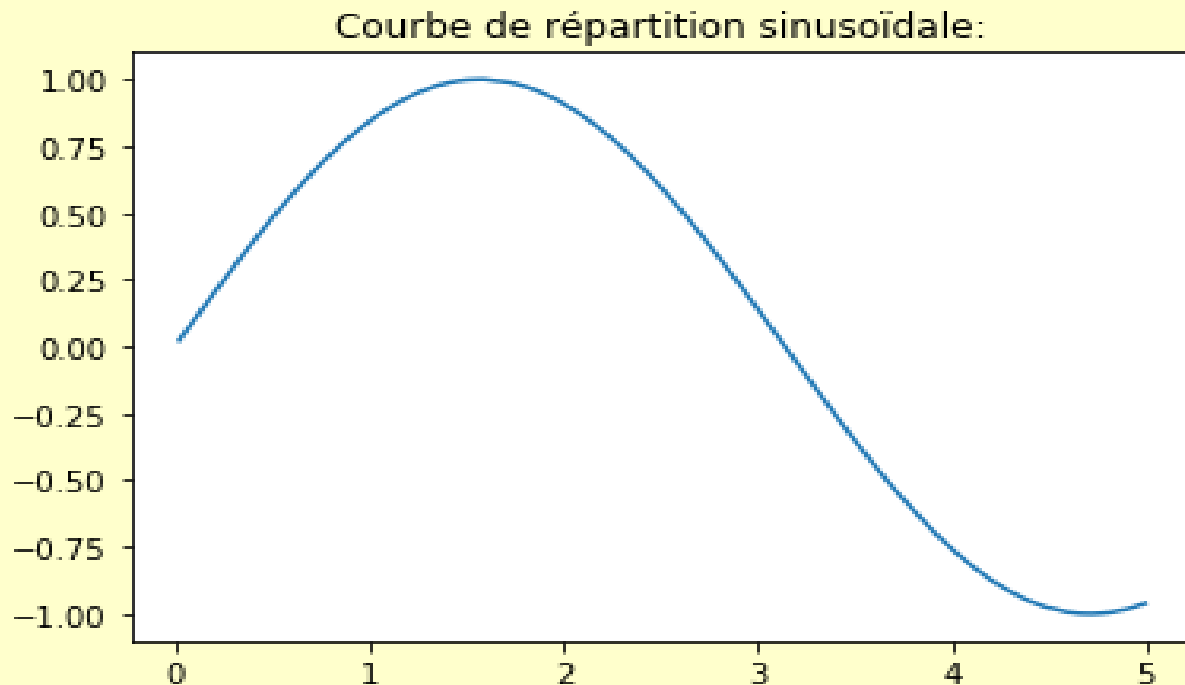
Pour la régression avec les arbres de décision, scikit-learn offre la classe `DecisionTreeRegressor`. Comme pour la classification, la méthode `fit(...)` prend en entrée le paramètre `X` (attributs des observations). Attention : les `y` ne sont pas des étiquettes de classes mais des valeurs réelles.

```
1  from sklearn import tree
2  X = [[0, 0], [2, 2]]
3  y = [0.5, 2.5]
4  clf = tree.DecisionTreeRegressor()
5  clf = clf.fit(X, y)
6  clf.predict([[1, 1]])
```

```
array([0.5])
```

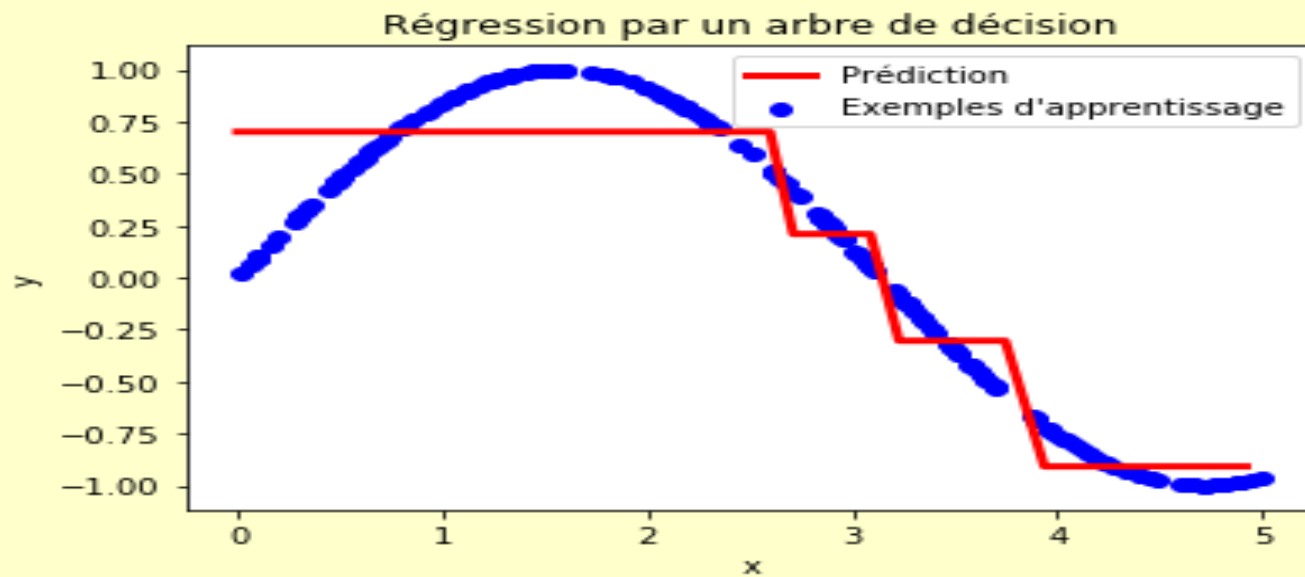
Les arbres de décision pour la regression

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3 from sklearn.tree import DecisionTreeRegressor
4 # Créer Les données d'apprentissage
5 np.random.seed(0)
6 X = np.sort(5 * np.random.rand(200, 1), axis=0)
7 y = np.sin(X)
8 plt.plot(X, y)
9 plt.title("Courbe de répartition sinusoïdale:")
```



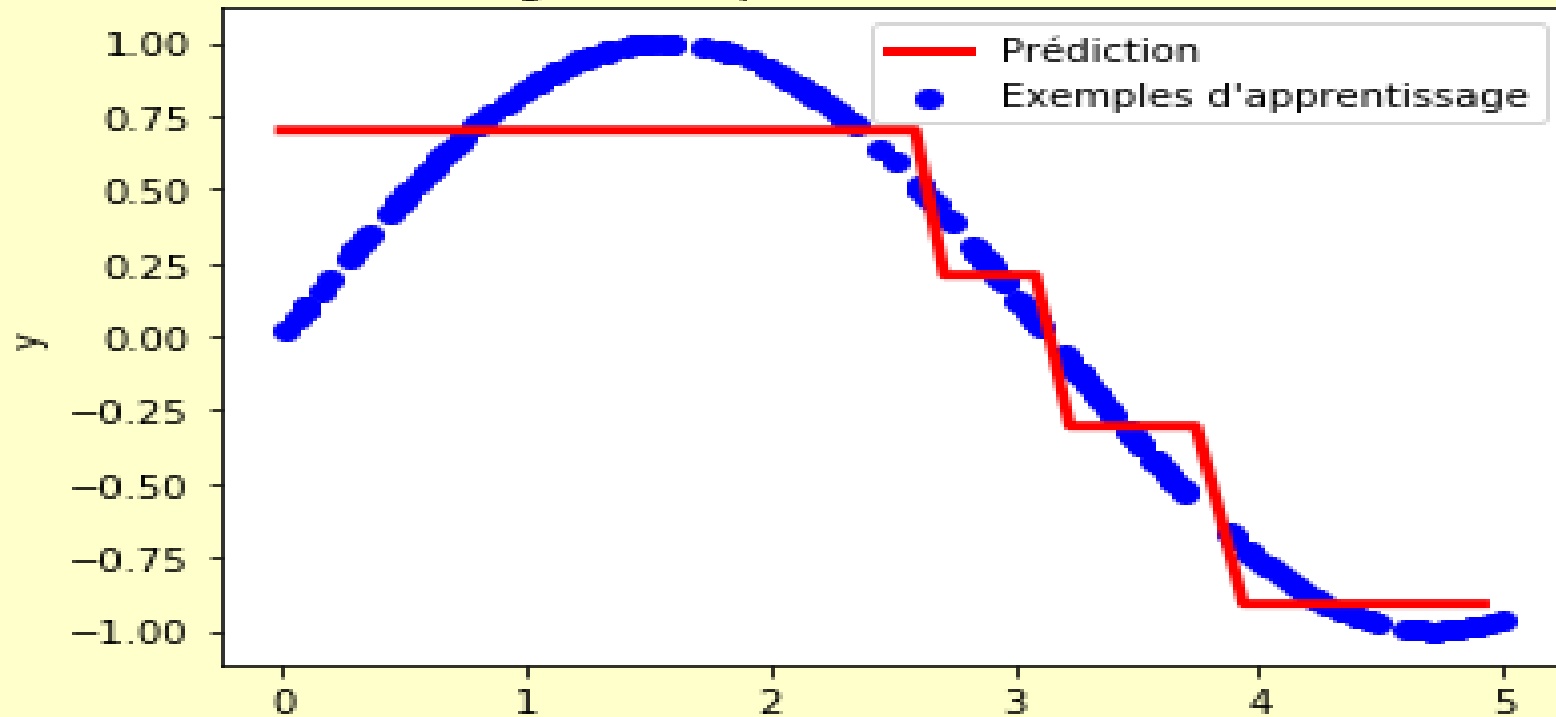
Les arbres de décision pour la regression

```
from sklearn.metrics import mean_squared_error
reg = DecisionTreeRegressor(max_depth=2)
reg.fit(X, y)
X_test = np.sort(5 * np.random.rand(80, 1), axis=0)
y_predicted = reg.predict(X_test)
y_test = np.sin(X_test)
plt.figure()
plt.scatter(X, y, color="b", label="Exemples d'apprentissage")
plt.plot(X_test, y_predicted, color="r", label="Prédiction", linewidth=3)
plt.xlabel("x"); plt.ylabel("y")
plt.title("Régression par un arbre de décision")
plt.legend(); plt.show()
a=mean_squared_error(y_test, y_predicted); print("mean_squared_error: ",a)
```



Les arbres de décision pour la regression

Régression par un arbre de décision



Erreur quadratique moyenne:

mean_squared_error: 0.04055033011763535

$$\underbrace{\text{MES}(\text{node})}_{\text{mean-squared-error}} = \frac{1}{N_{\text{node}}} \sum_{i \in \text{node}} (y^{(i)} - \hat{y}_{\text{node}})^2$$

```
1 reg.predict([[0.5],[1.5],[2.5],[2.8],[3.4],[4.5],[4.8]])
```

```
array([ 0.70112934,  0.70112934,  0.70112934,  0.20958427, -0.30366824,  
       -0.90809362, -0.90809362])
```

Les arbres de décision pour la regression

- Les arbres de décision ont plusieurs **avantages** qui les rendent intéressants dans des contextes où il est utile de comprendre la séquence de décisions prise par le modèle :
- Ils sont simples à comprendre et à visualiser.
- Le coût d'utilisation des arbres est logarithmique.
- Ils sont capables d'utiliser des données catégorielles et numériques.
- Ils sont capables de traiter des problèmes multi-classes.
- Modèle en boîte blanche : le résultat est facile à conceptualiser et à visualiser.

Les arbres de décision pour la regression

- Les arbres de décision ont plusieurs **avantages** qui les rendent intéressants dans des contextes où il est utile de comprendre la séquence de décisions prise par le modèle :
- Ils sont simples à comprendre et à visualiser.
- Le coût d'utilisation des arbres est logarithmique.
- Ils sont capables d'utiliser des données catégorielles et numériques.
- Ils sont capables de traiter des problèmes multi-classe.
- Modèle en boîte blanche : le résultat est facile à conceptualiser et à visualiser.

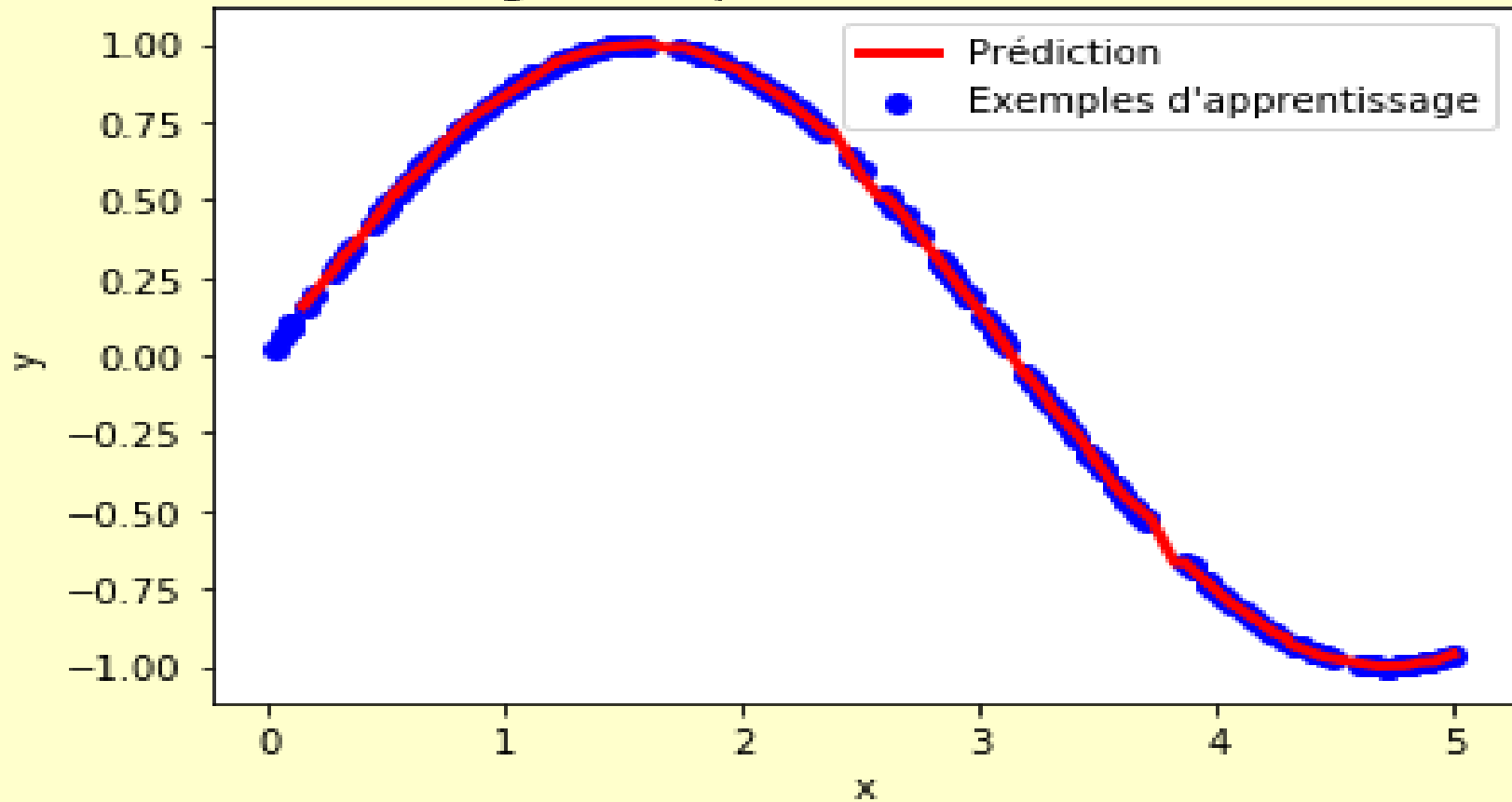
Sur-apprentissage : parfois les arbres générés sont trop complexes et généralisent mal. Choisir des bonnes valeurs pour les paramètres profondeur maximale (**max_depth**) et nombre minimal d'exemples par feuille (**min_samples_leaf**) permet d'éviter ce problème.

Les arbres de décision pour la regression

```
reg = DecisionTreeRegressor(max_depth=200)
reg.fit(X, y)
X_test = np.sort(5 * np.random.rand(80, 1), axis=0)
y_predicted = reg.predict(X_test)
y_test = np.sin(X_test)
plt.figure()
plt.scatter(X, y, color="b", label="Exemples d'apprentissage")
plt.plot(X_test, y_predicted, color="r", label="Prédiction", linewidth=3)
plt.xlabel("x"); plt.ylabel("y")
plt.title("Régression par un arbre de décision")
plt.legend() ; plt.show()
a=mean_squared_error(y_test, y_predicted); print("mean_squared_error: ",a)
```

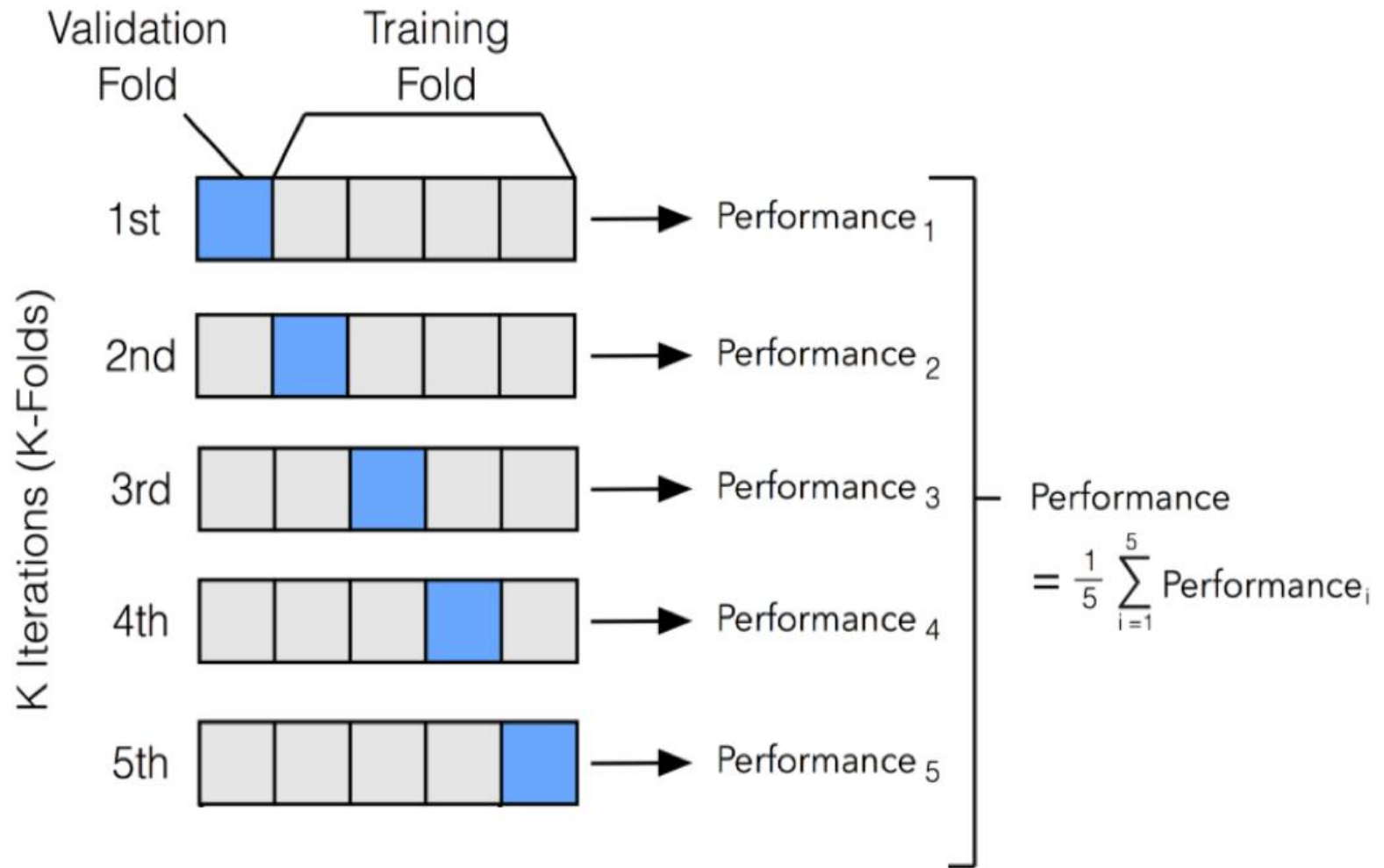
Les arbres de décision pour la regression

Régression par un arbre de décision



mean_squared_error: 7.244809185331687e-05

Cross validation



KNN: k Nearest Neighbors

```
2 from sklearn.neighbors import KNeighborsClassifier
3 from sklearn.model_selection import GridSearchCV, train_test_split
4 data=pandas.read_csv("d:\\images_chiffres_codees_niveau_de_gris.csv")
5 data=np.array(data); x=data[0:1000,1:]; y=data[0:1000,0];
6 x_train,x_test,y_train,y_test=train_test_split(x,y,test_size=0.2,
7                                               random_state=0)
8 model=KNeighborsClassifier(n_neighbors=5,metric='manhattan')
9 model.fit(x_train,y_train); score=model.score(x_test, y_test)
10 print("score sur test-set par k=5 --> "+str(score*100)+'%')
11 distances=['euclidean','manhattan'] ; valeurs_de_k=np.arange(1,8)
12 parametres_grid={"n_neighbors":valeurs_de_k, 'metric':distances}
13 model=KNeighborsClassifier()
14 grid=GridSearchCV(model,parametres_grid,cv=3,scoring='accuracy')
15 grid.fit(x_train,y_train); print("best parameters:",grid.best_params_)
16 model=grid.best_estimator_ ; model.fit(x_train,y_train)
17 score=model.score(x_test, y_test)
18 print("score sur test-set-paramètres automatiques="+str(score*100)+'%')
```

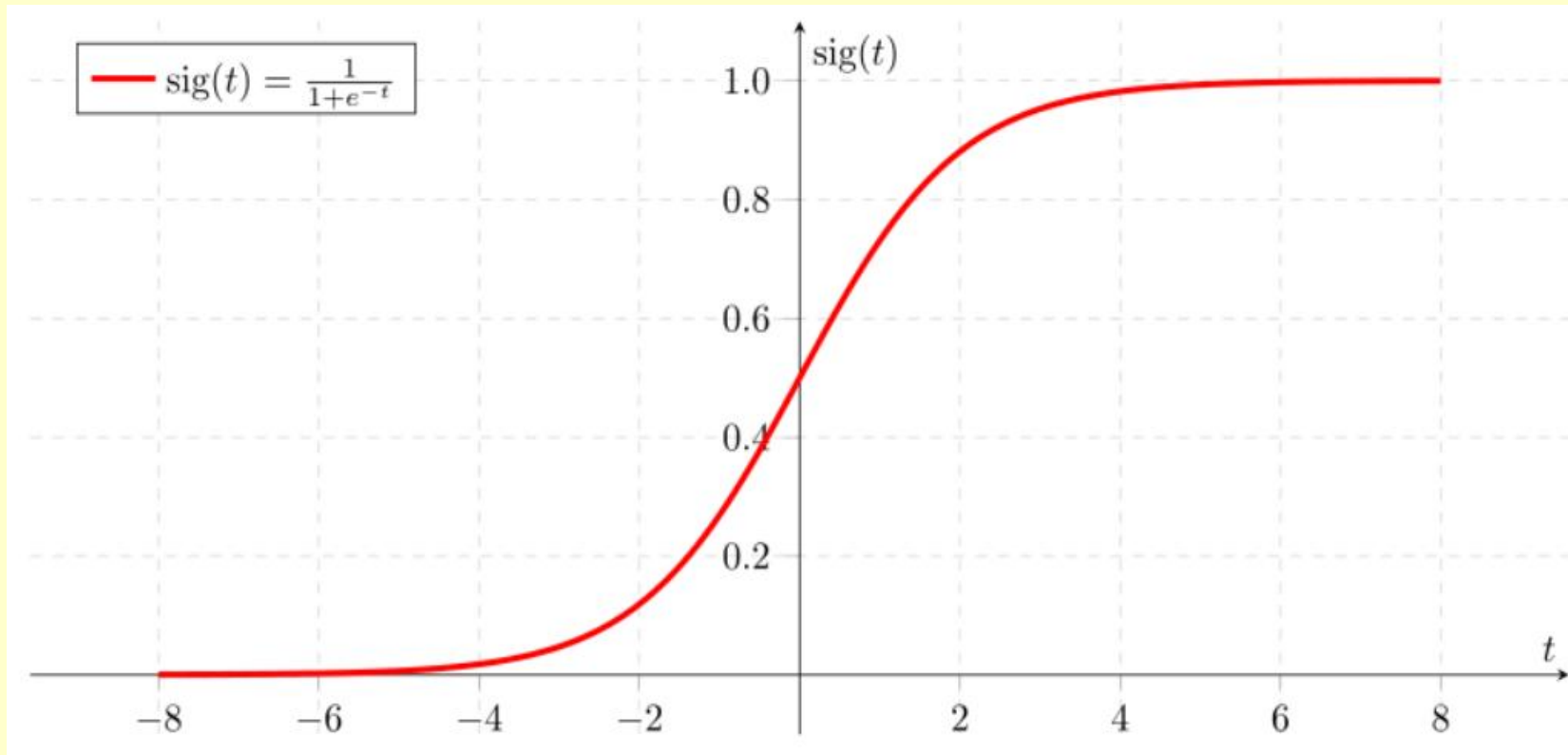
score sur test-set par k=5 --> 85.0%

best parameters: {'metric': 'euclidean', 'n_neighbors': 5}

score sur test-set-paramètres automatiques=88.0%

Sigmoid Function : La fonction pour la régression Logistique

la fonction **sigmoid** (**Sigmoid Function**) produit des valeurs comprises entre 0 et 1.



$$\text{sigmoide}(x) = \frac{1}{1 + e^{-x}}$$

Sigmoid Function : La fonction pour la régression Logistique

- La régression logistique est utilisée pour trouver la probabilité d'un événement. On veut déterminer le succès ou l'échec d'un événement.
- Largement utilisé pour les problèmes de classification
- Ne nécessite pas de relation linéaire entre les variables dépendantes et indépendantes.
- La régression logistique permettra de répondre à des problèmes comme :

Est-ce que le client est solvable pour lui accorder un crédit ou non?

Est-ce que la tumeur diagnostiquée est bénigne ou maligne ?

Est-ce qu'un message est spam ou non ?

Sigmoid Function : La fonction pour la régression Logistique

- La fonction hypothèse:

$$S(X^{(i)}) = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \theta_3 x_3 + \dots + \theta_n x_n$$

- $X^{(i)}$: une observation (du *Training Set* ou du *Test Set*), cette variable est un vecteur contenant:
 $[x_1, x_2, x_3, \dots, x_n]$
- x_i : est une variable prédictive (feature) qui servira dans le calcul du modèle prédictif
- θ_i : est un poids/paramètre de la fonction hypothèse. Ce sont ces θ_i qu'on cherche à calculer pour obtenir notre fonction de prédiction.
- θ_0 : est une constante nommée le **bias (biais)**

Sigmoid Function : La fonction pour la régression Logistique

- On peut réécrire $\theta_0 = \theta_0 * x_0$: avec $x_0=1$, Cela nous permet de réécrire notre fonction $S(X)$ de façon plus compacte comme suit :
$$S(X) = \sum_{i=0}^{n+1} \theta_i * x_i$$

On appelle cette fonction hypothèse : **la fonction score**. L'idée est de trouver des coefficients: $[\theta_0, \theta_1, \theta_2, \theta_3, \dots, \theta_n]$ de sorte que :

- $S(X^{(i)}) > 0$: quand la classe (étiquette) vaut 1
- $S(X^{(i)}) < 0$: quand la classe (étiquette) vaut 0

$$Pr obabilité(X \in Classe "1") = Sigmoide(\theta.X)$$

$$= \frac{1}{1 + \exp(-S(X))} = \frac{1}{1 + \exp(-\theta.X)}$$

Le résultat obtenu par la fonction **sigmoid** ($\theta.X$) est interprété comme la **probabilité que l'observation X soit d'un label (étiquette) 1**.

la régression Logistique implémenté dans sklearn

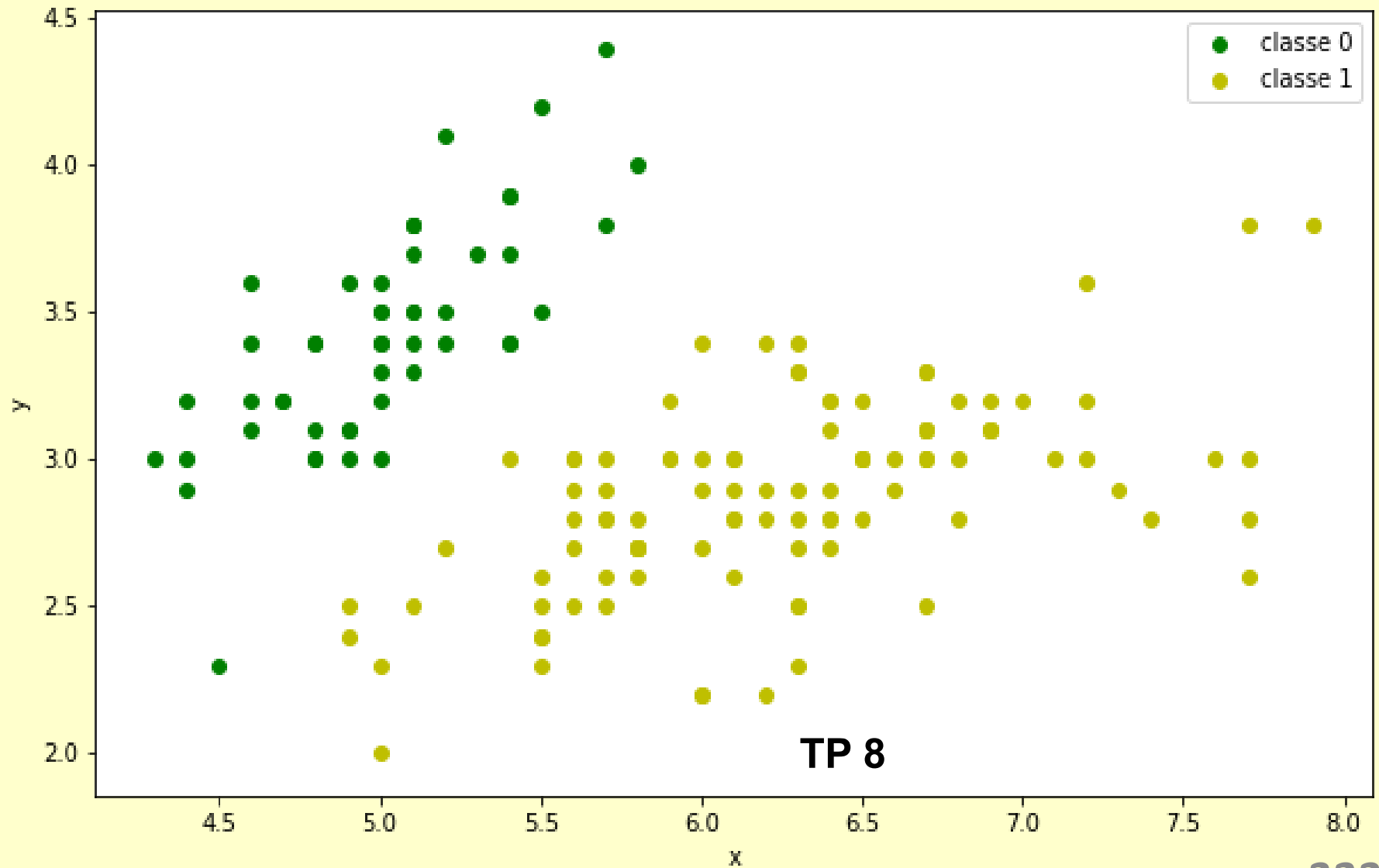
```
import numpy as np # Chargement de numpy
import matplotlib.pyplot as plt # import de Matplotlib
from sklearn import datasets
from sklearn.linear_model import LogisticRegression
iris = datasets.load_iris()
X = iris.data[:, :2] # avoir un problème de classification binaire.
# re-etiquetage des fleurs pour se retrouver avec deux classes au lieu de trois
y = []
for e in iris.target:
    if e==0:        y+= [0]
    else:           y+= [1]
y=np.array(y)
```

TP 8

la régression Logistique implémenté dans sklearn

```
plt.figure(figsize=(10,6)) # Taille de la figure
plt.scatter(X[y == 0][:, 0], X[y == 0][:, 1], color='g')
# En Vert Les fleurs ayant l'étiquette 0
plt.scatter(X[y == 1][:, 0], X[y == 1][:, 1], color='y')
# en Jaune Les fleurs ayant l'étiquette 1
plt.legend(labels = ("classe 0", "classe 1"));
plt.xlabel("x")
plt.ylabel("y")
```

la régression Logistique implémenté dans sklearn



la régression Logistique implémenté dans sklearn

```
model = LogisticRegression(C=1e20)#tester avec 0.01
x_train, x_test, y_train, y_test = train_test_split(X,y, test_size=0.2, random_state=0)
#la fonction train_test_split mélange les données avant de faire la répartition
#si on fixe random_state sur une valeur précise, on exécute sur deux machines différentes
#les Le mélange sera fait de la même façon sur les deux machines et on obtient les mêmes
model.fit(x_train, y_train)
score=cross_val_score(model,x_train, y_train, cv=5,scoring='accuracy')
score1=model.score(x_test, y_test)
score2=model.score(x_train, y_train)
print("score sur test-set =" +str(score1*100)+'%')
print("score sur train-set =" +str(score2*100)+'%')
theta0=model.intercept_; theta1=model.coef_[0][0]; theta2=model.coef_[0][1]
Iries_To_Predict = [ [5.5, 2.5], [7, 3], [3,2], [5,3]]
# demande de prédiction
print(model.predict(Iries_To_Predict))
```

```
score sur test-set =100.0%
score sur train-set =100.0%
[1 1 0 0]
```

TP 8

la régression Logistique implémenté dans sklearn

```
score=cross_val_score(model,x_train, y_train, cv=5,scoring='accuracy')
print(score)
print(score.mean())
def fct_reg_logistic(x1,x2):
    z=(theta0)+theta1*x1+(theta2*x2)
    return(z)
print([fct_reg_logistic(e[0],e[1]) for e in Iries_To_Predict])
```

```
[1.          0.95833333 1.          1.          1.          ]
```

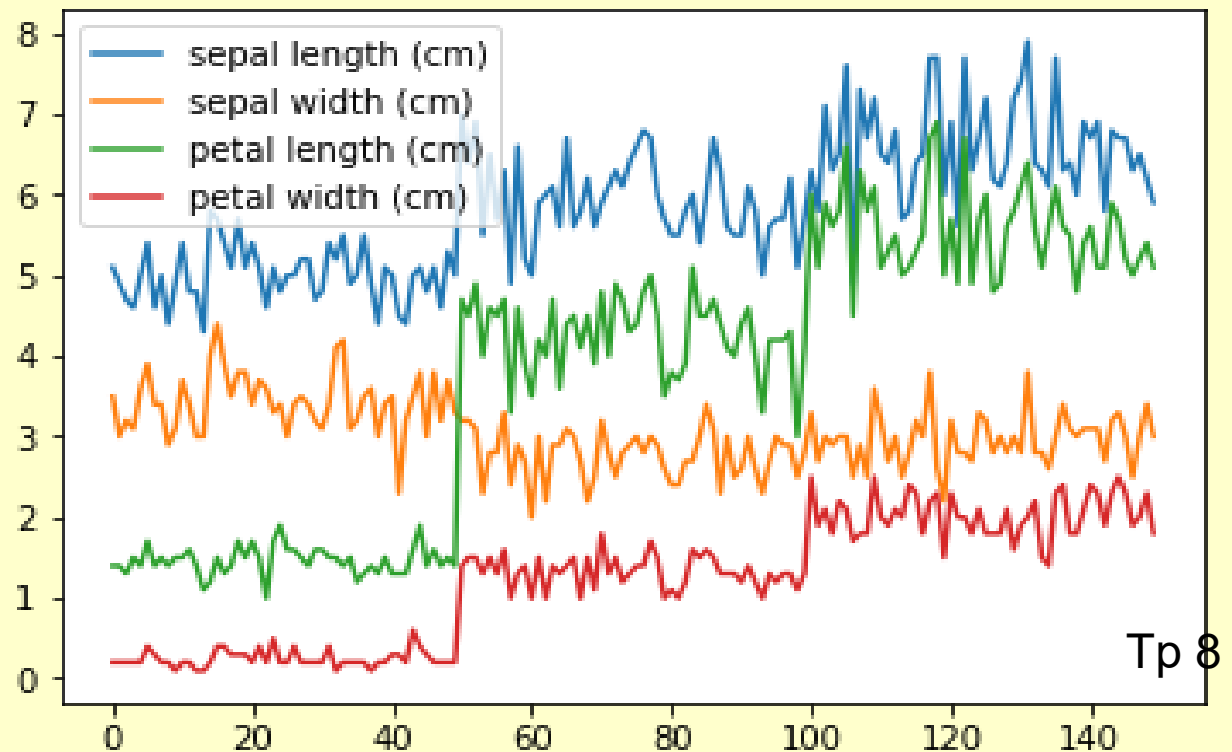
```
0.9916666666666668
```

```
[array([24.57979467]), array([60.629162]), array([-45.97305733]), array([-8.37780735])]
```

TP 8

la régression Logistique implémenté dans sklearn

```
from sklearn.datasets import load_iris
iris = load_iris (); X=iris.data ; Y=iris.target
import matplotlib.pyplot as plt
plt.plot(X); plt.legend(iris.feature_names); plt.show()
```



Tp 8

Représentation des données textuelles

Matrice sparse binaire: `CountVectorizer(binary=True)`

```
1 from sklearn.feature_extraction.text import CountVectorizer
2 texte = ["La vie est douce","La vie est tranquille, est belle, est douce"]
3 vect = CountVectorizer(binary=True)
4 T= vect.fit_transform(texte)
5 dictionnaire_des_mots=vect.vocabulary_
6 print("dictionnaire_des_mots :", dictionnaire_des_mots)
7 liste_des_mots=list(dictionnaire_des_mots.keys())
8 print("liste_des_mots :", liste_des_mots)
9 Matrice_sparse_correspondante=T.toarray()
10 print("Matrice_sparse_correspondante:\n",Matrice_sparse_correspondante)
```

```
dictionnaire_des_mots : {'la': 3, 'vie': 5, 'est': 2, 'douce': 1, 'tranquille': 4, 'belle': 0}
```

```
liste_des_mots : ['la', 'vie', 'est', 'douce', 'tranquille', 'belle']
```

```
Matrice_sparse_correspondante:
```

```
[[0 1 1 1 0 1]
```

```
[1 1 1 1 1 1]]
```

Application de la régression Logistique sur les Spams

```
import numpy as np
import pandas
spams = pandas.read_table("D:\\SMSSpamCollection.txt", sep="\t", header=0)
spamsTrain, spamsTest = train_test_split(spams, train_size=0.7, random_state=1)
from sklearn.feature_extraction.text import CountVectorizer
parseur = CountVectorizer()
XTrain = parseur.fit_transform(spamsTrain['message'])
mdtTrain = XTrain.toarray()
from sklearn.linear_model import LogisticRegression
modelFirst = LogisticRegression()
#essayer cette version, pourquoi ça ne marche pas ?
#modelFirst.fit(spamsTrain['message'], spamsTrain['classe'])
modelFirst.fit(mdtTrain, spamsTrain['classe'])
score1 = modelFirst.score(mdtTrain, spamsTrain['classe'])
print("score sur data train:" + str(score1))
mdtTest = parseur.transform(spamsTest['message'])
#predTest = modelFirst.predict(mdtTest)
score2 = modelFirst.score(mdtTest, spamsTest['classe'])
print("score sur data test:" + str(score2))
```

score sur data train:0.9976923076923077

score sur data test:0.9838516746411483

Sans binary=True

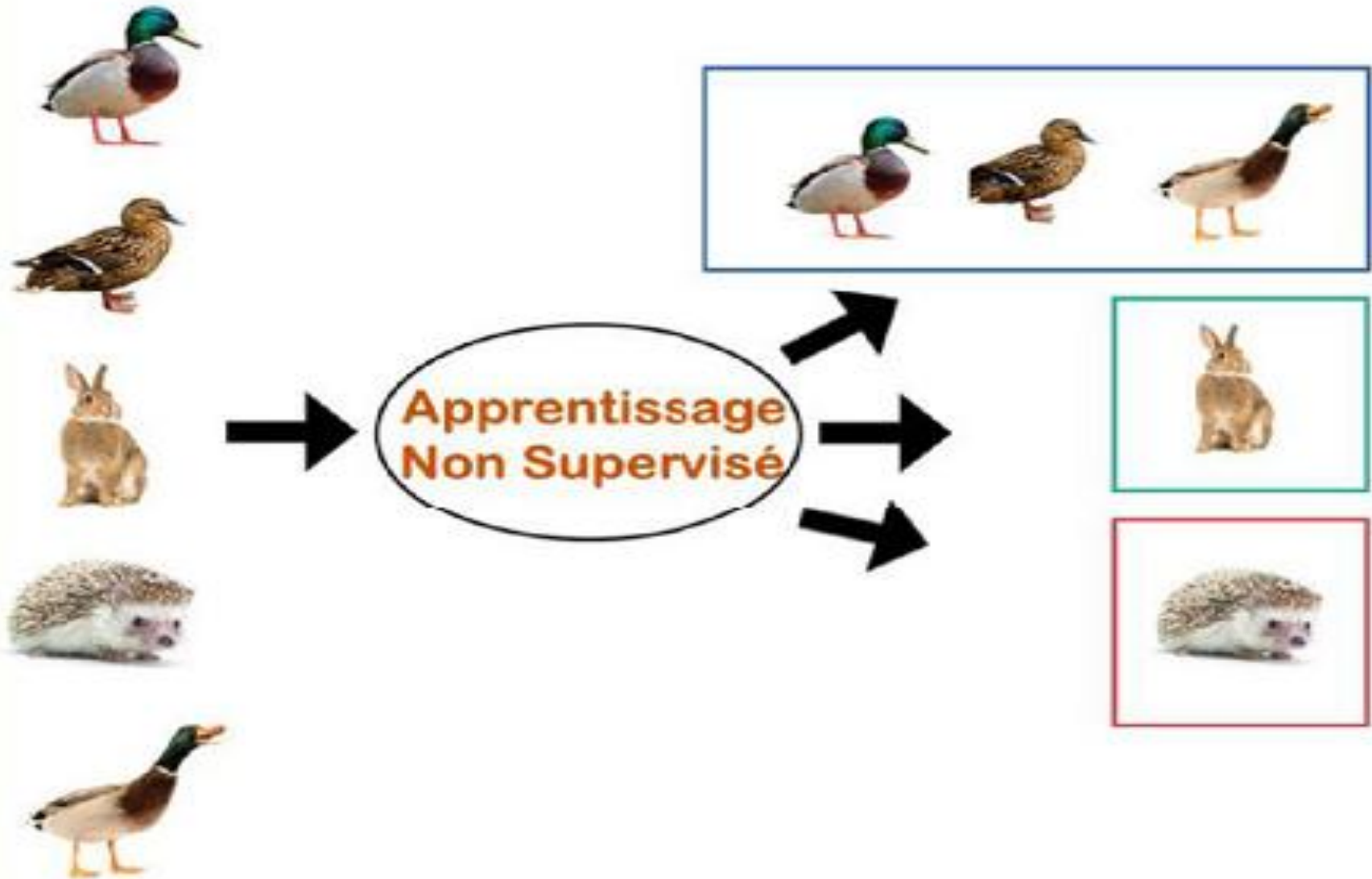
```
1 import string
2 import nltk
3 nltk.download('stopwords')
4 from nltk.corpus import stopwords
5 print(stopwords.words('english'))
6 print(stopwords.words('french'))
7 print(stopwords.words('arabic'))
```

```
['i', 'me', 'my', 'myself', 'we', 'our', 'ours', 'ourselves', 'you', "you're", "you've", "you'll", "you'd", 'your', 'yours', 'yourself', 'yourselves', 'he', 'him', 'his', 'himself', 'she', "she's", 'he'r', 'hers', 'herself', 'it', "it's", 'its', 'itself', 'they', 'them', 'their', 'theirs', 'themselves', 'what', 'which', 'who', 'whom', 'this', 'that', "that'll", 'these', 'those', 'am', 'is', 'are', 'was', 'were', 'be', 'been', 'being', 'have', 'has', 'had', 'having', 'do', 'does', 'did', 'doing', 'a', 'an', 'the', 'and', 'but', 'if', 'or', 'because', 'as', 'until', 'while', 'of', 'at', 'by', 'for', 'with', 'about', 'against', 'between', 'into', 'through', 'during', 'before', 'after', 'above', 'below', 'to', 'from', 'up', 'down', 'in', 'out', 'on', 'off', 'over', 'under', 'again', 'further', 'then', 'once', 'here', 'there', 'when', 'where', 'why', 'how', 'all', 'any', 'both', 'each', 'few', 'more', 'most', 'other', 'some', 'such', 'no', 'nor', 'not', 'only', 'own', 'same', 'so', 'than', 'too', 'very', 's', 't', 'can', 'will', 'just', 'don', "don't", 'should', "should've", 'now', 'd', 'll', 'm', 'o', 're', 've', 'y', 'ain', 'aren', "aren't", 'couldn', "couldn't", 'didn', "didn't", 'doesn', "doesn't", 'hadn', "hadn't", 'hasn', "hasn't", 'haven', "haven't", 'isn', "isn't", 'ma', 'mightn', "mightn't", 'mustn', "mustn't", 'needn', "needn't", 'shan', "shan't", 'shouldn', "shouldn't", 'wasn',
```

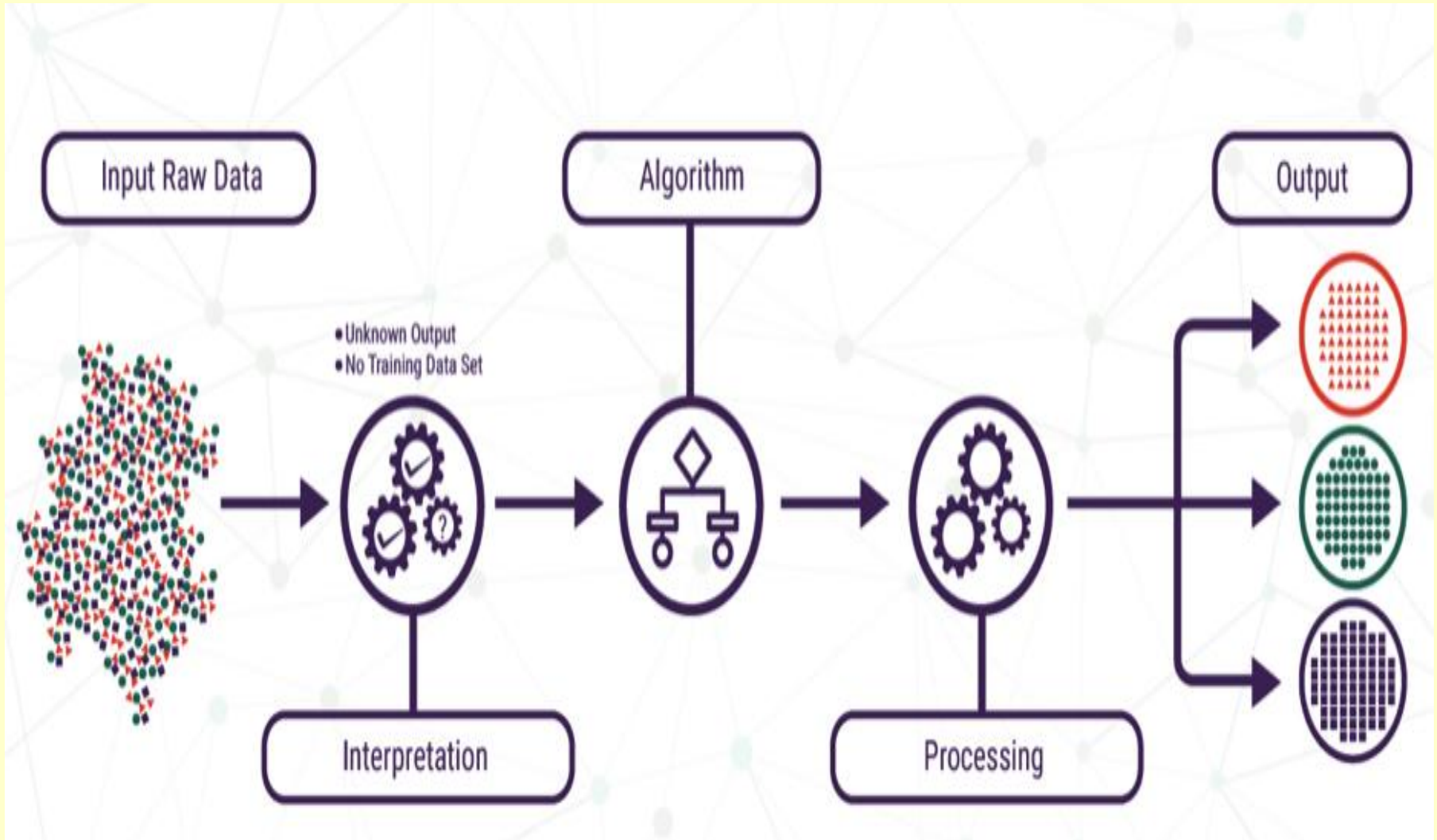
Apprentissage non supervisé (unsupervised learning)

En apprentissage non supervisé, les données d'entrées ne sont pas annotées. L'algorithme d'entraînement s'applique dans ce cas à trouver les similarités et distinctions au sein de ces données, et à regrouper ensemble celles qui partagent des caractéristiques communes. Dans notre exemple, les photos similaires seraient ainsi regroupées automatiquement au sein d'une même catégorie.

Apprentissage non supervisé (unsupervised learning)



Apprentissage non supervisé



Classification supervisée et non supervisée

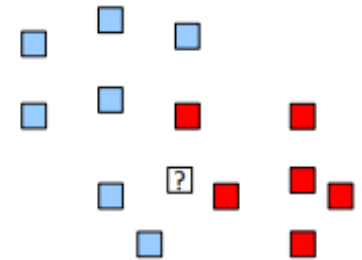
Classification supervisée :

- On dispose d'**éléments déjà classés**

Exemple : articles en rubrique économie, politique, sport, culture...

- On veut **classer un nouvel élément**

Exemple : lui attribuer une étiquette parmi économie, politique, sport, culture...



Classification non supervisée

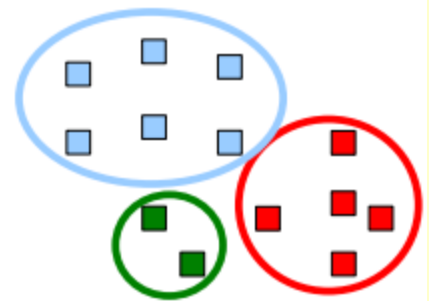
- On dispose d'**éléments non classés**

Exemple : mots d'un texte

- On veut les **regrouper en classes**

Exemple : si deux mots ont la même étiquette, ils sont en rapport avec une même thématique...

Ce qui ressemble
s'assemble !



Classification non supervisée

Contexte : Pour n individus d'une population, on dispose des valeurs de p caractères $X_1, \dots, X_p$. Ces valeurs constituent les données.

Objectif : Partant des données, l'objectif est de regrouper/classer les individus qui se ressemblent le plus/qui ont des caractéristiques semblables.

Ce regroupement peut avoir des buts divers : tenter de séparer des individus appartenant à des sous-populations distinctes, décrire les données en procédant à une réduction du nombre d'individus pour communiquer, simplifier, exposer les résultats. . .

Classification non supervisée

Exemple: Dans une classe, un professeur souhaite faire des binômes constitués d'élèves ayant des compétences semblables. Parmi ceux-ci, 6 élèves ont obtenu les notes suivantes :

	Math	P.C	Fr	Ar
B	20	20	0	0
M	8	8	12	12
S	20	20	0	0
J	0	0	20	20
L	10	10	10	10
A	2	2	18	18

Tous les élèves ont une moyenne de 10/20 mais, vu les notes:

- B et S ont un profil similaire,
- M et L ont un profil similaire,
- J et A ont un profil similaire.

Finalement, le professeur décide de faire 2 groupes cohérents de 3 élèves avec ces 6 élèves. Lesquels proposez-vous ?

Classification non supervisée

	Math	P.C	Fr	Ar
B	20	20	0	0
M	8	8	12	12
S	20	20	0	0
J	0	0	20	20
L	10	10	10	10
A	2	2	18	18

En comparant les notes par matière, on propose :

- Groupe 1 : B,S,L
- Groupe 2 : M,J,A.

De plus, par exemple, le profil de L est plus proche de celui de B, que celui de A.

Bien entendu, cette analyse intuitive n'est pas facile si, par exemple, on a 30 élèves à classer par groupes de 3 et on considère 12 matières. C'est pourquoi des méthodes de clustering (regroupement) ont été mises en place.

Apprentissage non supervisé

- Matrice de données
$$\begin{bmatrix} x_{11} & \dots & x_{1f} & \dots & x_{1p} \\ \dots & \dots & \dots & \dots & \dots \\ x_{i1} & \dots & x_{if} & \dots & x_{ip} \\ \dots & \dots & \dots & \dots & \dots \\ x_{n1} & \dots & x_{nf} & \dots & x_{np} \end{bmatrix}$$
- Matrice de similarité
$$\begin{bmatrix} 0 & & & & \\ d(2,1) & 0 & & & \\ d(3,1) & d(3,2) & 0 & & \\ \vdots & \vdots & \vdots & & \\ d(n,1) & d(n,2) & \dots & \dots & 0 \end{bmatrix}$$

Apprentissage non supervisé

Similarité entre objets

- Les distances expriment une similarité
- Ex: la *distance de Minkowski* :

$$d(i, j) = \sqrt[q]{(|x_{i_1} - x_{j_1}|^q + |x_{i_2} - x_{j_2}|^q + \dots + |x_{i_p} - x_{j_p}|^q)}$$

où $i = (x_{i_1}, x_{i_2}, \dots, x_{i_p})$ et $j = (x_{j_1}, x_{j_2}, \dots, x_{j_p})$ sont deux objets p -dimensionnels et q un entier positif

- Si $q = 1$, d est la distance de Manhattan

$$d(i, j) = |x_{i_1} - x_{j_1}| + |x_{i_2} - x_{j_2}| + \dots + |x_{i_p} - x_{j_p}|$$

Similarité entre objets

- *Si $q = 2$, d est la distance Euclidienne :*

$$d(i, j) = \sqrt{(|x_{i_1} - x_{j_1}|^2 + |x_{i_2} - x_{j_2}|^2 + \dots + |x_{i_p} - x_{j_p}|^2)}$$

– Propriétés

- $d(i, j) \geq 0$
- $d(i, i) = 0$
- $d(i, j) = d(j, i)$
- $d(i, j) \leq d(i, k) + d(k, j)$

Apprentissage non supervisé

Exemple: distance de Manhattan

	Age	Salaire
Personne1	50	11000
Personne2	70	11100
Personne3	60	11122
Personne4	60	11074

$$d(p1,p2)=120$$

$$d(p1,p3)=132$$

Conclusion: p1 ressemble plus à p2 qu'à p3

Classification non supervisée

Soit Γ un ensemble d'individus (population en statistiques)

Écarts: Soit $P(\Gamma)$ l'ensemble des parties de Γ . On appelle écart toute application $e : P(\Gamma) \times P(\Gamma) \rightarrow [0, \infty[$ définie à partir d'une distance et évaluant la ressemblance entre deux groupes d'individus.

Règle centrale: Plus l'écart entre deux éléments est petit, plus ils se ressemblent.

Écarts usuels: Parmi les écarts usuels entre deux groupes A et B (méthodes usuelles mesurant la ressemblance entre deux groupes A et B), il y a :

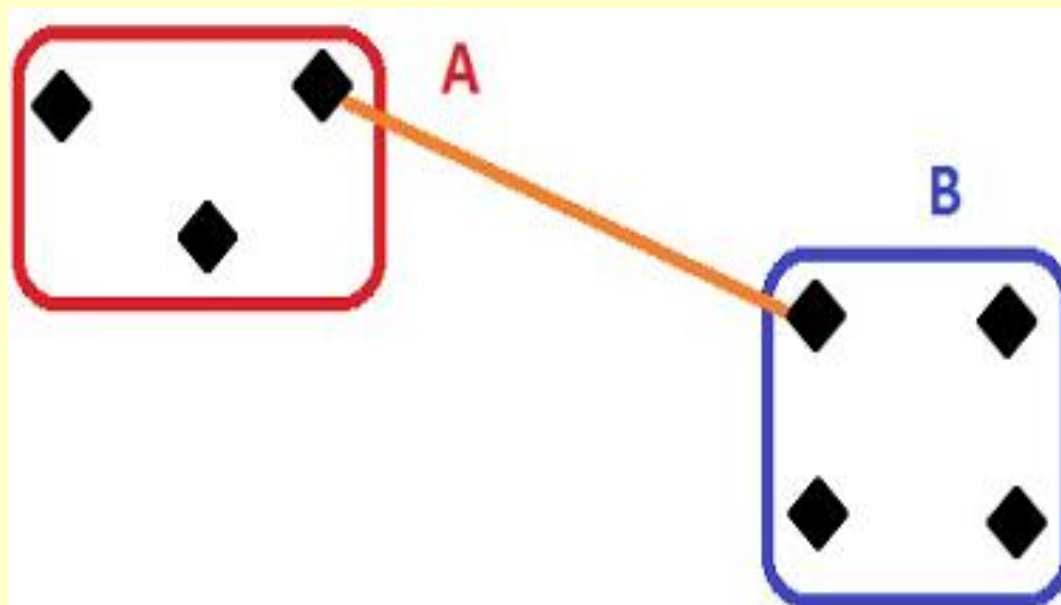
- Écart simple (single linkage)/Méthode du plus proche voisin
- Écart complet (complete linkage)/Méthode du voisin le plus éloigné
- Écart moyen (average linkage)/Méthode de la distance moyenne
- Écart de Ward

Classification non supervisée

Écart simple (single linkage)/Méthode du plus proche voisin :

L'écart entre deux groupes A et B est caractérisé par la distance la plus faible entre un point de A et un point de B :

$$e(A, B) = \min_{(\omega, \omega_*) \in A \times B} d(\omega, \omega_*)$$

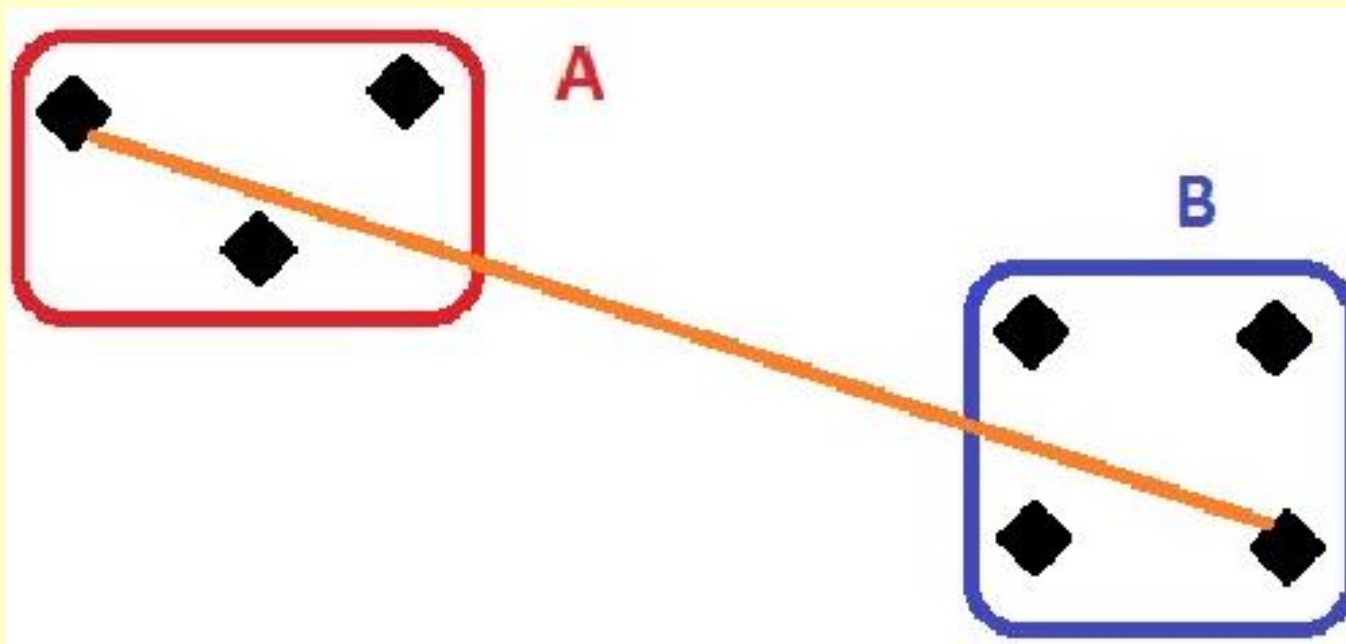


Classification non supervisée

Écart complet (complete linkage)/Méthode du voisin le plus éloigné

L'écart entre deux groupes A et B est caractérisé par la distance la plus forte entre un point de A et un point de B :

$$e(A, B) = \max_{(\omega, \omega_*) \in A \times B} d(\omega, \omega_*)$$

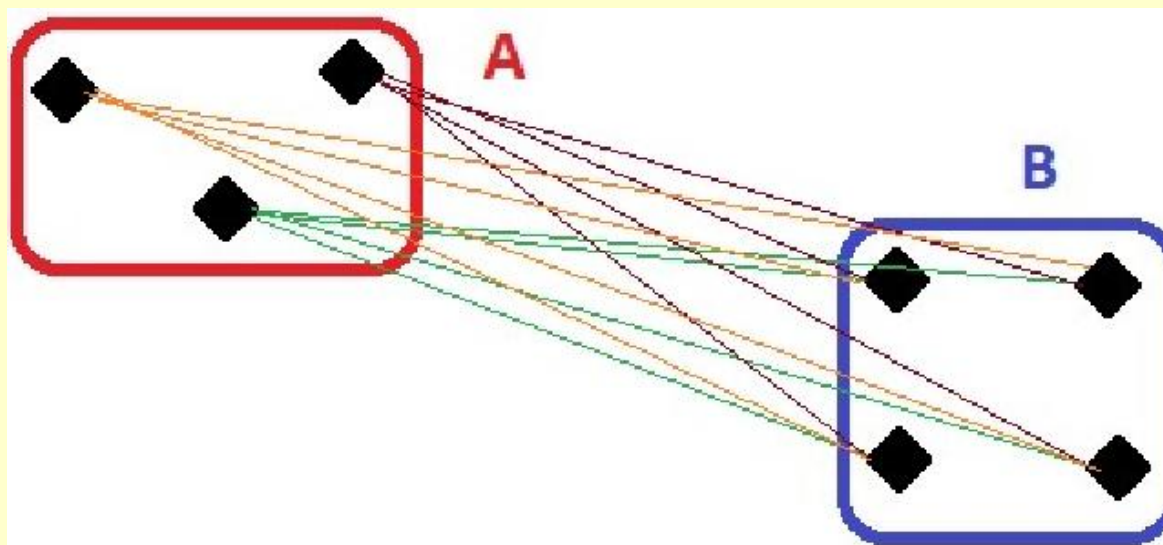


Classification non supervisée

Écart moyen (average linkage)/Méthode de la distance moyenne

$$e(A, B) = \frac{1}{n_A n_B} \sum_{\omega \in A} \sum_{\omega_* \in B} d(\omega, \omega_*)$$

où n_A est le nombre d'individus dans A, et n_B le nombre d'individus dans B. L'écart entre deux groupes A et B est caractérisé par la distance moyenne entre les points de A et B :



Classification non supervisée

Écart de Ward : Soit d la distance euclidienne.

La méthode de Ward considère l'écart :

$$e(A, B) = \frac{n_A n_B}{n_A + n_B} d^2(g_A, g_B)$$

où g_A est le centre de gravité de A , et g_B celui de B . On rappelle que g_A est le point de coordonnées $(m_1, \dots, m_p)$, où, pour tout $j \in \{1, \dots, p\}$, m_j désigne la moyenne des valeurs observées du caractère X_j sur les n_A individus du groupe A . De même pour g_B et n_B .

Dans l'exemple de A ici on a: $n_A=5$ et $p=2$

La première coordonné est: $3.8=(2+7.5+3+0.5+6)/5$

La deuxième coordonné est: $3.6=(2+4+3+5+4)/5$

Donc g_A est de coordonnées $g_A(3.8,3.6)$

$$\mathbf{A} = \begin{pmatrix} 2 & 2 \\ 7.5 & 4 \\ 3 & 3 \\ 0.5 & 5 \\ 6 & 4 \end{pmatrix}$$

Apprentissage non supervisé

Tableau des écarts :

Soit e un écart défini par une des méthodes précédentes. On appelle tableau des écarts associé aux groupes d'individus $(A_1, \dots, A_n)$ le tableau ci-dessous $\forall (u, v) \in \{1, \dots, n\}^2$ avec $u \neq v$, $e_{u,v} = e(A_u, A_v)$.

$\mathbf{E} =$

	A_1	A_2	$\dots$	A_{n-1}	A_n
A_1	0	$e_{1,2}$	$\dots$	$e_{1,n-1}$	$e_{1,n}$
A_2	$e_{2,1}$	0	$\dots$	$\dots$	$\dots$
$\dots$	$\dots$	$\dots$	$\dots$	$\dots$	$\dots$
A_{n-1}	$e_{n-1,1}$	$\dots$	$\dots$	0	$e_{n-1,n}$
A_n	$e_{n,1}$	$\dots$	$\dots$	$e_{n,n-1}$	0

Apprentissage non supervisé

Exemple : On considère la matrice de données X dans $\mathbb{R}^2$ définie par:

On considère la méthode du voisin le plus éloigné munie de la distance euclidienne. Le tableau des écarts associé à $(\{\omega_1\}, \dots, \{\omega_5\})$ est en fait le tableau des distances :

	ω_1	ω_2	ω_3	ω_4	ω_5
ω_1	0	5.85	1.41	3.35	4.47
ω_2	5.85	0	4.60	7.07	1.50
ω_3	1.41	4.60	0	3.20	3.16
ω_4	3.35	7.07	3.20	0	5.59
ω_5	4.47	1.50	3.16	5.59	0

$$X = \begin{pmatrix} 2 & 2 \\ 7.5 & 4 \\ 3 & 3 \\ 0.5 & 5 \\ 6 & 4 \end{pmatrix} \begin{matrix} W1 \\ W2 \\ W3 \\ W4 \\ W5 \end{matrix}$$

Apprentissage non supervisé

E =

	ω_1	ω_2	ω_3	ω_4	ω_5
ω_1	0	5.85	1.41	3.35	4.47
ω_2	5.85	0	4.60	7.07	1.50
ω_3	1.41	4.60	0	3.20	3.16
ω_4	3.35	7.07	3.20	0	5.59
ω_5	4.47	1.50	3.16	5.59	0

Soit A le couple d'individus : $A = \{\omega_1, \omega_3\}$.

Par la même méthode, on obtient:

$$e(\omega_2, A) = \max(e(\omega_2, \omega_1), e(\omega_2, \omega_3)) = \max(5.85, 4.60) = 5.85,$$

$$e(\omega_4, A) = \max(e(\omega_4, \omega_1), e(\omega_4, \omega_3)) = \max(3.35, 3.20) = 3.35$$

$$e(\omega_5, A) = \max(e(\omega_5, \omega_1), e(\omega_5, \omega_3)) = \max(4.47, 3.16) = 4.47.$$

Apprentissage non supervisé

Le tableau des écarts associé à $(\{\omega_2\}, \{\omega_4\}, \{\omega_5\}, A)$ est:

E =

	ω_2	ω_4	ω_5	A
ω_2	0	7.07	1.50	5.85
ω_4	7.07	0	5.59	3.35
ω_5	1.50	5.59	0	4.47
A	5.85	3.35	4.47	0

Clustering

le processus de regroupement d'entités similaires. Les données groupées sont utilisées pour créer des clusters. Le but de cette technique est de rechercher des similitudes dans les données ainsi que de déterminer à quel groupe de nouvelles données doivent appartenir. Ex : k-means, hierarchical clustering, db-scan..

Qu'est ce qu'un bon regroupement ?

- Une bonne méthode de regroupement permet de garantir
 - Une grande similarité intra-groupe
 - Une faible similarité inter-groupe
- La qualité d'un regroupement dépend donc de la mesure de similarité utilisée par la méthode et de son implémentation

La méthode des k-moyennes (K-Means)

Méthode des centres mobiles

L'algorithme *k-means* se déroule en 4 étapes :

1. Choisir k objets formant ainsi k clusters
2. (Ré)affecter chaque objet O au cluster C_i de centre M_i tel que $\text{dist}(O, M_i)$ est minimal
3. Recalculer M_i de chaque cluster (le barycentre)
4. Aller à l'étape 2 si on vient de faire une affectation

Apprentissage non supervisé - Clustering

Méthode des centres mobiles: K-means

K-Means :Exemple

- $A=\{1,2,3,6,7,8,13,15,17\}$. Créer 3 clusters à partir de A
- On prend 3 objets au hasard. Supposons que c'est 1, 2 et 3. Ça donne $C_1=\{1\}$, $M_1=1$, $C_2=\{2\}$, $M_2=2$, $C_3=\{3\}$ et $M_3=3$
- Chaque objet O est affecté au cluster au milieu duquel, O est le plus proche. 6 est affecté à C_3 car $\text{dist}(M_3,6) < \text{dist}(M_2,6)$ et $\text{dist}(M_3,6) < \text{dist}(M_1,6)$

On a $C_1=\{1\}$, $M_1=1$, $C_2=\{2\}$, $M_2=2$

$C_3=\{3, 6,7,8,13,15,17\}$, $M_3=69/7=9.86$



Apprentissage non supervisé - Clustering

Méthode des centres mobiles: K-means

- $\text{dist}(3, M_2) < \text{dist}(3, M_3) \rightarrow 3$ passe dans C_2 . Tous les autres objets ne bougent pas.
 $C_1 = \{1\}$, $M_1 = 1$, $C_2 = \{2, 3\}$, $M_2 = 2.5$, $C_3 = \{6, 7, 8, 13, 15, 17\}$ et $M_3 = 66/6 = 11$



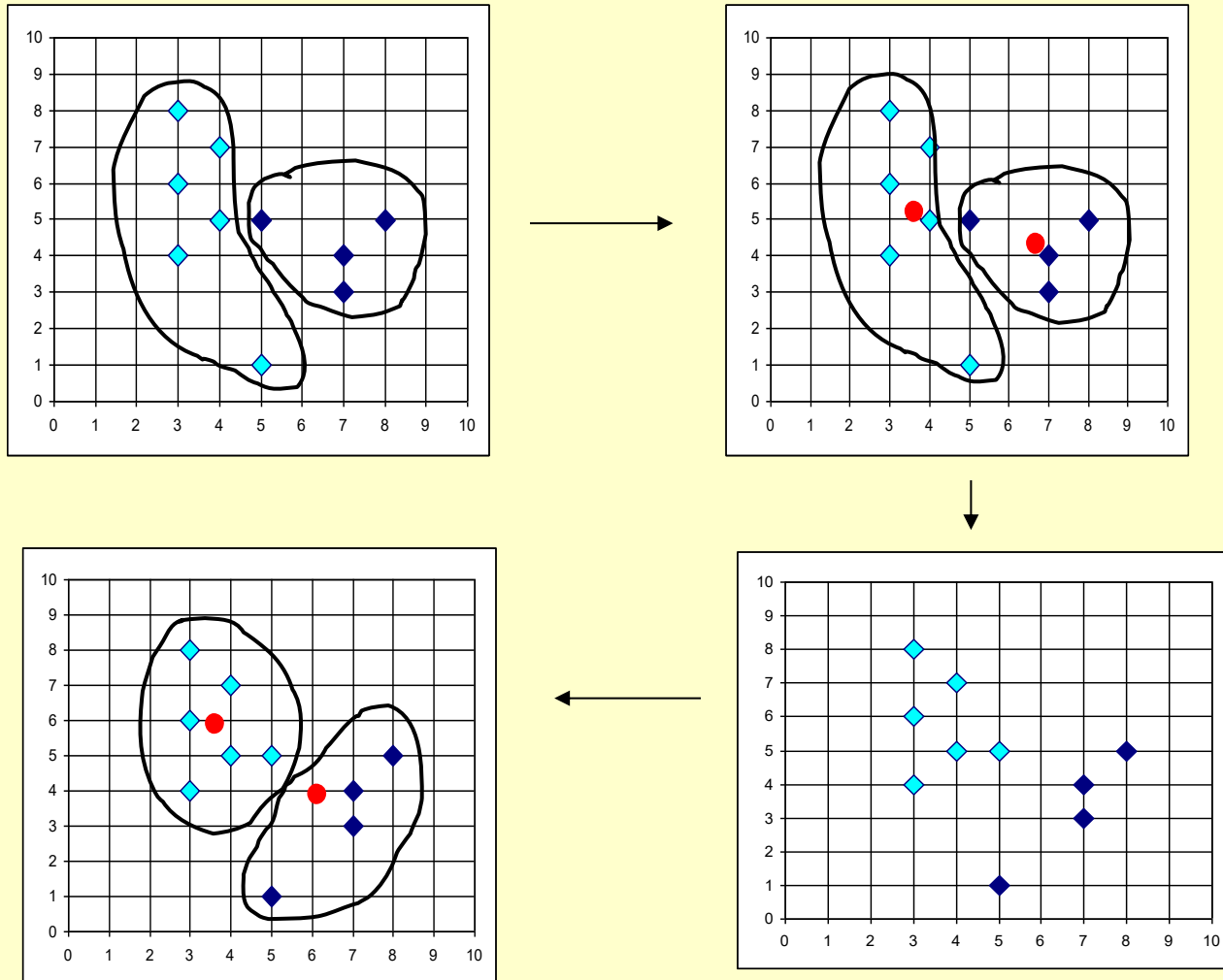
- $\text{dist}(6, M_2) < \text{dist}(6, M_3) \rightarrow 6$ passe dans C_2 . Tous les autres objets ne bougent pas.
 $C_1 = \{1\}$, $M_1 = 1$, $C_2 = \{2, 3, 6\}$, $M_2 = 11/3 = 3.67$, $C_3 = \{7, 8, 13, 15, 17\}$, $M_3 = 12$
- $\text{dist}(2, M_1) < \text{dist}(2, M_2) \rightarrow 2$ passe en C_1 . $\text{dist}(7, M_2) < \text{dist}(7, M_3) \rightarrow 7$ passe en C_2 . Les autres ne bougent pas. $C_1 = \{1, 2\}$, $M_1 = 1.5$, $C_2 = \{3, 6, 7\}$, $M_2 = 5.34$, $C_3 = \{8, 13, 15, 17\}$, $M_3 = 13.25$
- $\text{dist}(3, M_1) < \text{dist}(3, M_2) \rightarrow 3$ passe en 1. $\text{dist}(8, M_2) < \text{dist}(8, M_3) \rightarrow 8$ passe en 2
 $C_1 = \{1, 2, 3\}$, $M_1 = 2$, $C_2 = \{6, 7, 8\}$, $M_2 = 7$, $C_3 = \{13, 15, 17\}$, $M_3 = 15$

Plus rien ne bouge

Apprentissage non supervisé - Clustering

Méthode des centres mobiles: K-means

- Exemple



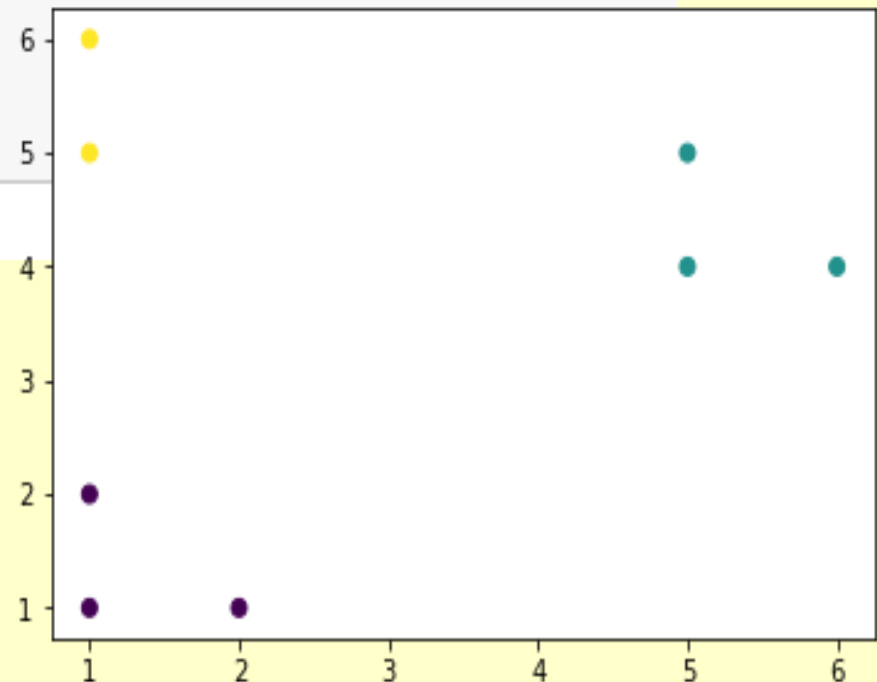
Apprentissage non supervisé - Clustering

Méthode des centres mobiles: K-means

```
1 import matplotlib.pyplot as plt
2 from sklearn.cluster import KMeans
3 import numpy as np
4 X = np.array([[1,1],[1,6],[6,4],[1,5],[1,2],[2,1],[5,4],[5,5]])
5 kmeans = KMeans(n_clusters=3, random_state=0).fit(X)
6 #c'est le travail de kmeans pour classer de façon non supervisée
7 print(kmeans.labels_)
8 T=kmeans.predict(X)
9 plt.scatter(X[:, 0],X[:, 1],c=T)
10 plt.show()
```

[0 2 1 2 0 0 1 1]

Tp 51



Apprentissage non supervisé - Clustering

Méthode des centres mobiles: K-means

```
1 import pandas as pd; import numpy as np
2 import sklearn.metrics as sm
3 import matplotlib.pyplot as plt
4 from sklearn.cluster import KMeans
5 from sklearn import datasets
6 iris = datasets.load_iris()
7 x=pd.DataFrame(iris.data); y=pd.DataFrame(iris.target)
8 x.columns=['Sepal_Length','Sepal_width','Petal_Length','Petal_width']
9 y.columns=['Targets']
10 model=KMeans(n_clusters=3); model.fit(x); print(model.labels_)
```

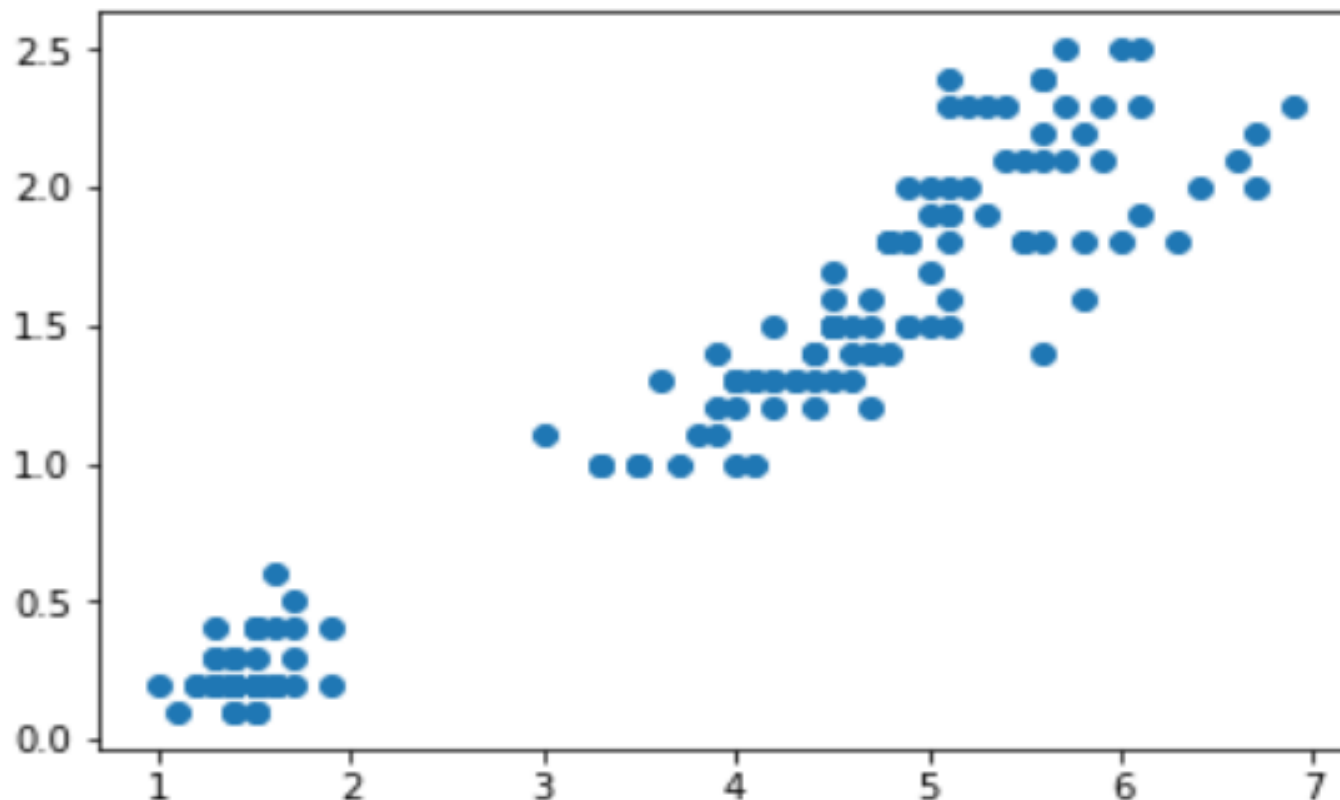
```
[0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0  
0 0 0 0 0 0 0 0 0 0 0 0 0 1 1 2 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1  
1 1 1 2 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 2 1 2 2 2 2 1 2 2 2 2  
2 2 1 1 2 2 2 2 1 2 1 2 1 2 2 1 1 2 2 2 2 2 1 2 2 2 2 1 2 2 2 1 2  
2 1]
```

Apprentissage non supervisé - Clustering

Méthode des centres mobiles: K-means

```
1 #Visualisation des clusters  
2 plt.scatter(x.Petal_Length, x.Petal_width)
```

<matplotlib.collections.PathCollection at 0x29bd1002470>



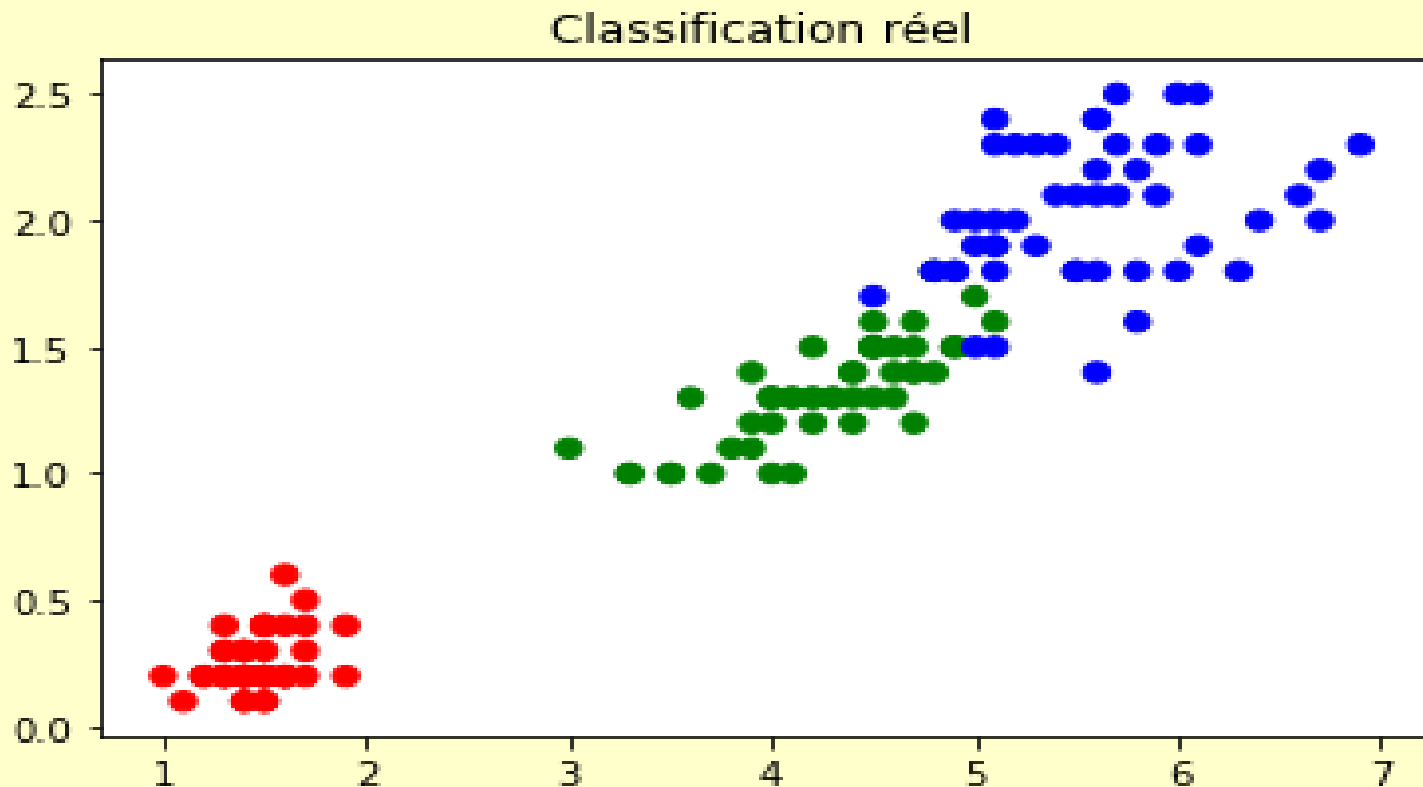
Apprentissage non supervisé - Clustering

Méthode des centres mobiles: K-means

```
colormap=np.array(['Red','green','blue'])
```

```
#Visualisation des clusters
```

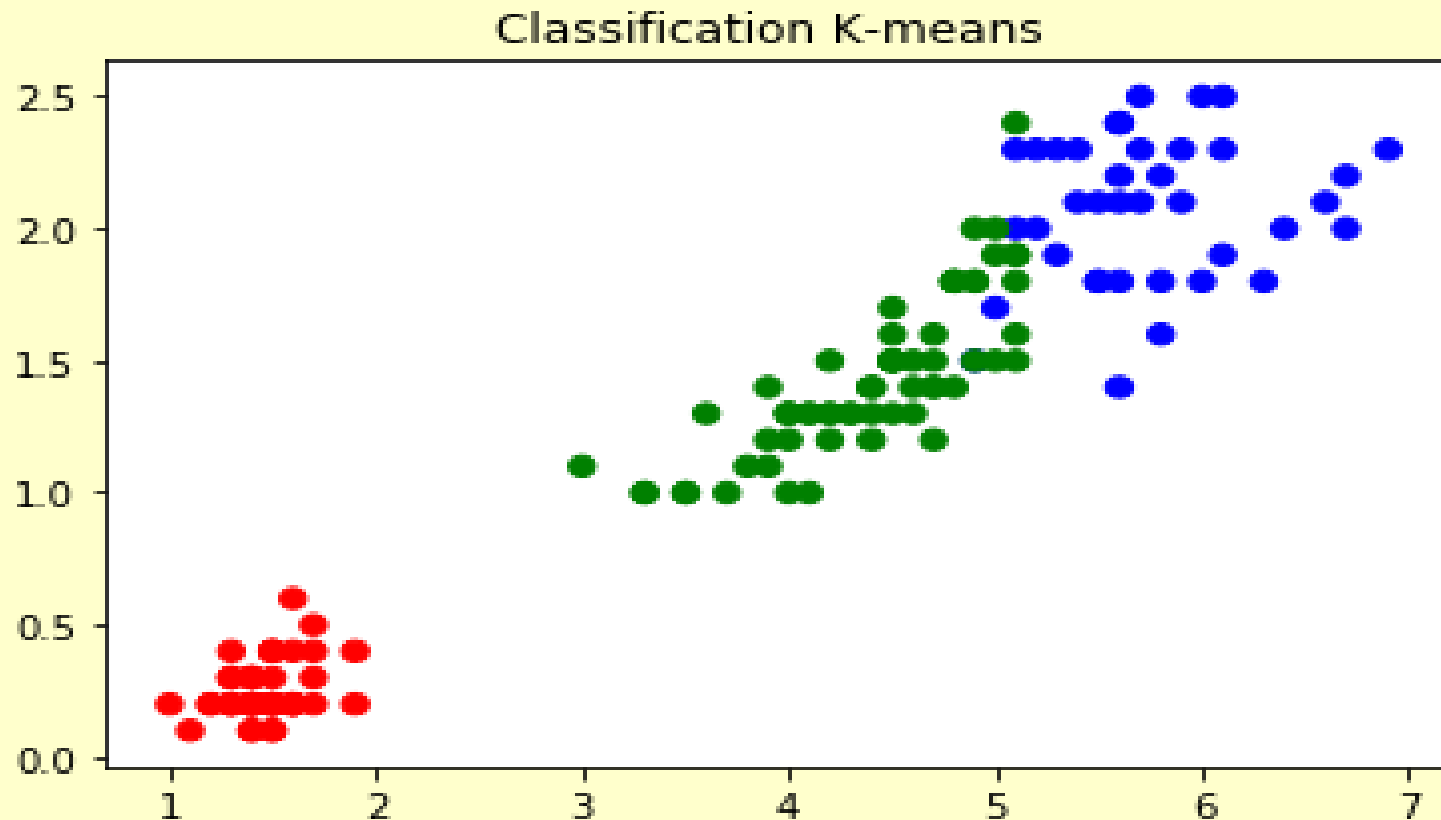
```
plt.scatter(x.Petal_Length, x.Petal_width,c=colormap[y.Targets],s=40)  
plt.title('Classification réel')
```



Apprentissage non supervisé - Clustering

Méthode des centres mobiles: K-means

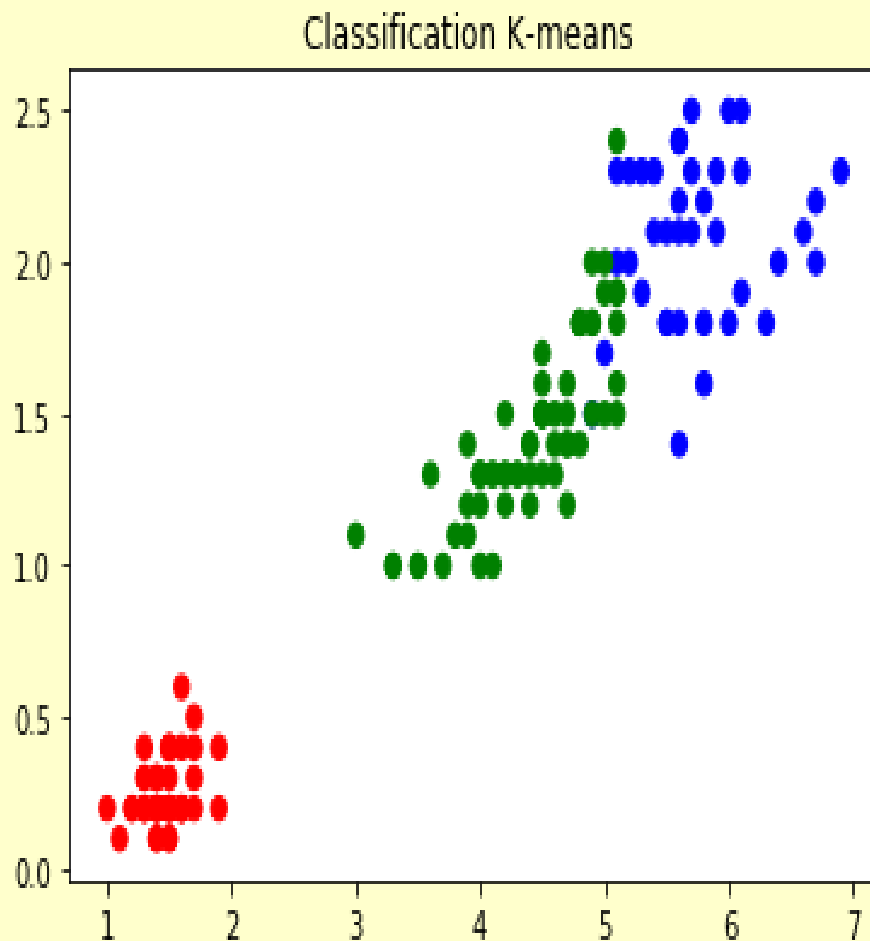
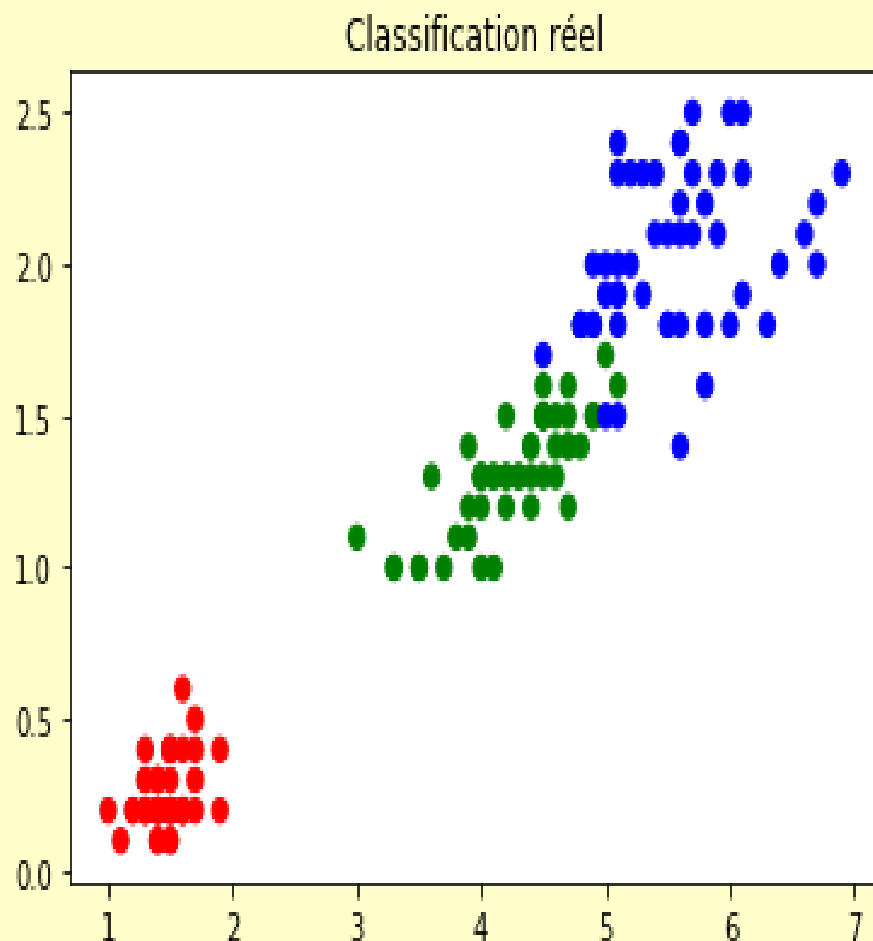
```
plt.scatter(x.Petal_Length, x.Petal_width, c=colormap[model.labels_], s=40)  
plt.title('Classification K-means ')
```



Apprentissage non supervisé - Clustering

Méthode des centres mobiles: K-means

Comparaison du regroupement réel (selon la colonne target) du jeux de données IRIS avec un regroupement obtenu par K-means (3-Means).



Apprentissage non supervisé - Clustering

Méthode des centres mobiles: K-means

```
1 A=list(model.labels_); print (A)
2 B=list(y['Targets']); print (B); C=[A[i]!=B[i] for i in range(len(A))]
3 Nbr_fleurs_mal_regroupées=C.count(True); print(Nbr_fleurs_mal_regroupées)
```

```
[0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 1,
2, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 2,
1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 2, 1, 2, 2,
2, 2, 1, 2, 2, 2, 2, 2, 2, 1, 1, 2, 2, 2, 2, 1, 2, 1, 2, 1, 2, 2, 1, 1, 2, 2,
2, 2, 2, 1, 2, 2, 2, 2, 1, 2, 2, 2, 1, 2, 2, 2, 1, 2, 2, 1]
[0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 1,
1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 2, 2, 2, 2,
2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2,
2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2]
```

Apprentissage non supervisé – Clustering

Classification ascendante hiérarchique (CAH)

CAH: L'idée de l'algorithme de classification ascendante hiérarchique (CAH) est de créer, à chaque étape, une partition de $\Gamma = \{\omega_1, \dots, \omega_n\}$ en regroupant les deux éléments les plus proches. Le terme "élément" désigne aussi bien un individu qu'un groupe d'individus.

Objectif : On veut mettre en relief les liens hiérarchiques entre les individus ou groupe d'individus ou on veut détecter les groupes d'individus qui se démarquent le plus.

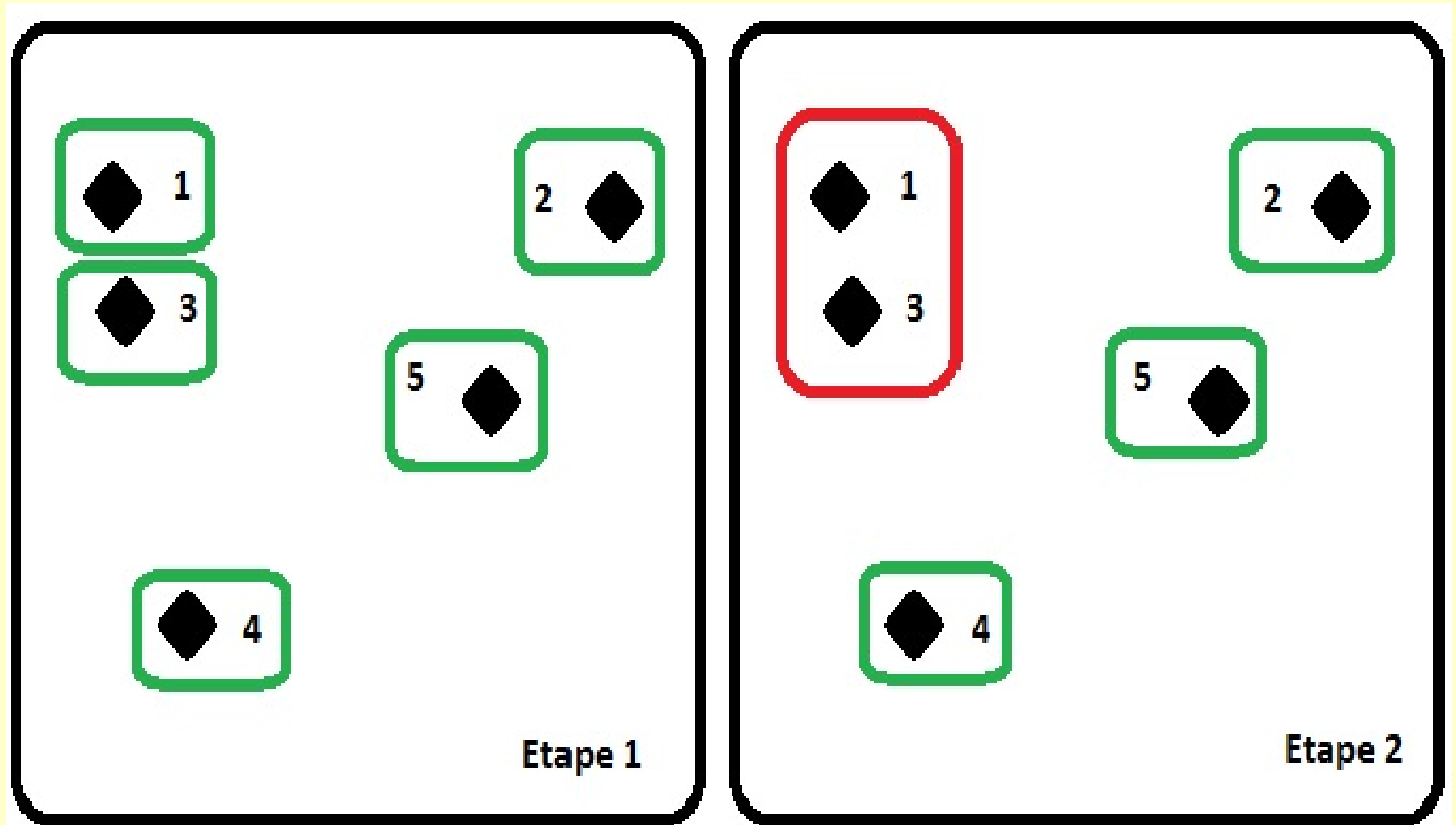
Classification ascendante hiérarchique (CAH)

L'algorithme de CAH fonctionne comme suit:

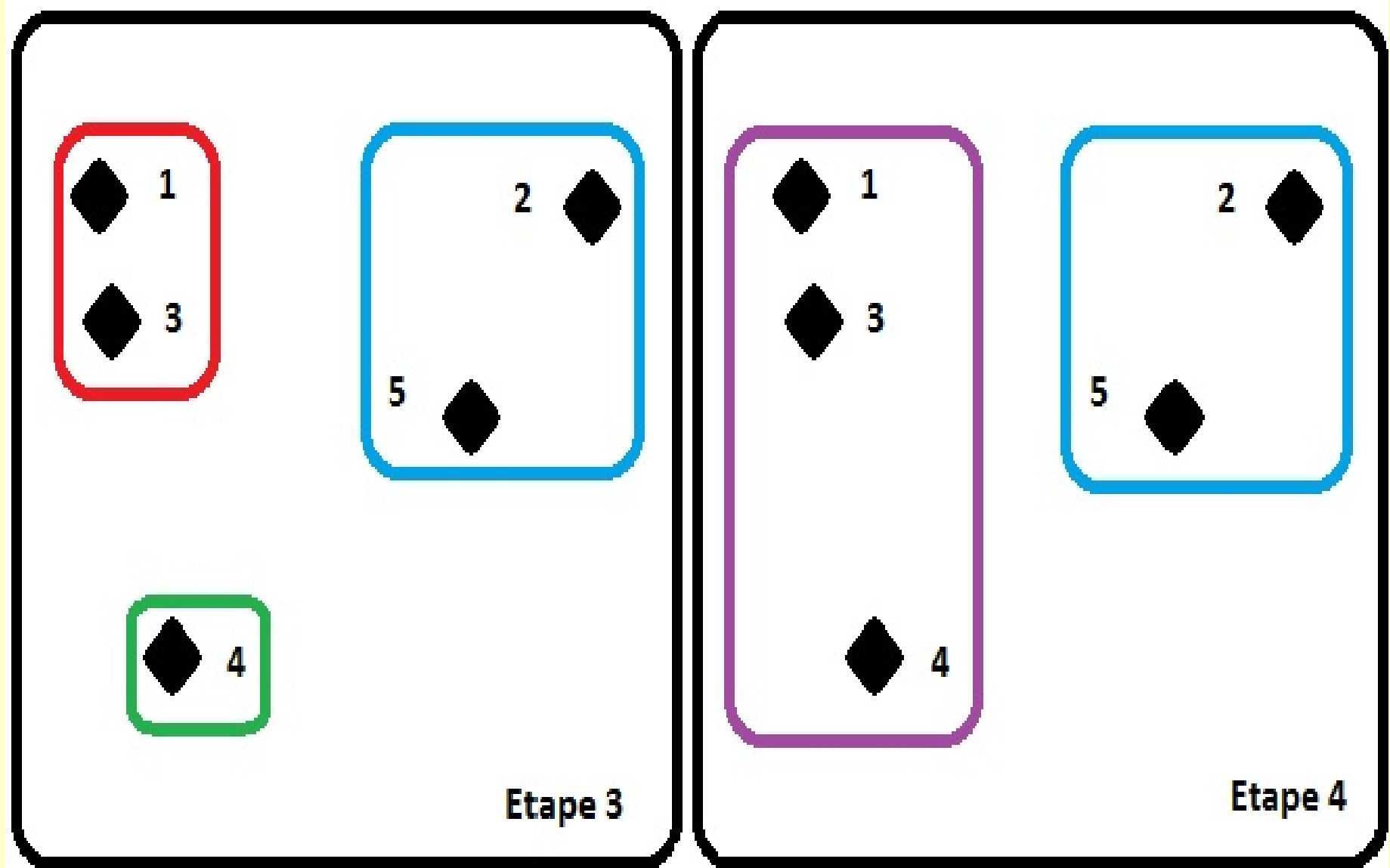
- 1) On choisit un écart (Ward, complet, simple...) et une distance (distance euclidienne, Manhattan, ...). On construit le tableau des écarts pour la partition initiale des n individus de Γ :
 $P_0 = (\{\omega_1\}, \dots, \{\omega_n\})$. Chaque individu constitue un élément.
- 2) On parcourt le tableau des écarts pour identifier le couple d'individus ayant l'écart le plus petit. Le regroupement **de ces deux individus forme un groupe A**. On a donc une partition de Γ de $n - 1$ éléments : A et les $n - 2$ individus restants.
- 3) On calcule le tableau des écarts (distances) entre les $n - 1$ éléments obtenus à l'étape précédente et **on regroupe les deux éléments ayant l'écart le plus petit** (cela peut être deux des $n-2$ individus, ou un individu des $n-2$ individus restants avec A). On a donc une partition de Γ de $n-2$ éléments.
- 4) On itère cette procédure jusqu'à ce qu'il ne reste que deux éléments.
- 5) On regroupe les deux éléments restants.

Classification ascendante hiérarchique (CAH)

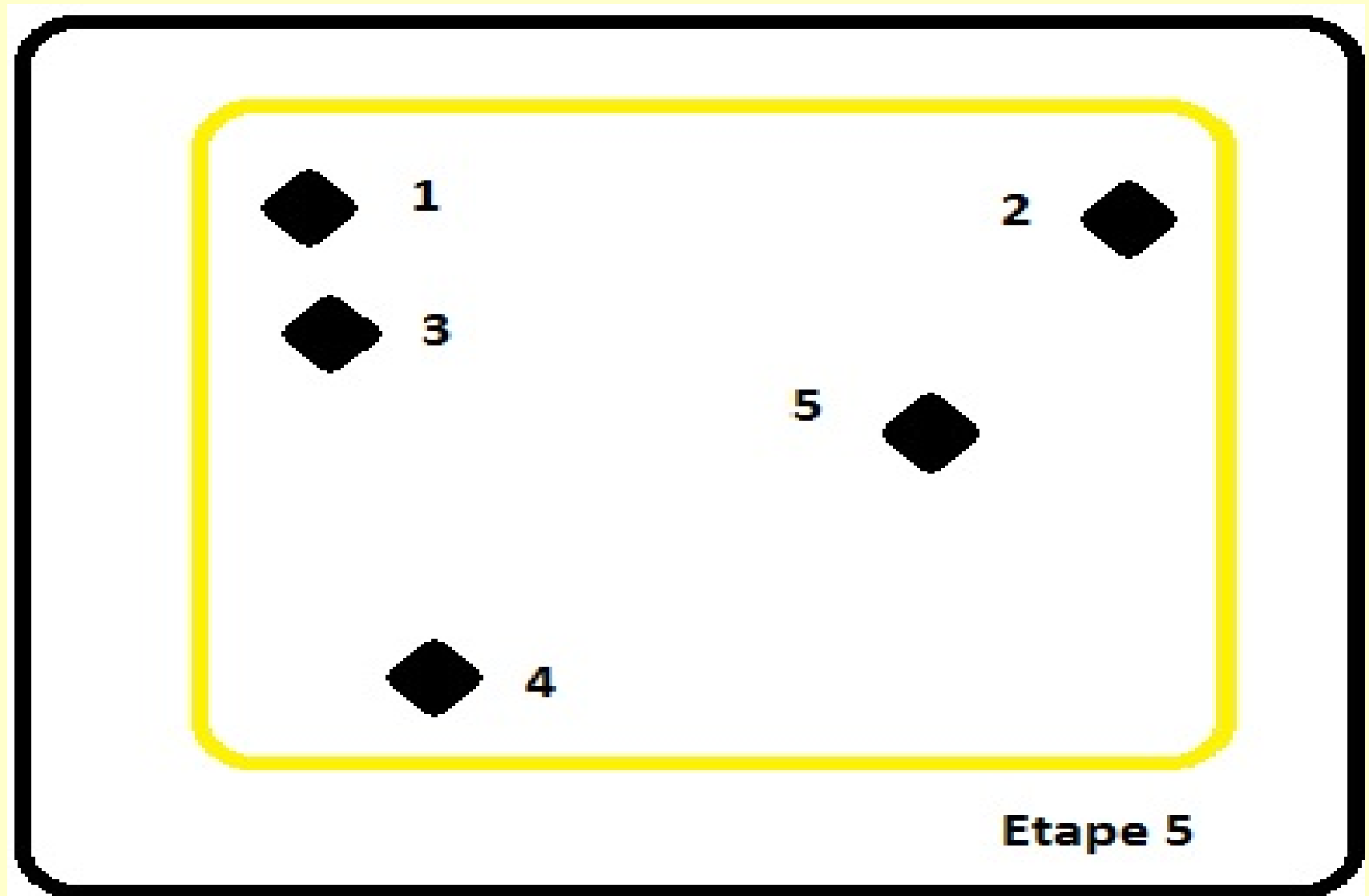
Exemple graphique 1: Etapes de CAH



Classification ascendante hiérarchique (CAH)



Classification ascendante hiérarchique (CAH)



Dendrogramme

Les partitions de Γ faites à chaque étape de l'algorithme de la CAH peuvent se visualiser via un arbre appelé dendrogramme. Sur un axe apparait les individus à regrouper et sur l'autre axe sont indiqués les écarts correspondants aux différents niveaux de regroupement.

Cela se fait graphiquement par le biais de branches et de nœuds.

Une partition naturelle se fait en coupant l'arbre au niveau du plus grand saut de nœuds.

Classification ascendante hiérarchique (CAH)

Exemple: On considère la matrice de données X dans $\mathbb{R}^2$:

On va regrouper les individus avec l'algorithme CAH et la méthode du voisin le plus éloigné munie de la distance euclidienne.

$P_0 = (\{\omega_1\}, \dots, \{\omega_5\})$

Le tableau des écarts associé à P_0 est:

	ω_1	ω_2	ω_3	ω_4	ω_5
ω_1	0	5.85	1.41	3.35	4.47
ω_2	5.85	0	4.60	7.07	1.50
ω_3	1.41	4.60	0	3.20	3.16
ω_4	3.35	7.07	3.20	0	5.59
ω_5	4.47	1.50	3.16	5.59	0

$\begin{pmatrix} 2 & 2 \\ 7.5 & 4 \\ 3 & 3 \\ 0.5 & 5 \\ 6 & 4 \end{pmatrix}$	w_1
	w_2
	w_3
	w_4
	w_5

$$\begin{aligned} e(w_1, w_2) &= \text{racine_carré} ((7.5-2)**2 + (4-2)**2) \\ &= \text{racine_carré} (34.25) \\ &= 5.85 \end{aligned}$$

Les éléments (individus) ω_1 et ω_3 ont l'écart le plus petit : ce sont les éléments les plus proches.

Classification ascendante hiérarchique (CAH)

On les rassemble pour former le groupe : $A = \{\omega_1, \omega_3\}$. On a une nouvelle partition de Γ : $P_1 = (\{\omega_2\}, \{\omega_4\}, \{\omega_5\}, A)$. Le tableau des écarts associé à P_1 est:

On a:

$$e(\omega_2, A) = \max(e(\omega_2, \omega_1), e(\omega_2, \omega_3)) \\ = \max(5.85, 4.60) = 5.85$$

$$e(\omega_4, A) = \max(e(\omega_4, \omega_1), e(\omega_4, \omega_3)) \\ = \max(3.35, 3.20) = 3.35$$

$$e(\omega_5, A) = \max(e(\omega_5, \omega_1), e(\omega_5, \omega_3)) \\ = \max(4.47, 3.16) = 4.47.$$

	ω_2	ω_4	ω_5	A
ω_2	0	7.07	1.50	5.85
ω_4	7.07	0	5.59	3.35
ω_5	1.50	5.59	0	4.47
A	5.85	3.35	4.47	0

Les éléments (individus) ω_2 et ω_5 sont les plus proches. On les rassemble pour former le groupe : **$B = \{\omega_2, \omega_5\}$** .

On a une nouvelle partition de Γ : **$P_2 = (\{\omega_4\}, A, B)$**

Remarque: ci-dessus nous avons utilisé la méthode d'Écart complet (voisin le plus éloigné) pour mesurer l'écart entre 2 groupes. Vous pouvez tester les autres méthodes.

Classification ascendante hiérarchique (CAH)

Le tableau des écarts associé à P2 est:

	ω_4	A	B
ω_4	0	3.35	7.07
A	3.35	0	5.85
B	7.07	5.85	0

On a : $e(B, \omega_4) = \max(e(\omega_2, \omega_4), e(\omega_5, \omega_4)) = \max(7.07, 5.59) = 7.07$
 $e(B, A) = \max(e(\omega_2, A), e(\omega_5, A)) = \max(5.85, 4.47) = 5.85$.

Les éléments ω_4 et A sont les plus proches. On les rassemble pour former le groupe : **$C = \{\omega_4, A\} = \{\omega_1, \omega_3, \omega_4\}$** .

On a une nouvelle partition de Γ : $P_3 = (B, C)$.

Classification ascendante hiérarchique (CAH)

Le tableau des écarts associé à P3 est:

	<i>B</i>	<i>C</i>
<i>B</i>	0	7.07
<i>C</i>	7.07	0

On a: $e(C,B) = \max(e(\omega_4,B), e(A,B)) = \max(7.07, 5.85) = 7.07$.

Il ne reste plus que 2 éléments, B et C ; on les regroupe.

On obtient la partition $P_4 = \{\omega_1, \dots, \omega_5\} = \Gamma$.

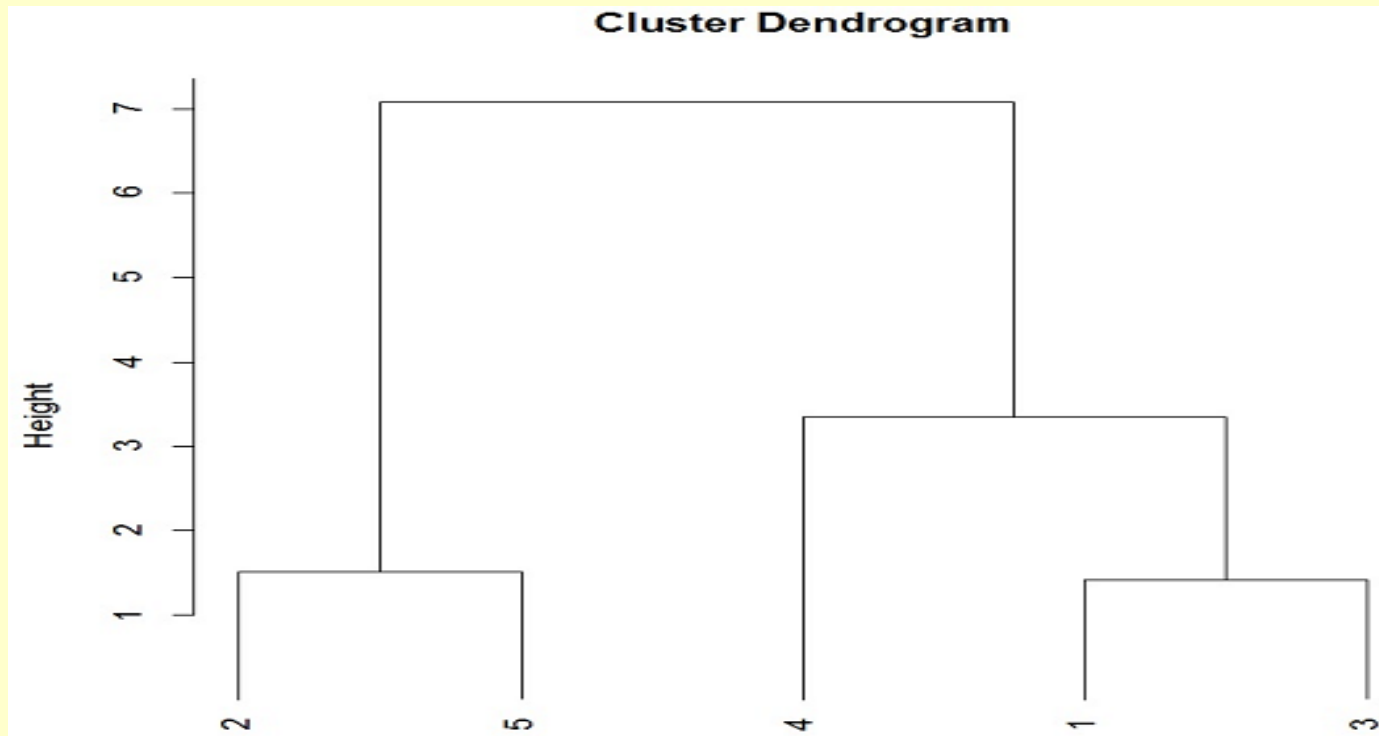
Cela termine l'algorithme de CAH.

Classification ascendante hiérarchique (CAH)

Au final:

- les éléments $\{\omega_1\}$ et $\{\omega_3\}$ ont été regroupés avec un écart de **1.41**,
- les éléments $\{\omega_2\}$ et $\{\omega_5\}$ ont été regroupés avec un écart de **1.50**,
- les éléments $A = \{\omega_1, \omega_3\}$ et $\{\omega_4\}$ ont été regroupés avec un écart de **3.35**,
- les éléments $C = \{\omega_4, A\}$ et $B = \{\omega_2, \omega_5\}$ ont été regroupés avec un écart de **7.07**.

On peut donc construire le dendrogramme associé :



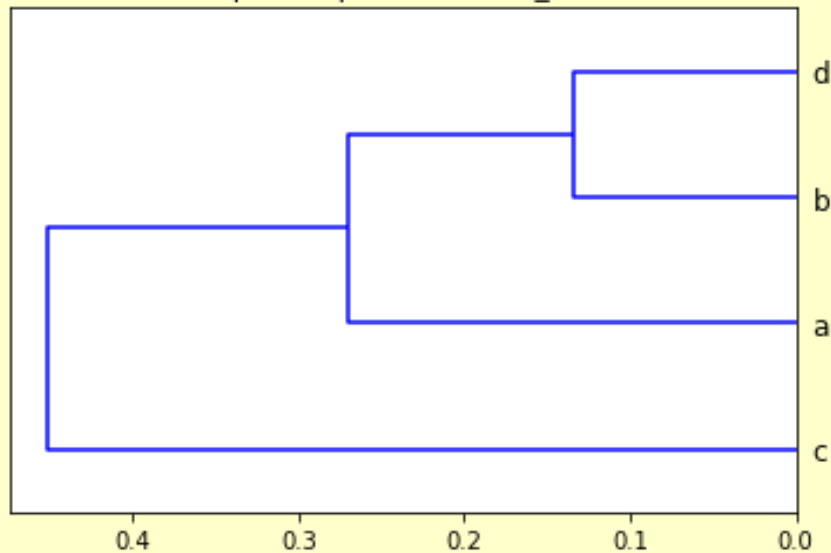
Comme le plus grand saut se situe entre les éléments B et C (on a $7.07 - 3.35 = 3.72$), on propose les deux groupes : B et C.

Classification ascendante hiérarchique (CAH)

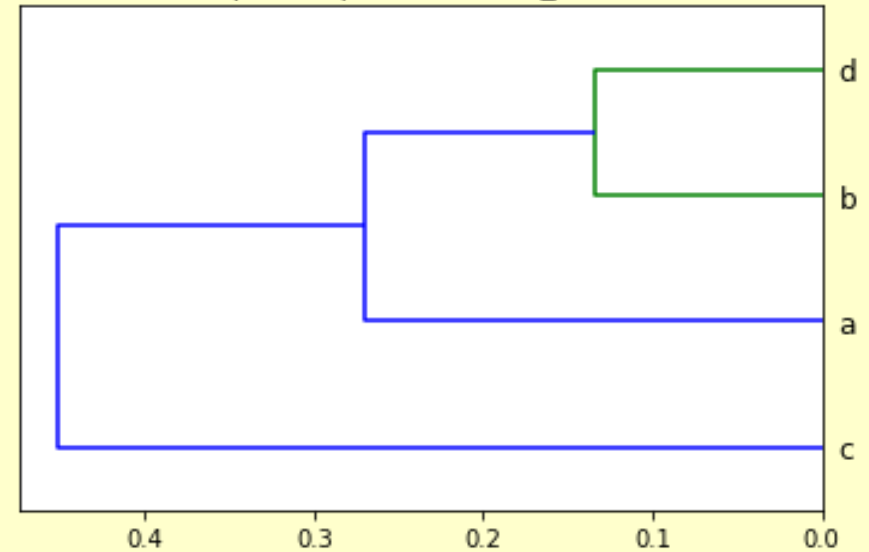
```
import numpy as np
from scipy.cluster.hierarchy import dendrogram, linkage
import matplotlib.pyplot as plt
M = np.array([[1.0, 0.5, 0.0],[0.5, 1.0, -0.5],
              [1.0, -0.5, 0.5],[0.0, 0.5, -0.5]])
Z = linkage(M,method='single',metric='cosine')
M_labels=["a", "b", "c", "d"]
plt.title("CAH-Exemple Simple avec color_threshold=0")
dendrogram(Z,labels=M_labels,orientation='left',color_threshold=0)
plt.show()
plt.title("CAH-Exemple Simple avec color_threshold=0.2")
dendrogram(Z,labels=M_labels,orientation='left',color_threshold=0.2)
plt.show()
plt.title("CAH-Exemple Simple avec color_threshold=0.4")
dendrogram(Z,labels=M_labels,orientation='left',color_threshold=0.4)
plt.show()
```

Classification ascendante hiérarchique (CAH)

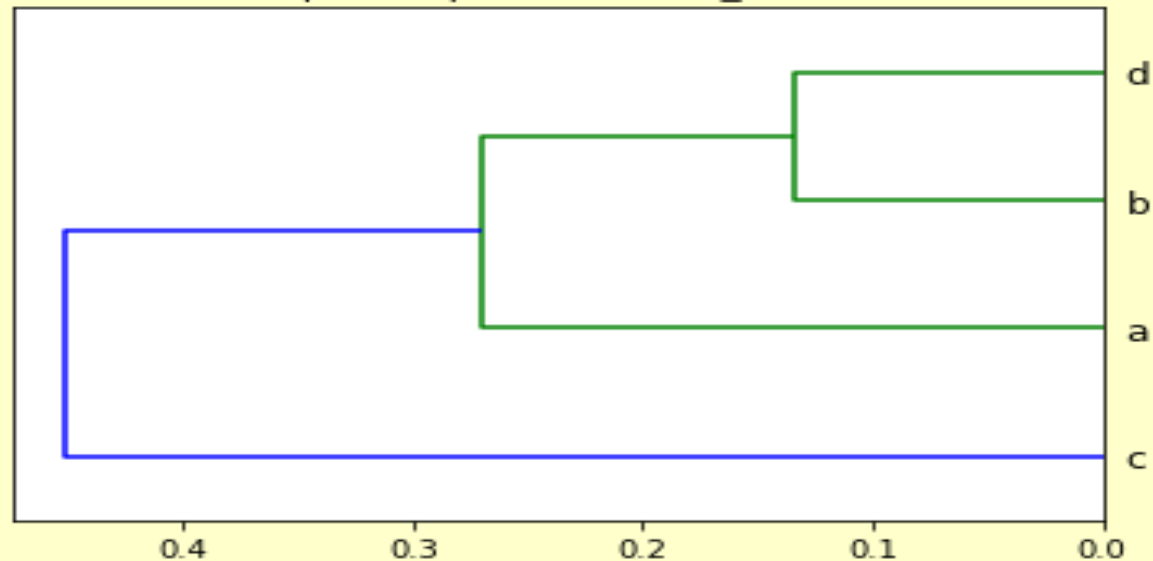
CAH-Exemple Simple avec color_threshold=0



CAH-Exemple Simple avec color_threshold=0.2



CAH-Exemple Simple avec color_threshold=0.4



Classification ascendante hiérarchique (CAH)

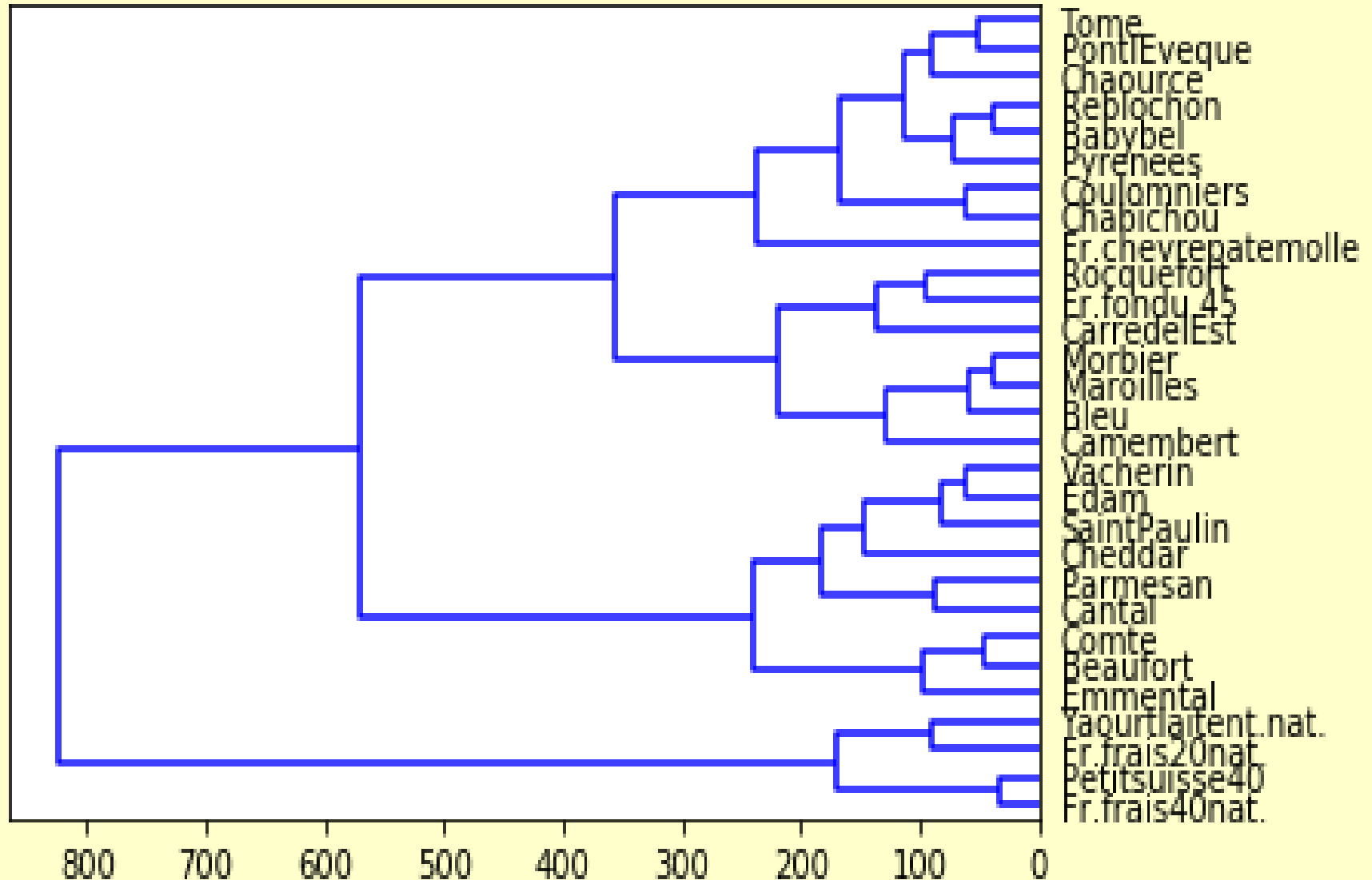
```
import pandas
fromage = pandas.read_table("D:\\fromage.txt",sep="\t",header=0,index_col=0)
print(fromage) ; print(fromage.shape)
from matplotlib import pyplot as plt
from scipy.cluster.hierarchy import dendrogram, linkage
#générer la matrice des liens
Z = linkage(fromage,method='ward',metric='euclidean')#ward-Euclidean
#méthodes de linkage:"single","average","weighted","centroid","ward".
#distances:"euclidean","l1","l2","manhattan", "cosine", ou "precomputed"
#affichage du dendrogramme
dendrogram(Z,labels=fromage.index,orientation='left',color_threshold=0)
plt.title("CAH") ; plt.show()
```

Classification ascendante hiérarchique (CAH)

Fromages	calories		sodium	calcium	lipides	retinol	folates	protéines	
Carre del Est	314	353.5	72.6	26.3	51.6	30.3	21	70	20
Babybel	314	238	209.8	25.1	63.7	6.4	22.6	70	27
Beaufort	401	112	259.4	33.3	54.9	1.2	26.6	120	41
Bleu	342	336	211.1	28.9	37.1	27.5	20.2	90	27
Camembert	264	314	215.9	19.5	103	36.4	23.4	60	20
Cantal	367	256	264	28.8	48.8	5.7	23	90	30
Chabichou	344	192	87.2	27.9	90.1	36.3	19.5	80	36
Chaource	292	276	132.9	25.4	116.4	32.5	17.8	70	25
Cheddar	406	172	182.3	32.5	76.4	4.9	26	110	28
Comte	399	92	220.5	32.4	55.9	1.3	29.2	120	51
Coulomniers	308	222	79.2	25.6	63.6	21.1	20.5	80	13
Edam	327	148	272.2	24.7	65.7	5.5	24.7	80	44
Emmental	378	60	308.2	29.4	56.3	2.4	29.4	110	45
Fr. chevrep	206	160	72.8	18.5	150.5	31	11.1	50	16
Fr. fondu.45	292	390	168.5	24.0	77.4	5.5	16.8	70	20
Fr. frais20nat.	80	41	146.3	3.5	50	20	8.3	10	11
Fr. frais40nat.	115	25	94.8	7.8	64.3	22.6	7	30	10
Maroilles	338	311	236.7	29.1	46.7	3.6	20.4	90	40
Morbier	347	285	219	29.5	57.6	5.8	23.6	80	30

Classification ascendante hiérarchique (CAH)

CAH

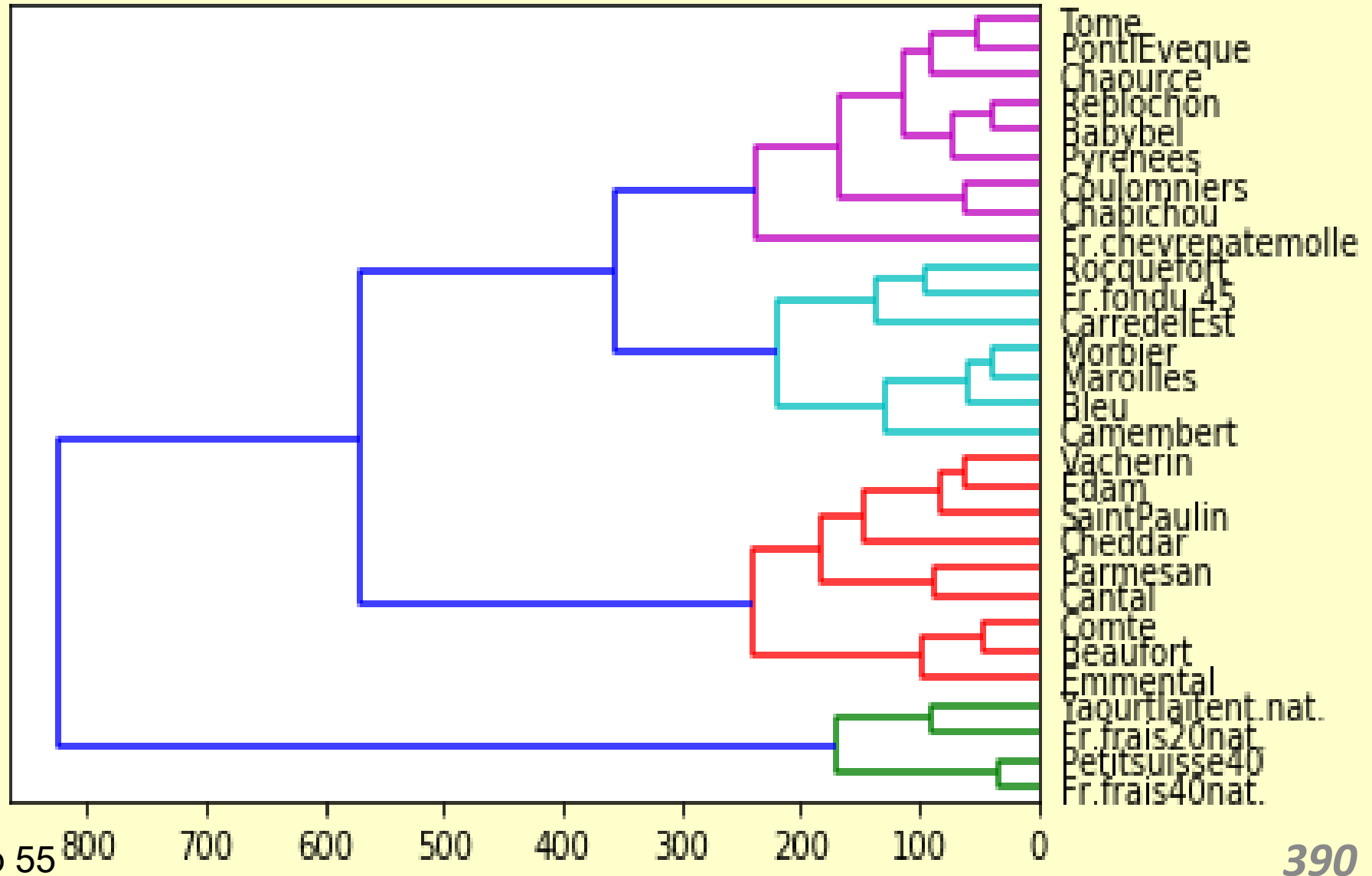


Classification ascendante hiérarchique (CAH)

```
from scipy.cluster.hierarchy import fcluster
#matérialisation des 4 classes (hauteur t = 300)
plt.title('CAH avec matérialisation des 4 classes')
dendrogram(Z,labels=fromage.index,orientation='left',color_threshold=300)
plt.show()
#découpage à la hauteur t = 300 ==> identifiants de 4 groupes obtenus
groupes_cah = fcluster(Z,t=300,criterion='distance')
print(groupes_cah)
#index triés des groupes
import numpy as np
idg = np.argsort(groupes_cah)
#affichage des observations et leurs groupes
print(pandas.DataFrame(fromage.index[idg],groupes_cah[idg]))
plt.show()
```

Classification ascendante hiérarchique (CAH)

CAH avec matérialisation des 4 classes



Classification ascendante hiérarchique (CAH)

[3 4 2 3 3 2 4 4 2 2 4 2 2 4 3 1 1 3 3 2 1 4 4 4 3 2 4 2 1]

Fromages

1	Yaourt	laitent.nat.	3	Morbier
1		Petitsuisse40	3	Maroilles
1		Fr.frais40nat.	3	Carre delEst
1		Fr.frais20nat.	3	Camembert
2		SaintPaulin	3	Bleu
2		Beaufort	3	Fr.fondu.45
2		Cantal	4	Coulomniens
2		Parmesan	4	Chaource
2		Cheddar	4	Chabichou
2		Comte	4	PontlEveque
2		Edam	4	Pyrenees
2		Emmental	4	Reblochon
2		Vacherin	4	Babybel
2		Roquefort	4	Tome
3			4	Fr.chevrepatemolle

Classification ascendante hiérarchique (CAH)

`scipy.cluster.hierarchy.fcluster(Z, t, criterion)`

Z : ndarray, The hierarchical clustering encoded with the matrix returned by the linkage function.

t : float, The threshold to apply when forming flat clusters.

criterion : str, optional, Le critère à utiliser pour former des grappes plates. Cela peut être l'une des valeurs suivantes :

distance, maxclust, monocrit, maxclust_monocrit, inconsistent